

UNIVERSIDADE DO AMAZONAS
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS
CURSO DE JAVA



Curso de Java I





UNIVERSIDADE DO AMAZONAS
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS
CURSO DE JAVA



PREFÁCIO

Esta apostila foi criada para o curso de Java I ministrado no laboratório de Processamento Digital de Sinais da Universidade Federal do Amazonas – UFAM, sendo um curso de extensão, que tem uma carga horária de 60 horas.



UNIVERSIDADE DO AMAZONAS
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS
CURSO DE JAVA



OBJETIVO

Esta apostila visa dar um bom conceito do que é Orientação a Objetos, buscando os conceitos mais profundos como polimorfismo, herança, métodos e outros mais. Teremos alguns conceitos básicos sobre o ambiente de programação java da SUN® como instalação, configuração e etc. Já na programação em Java também veremos processos de threads, I/O, criação de classes, applets e outros mais.



UNIVERSIDADE DO AMAZONAS
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS
CURSO DE JAVA



INFORMAÇÕES ADICIONAIS

AUTOR: LUIZ NAGATA.

MATÉRIA: JAVA I

CARGA HORÁRIA: 60 Hs.

INFORMAÇÕES ADICIONAIS: curso de extensão da Universidade Federal Do Amazonas (UFAM) oferecido pela Pro-Reitoria de Extensão (PROEXT).



UNIVERSIDADE DO AMAZONAS
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS
CURSO DE JAVA



AGRADECIMENTOS

A Deus em primeiro lugar por ter me dado conhecimento e forças para fazer esta apostila.

Ao Professor Doutor Ayres Mardem do Nascimento por pela iniciativa de fazer um curso de Java no Laboratório de Processamento Digital de Sinais na Universidade Federal do Amazonas (UFAM) e por proporcionar um ambiente propício para a criação desta apostila.

Ao professor Divaldo do Instituto de Tecnologia da Amazonia (UTAM) por ter cedido as apostila do curso de Java ministrado no proprio instituto



SUMÁRIO

1. ORIENTAÇÃO A OBJETOS (OO)	1
1.1 O QUE É ORIENTAÇÃO A OBJETOS	1
1.2 CLASSES	2
1.3 OBJETOS	2
1.4 PROPRIEDADES DOS OBJETOS	2
1.5 CONCEITOS CHAVES DE ORIENTAÇÃO A OBJETOS	3
1.5.1 Abstração	3
1.5.2 Herança	3
1.5.2.1 Formas de Herança	4
1.5.2.2 Benefícios e Custos do Uso de Herança	5
1.5.3 Encapsulamento	7
1.5.4 Polimorfismo	8
1.5.4.1 Polimorfismo Ad Hoc	8
1.5.4.1.1 Sobrecarga	8
1.5.4.1.2 Coerção	9
1.5.4.2 Polimorfismo Universal	10
1.5.4.2.1 Polimorfismo Puro	10
1.5.4.2.2 Genéricos	11
1.6 BENEFÍCIOS DA OO	12
2. INTRODUÇÃO A JAVA	13
2.1 CARACTERÍSTICAS DO JAVA	13
2.2 ONDE ENCONTRAR O JAVA E COMO INSTALAR	15
2.3 A TECNOLOGIA JAVA	17
3. PROGRAMAÇÃO EM JAVA	19
3.1 TIPOS PRIMITIVOS	19
3.1.1 Classes wrapper (Classes embrulho)	20
3.1.1.1 Classe Boolean	20
3.1.1.2 Classe Character	21
3.1.1.3 Classes para manipular valores numéricos inteiros	21
3.1.1.4 Classes para manipular valores numéricos reais	22
3.1.1.6 Classe String	22
3.2 ESCOPO	24
3.3 OPERADORES EM JAVA	25
3.3.1 Operadores Básicos	25
3.3.2 Operadores Unários	25
3.3.3 Operadores de Atribuição	26
3.3.4 Operadores Bit-a-Bit	26
3.3.5 Operadores relacionais e condicionais	27
3.3.6 Operador de Type Cast	27
3.4 ESTRUTURAS DE CONTROLE EM JAVA	28
3.4.1 If (ou Se) em Java	28
3.4.2 O comando Switch	28
3.5 LAÇOS DE REPETIÇÃO EM JAVA	29
3.5.1 Laço For	29
3.5.2 Laço While	29
3.5.3 Laço Do ... While	29
3.5.4 Comandos de salto	30
3.5.4.1 O Comando Break	30
3.5.4.2 O Comando Continue	31



UNIVERSIDADE DO AMAZONAS
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS
CURSO DE JAVA



3.5.4.3 O comando Return	31
3.6 COMENTÁRIOS.....	31
3.7 CLASSES EM JAVA	32
3.7.1 Classes.....	32
3.7.2 DEFINIÇÃO DE CLASSES EM JAVA	32
3.7.2.1 Palavra Reservada public	34
3.7.2.2 Palavra Reservada protected	34
3.7.2.3 Palavra Reservada private	34
3.7.2.4 Classes abstratas e finais	35
3.7.3 Atributos	37
3.7.3.1 Membros de Classe static	37
3.7.4 Métodos	38
3.7.6 Construtores	39
3.7.7 Interface.....	40
3.7.7.1 Declaração de interface	40
3.7.7.2 Definição de constantes.....	41
3.7.7.3 Implementação de interfaces	41
3.7.8 O MÉTODO MAIN	42
3.7.8.1 Valor de saída do main.....	42
3.7.8.2 Argumento do main.....	43
3.7.8.3 Assinatura do método main.....	43
3.7.9 Controlando o nível de acesso dos membros das classes.....	44
3.8 OBJETOS EM JAVA	44
3.8.1 Propriedades de objetos	45
3.8.2 Como criar um objeto em Java.....	46
3.8.3 Referência para o objeto	46
3.8.4 Operador this	47
3.8.5 Manipulação de objetos	48
3.9 VETORES EM JAVA	50
3.9.1 Criação de Vetores em Java.....	50
3.9.1.1 Copiando vetores	51
3.9.2 Matrizes em Java.....	52
3.9.2.1 Criação de Matrizes	52
3.9.3 Vetores e Matrizes de Objetos em Java.....	53
3.9.4 Passando Vetores para métodos.....	53
3.10 HERANÇA.....	54
3.10.1 Construção de objetos derivados	56
3.10.2 A palavra-chave super	56
3.10.3 Restrições de acesso	57
3.10.4 Herança e interfaces.....	59
3.10.5 Interfaces.....	59
3.10.5.1 Definindo Interfaces.....	60
3.10.5.2 Implementado interfaces	61
3.11 POLIMORFISMO EM JAVA.....	61
3.11.1 Sobrescrevendo métodos	65
3.11.2 Sobrecarregando métodos.....	66
3.12 PACOTES	67
3.12.1 Pacotes e Diretórios	68
3.12.2 Ocultando Informações	68
3.12.3 Importação de Pacotes	69
3.12.4 Nomeando Pacotes e Unidades de Compilação.....	70
3.13 EXCEÇÕES EM JAVA	70
3.13.1 Capturando e tratando exceções	72
3.13.2 Propagando exceções.....	74
3.13.3 A hierarquia de exceções.....	75
3.13.4 Capturando várias exceções.....	76
3.13.5 Gerando exceções.....	77



UNIVERSIDADE DO AMAZONAS
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS
CURSO DE JAVA



3.13.6 Definição de novas exceções	77
3.13.7 Repassando a exceção	78
3.14 ENTRADA E SAÍDA EM JAVA (I/O)	79
3.14.1 Streams	81
3.14.2 Definição de Arquivos	82
3.14.3 O que são fluxos de dados?	83
3.14.4 Entrada e saída de dados binários	83
3.14.4.1 Entrada binária: InputStream	83
3.14.4.2 Saída de dados binários: OutputStream	84
3.14.5 Entrada e Saída de caracteres	84
3.14.5.1 Entrada de caracteres: Reader	84
3.14.5.2 Saída de caracteres: Writer	84
3.14.6 Transferência de objetos: Serialização	85
3.14.7 Buffers	86
3.14.8 Filtros	87
3.14.9 Comandos de Java para Entrada e Saída	88
3.15 MULTIPROCESSAMENTO EM JAVA (THREADS)	90
3.15.1 Estados de uma Thread	91
3.15.2 Prioridades das threads	92
3.15.3 Sincronização	92
3.15.4 Mensagens	93
3.15.5 A classe Thread e a interface Runnable	93
3.15.6 A thread principal	95
3.15.7 Criando uma thread	97
3.15.7.1 Implementando Runnable	97
3.15.7.3 Derivando de Thread	100
3.15.7.4 Escolhendo o enfoque certo	101
3.15.7.5 Criando várias threads	101
3.15.7.6 Os métodos <code>isAlive()</code> e <code>join()</code>	103
3.15.7.7 Os métodos <code>suspend()</code> e <code>resume()</code>	105
3.15.8 Definindo a prioridade das threads	108
3.15.9 Detalhes sobre a sincronização	111
3.15.9.1 Usando métodos sincronizados	111
3.15.9.2 Outro uso de <code>synchronized</code>	115
3.15.10 Comunicação inter-threads	117
3.15.11 Deadlock	122
3.15.12 Usando multithreading	124
3.16 DESENVOLVIMENTO DE APLICAÇÕES GRÁFICAS EM JAVA	125
3.16.1 Saída na forma Gráfica	126
3.16.1.1 Componentes gráficos	130
3.16.1.1.1 Containers	131
3.16.1.1.1.1 Gerenciadores de Layout	131
3.16.1.1.1.1.1 BorderLayout	132
3.16.1.1.2 Windows (Janelas)	133
3.16.1.1.2.1 Frames	133
3.16.1.1.2.2 Janelas de diálogo	135
3.16.1.1.2.3 Manipuladores de eventos de janela	139
3.16.1.1.2 Componentes de interfaces gráficas com usuários	139
3.16.1.1.2.1 Exibir informação de orientação ao usuário:	140
3.16.1.1.2.1.1 Rótulos(Label)	140
3.16.1.1.2.1.2 Dicas sobre componentes (tooltip)	141
3.16.1.1.2.1.4 Janelas de opções e mensagens	141
3.16.1.1.2.2 Disparar ações selecionadas pelo usuário:	142
3.16.1.1.2.2.1 Botões	142
3.16.1.1.2.2.2 Menus	143
3.16.1.1.2.3 Obter entradas ou escolhas do usuário:	143
3.16.1.1.2.3.1 Entradas de Texto e Senhas (Campos de texto)	143
3.16.1.1.2.3.2 Componentes para seleção	144



UNIVERSIDADE DO AMAZONAS
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS
CURSO DE JAVA



3.16.1.1.2.3.3 Lista de Opções (ou Listas de seleção).....	144
3.16.1.1.2.3.4 Janelas para Diálogo para seleção de arquivo (ou Seleção de Arquivos).....	145
3.16.1.1.2.3.5 Caixa de Diálogo para seleção de cores.....	146
3.16.1.1.2.3.6 Caixa de Janelas de opções e mensagens.....	147
3.16.1.1.3 Eventos.....	148
3.16.1.1.3.1 Eventos relacionados ao mouse.....	149
3.16.1.1.3.1.1 Manipuladores de eventos do mouse.....	150
3.16.1.1.3.2 Eventos de ação.....	153
3.16.1.1.3.3 Eventos de seleção de itens.....	153
3.16.1.1.3.3.1 Caixas de seleção.....	153
3.16.1.1.3.3.2 Seleção em drop-down.....	156
3.16.1.1.3.3.3 Listas.....	158
3.16.1.1.3.4 Eventos de modificação de texto.....	161
3.16.1.1.3.4.1 Componentes de texto.....	161
3.16.1.1.3.4.1.1 Campos de texto.....	162
3.16.1.1.3.4.1.2 Áreas de texto.....	164
3.16.1.1.3.4.1. Manipuladores de eventos de modificação de texto.....	164
3.16.1.1.3.5 Eventos de teclado.....	165
3.16.1.1.3.5.1 Manipuladores de eventos de teclado.....	165
3.16.1.1.3.6 Manipuladores de eventos de ação.....	167
3.16.1.1.3.6.1 Eventos de ação.....	167
3.16.2 Interfaces gráficas com usuários.....	168
3.16.2.1 Aplicações autônomas.....	169
4 APPLETS EM JAVA.....	170
4.1 A ESTRUTURA BÁSICA.....	170
4.2 DECLARAÇÃO DA CLASSE APPLET.....	172
4.3 DECLARAÇÃO DE VARIÁVEIS.....	172
4.4 CRIAÇÃO DE APPLET.....	174
4.5 EXECUÇÃO DE APPLETS.....	175
4.5.1 Passagem de argumentos.....	176
4.5.2 Contexto de execução.....	178
4.5.3 Modelo de segurança Java.....	180
4.6 OUTROS MÉTODOS DA CLASSE APPLET.....	181
4.7 MÉTODOS URL.....	181
4.8 MOSTRANDO IMAGENS DENTRO DE UM APPLET.....	181
4.9 APPLETS: VANTAGENS /DESVANTAGENS.....	182
4.10 RESTRIÇÕES DOS APPLETS.....	182
4.11 PROGRAMAÇÃO ORIENTADA A EVENTOS.....	182
APENDICE A : EXEMPLOS.....	184



UNIVERSIDADE DO AMAZONAS
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS
CURSO DE JAVA



1. ORIENTAÇÃO A OBJETOS (OO)

1.1 O QUE É ORIENTAÇÃO A OBJETOS

Não há uma definição precisa sobre o que é OO pois muitos autores dão uma definição diferente sobre o que seja a OO. Uma das definições de objetos é que são instâncias de classes, que contêm dados e procedimentos que juntos representam um objeto do mundo real com todas as suas características, apesar de toda a complexidade da OO ela trás consigo a facilidade de reuso e manutenção do código.

Como **[Martin]** diz:

“Um objeto é qualquer coisa, real ou abstrata, a respeito da qual armazenamos os dados e os métodos que o manipulam.”

“À especificação da OO permite-nos analisar qualquer área da realidade humana.”

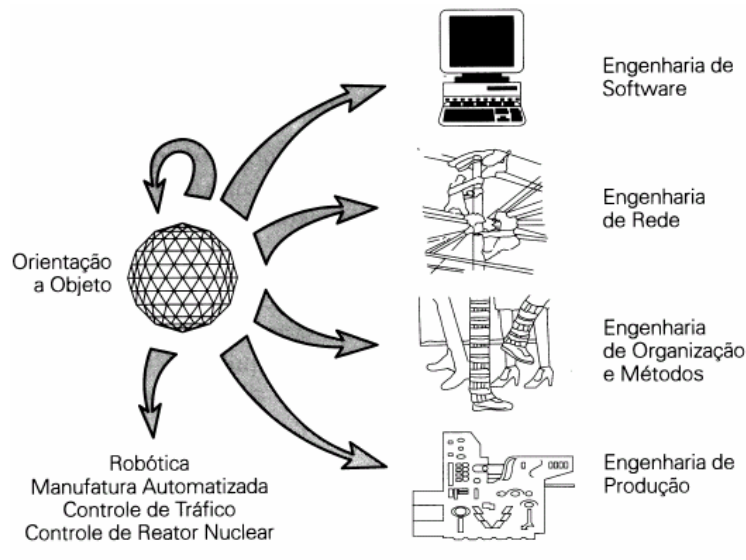


FIGURA 1: ÁREAS EM QUE PODEM SER APLICADAS A ORIENTAÇÃO A OBJETO

Atributo ou dados são as informações manipuladas pelo mundo real.

Uma operação é um processo que pode ser solicitado como uma unidade.

Um método é a especificação de uma operação.

Em outras palavras, a operação é o tipo de serviço solicitado, e o método é o código de programação.

Ou seja OBJETO = ATRIBUTO(ou DADOS) + OPERAÇÕES (MÉTODO ou CÓDIGO)



1.5 CONCEITOS CHAVES DE ORIENTAÇÃO A OBJETOS

1.5.1 Abstração

Abstração consiste de focalizar nos aspectos essenciais inerentes a uma entidade e ignorar propriedades “acidentais”. Em termos de desenvolvimento de sistemas, isto significa concentrar-se no que um objeto é e faz antes de se decidir como ele será implementado. O uso de abstração preserva a liberdade para tomar decisões de desenvolvimento ou de implementação apenas quando há um melhor entendimento do problema a ser resolvido. Muitas linguagens de programação modernas suportam o conceito de abstração de dados; porém, o uso de abstração juntamente com polimorfismo e herança, como suportado em orientação a objetos, é um mecanismo muito mais poderoso.

O uso apropriado de abstração permite que um mesmo modelo conceitual (orientação a objetos) seja utilizado para todas as fases de desenvolvimento de um sistema, desde sua análise até sua documentação. Segundo [Martin] : “Abstração é o ato ou resultado de remover certas distinções entre objetos de forma que possamos ver as características comuns. O resultado deste processo de abstração é um conceito (ou tipo de objeto).”

1.5.2 Herança

O conceito de Herança está ligado a reutilização de classes com outros fins, adaptando a classe a necessidade do problema. A mesma é uma forma de generalização ou especialização da classe. A herança nada mais é do que criar classes filhas, derivadas de outras classes, sendo que estas classes filhas herdam todas as características das classes que foram derivadas.

Segundo [Martin]: “A herança é uma implementação da generalização. Ela permite que todas as características de um classe de OOPL estejam fisicamente disponíveis ou sejam reusáveis por suas subclasses. A herança de classe-objeto permite que as instâncias de uma classe herdem os valores default. A herança objeto-objeto, também denominada delegação, transfere o estado de um objeto para outro.”

Generalização e herança são abstrações que permitem modelar aspectos semelhantes entre classes, preservando suas diferenças.



1.5.2.1 Formas de Herança

Herança é usada de varias formas. Vamos examinar algumas das formas mais comuns:

Especialização: Provavelmente, o uso mais comum de Herança é por especialização. Neste uso, a classe nova é uma forma especializada da classe pai, mas satisfaz as especificações da classe pai em todos os aspectos relevantes. Por exemplo, uma classe Janela fornece as operações gerais de janela (mover, redimensionar, transformar em ícone e assim por diante). Uma subclasse especializada JanelaTexto herda as operações de janela e adiciona facilidades que permitem a exibição e edição de texto. Devido ao fato que JanelaTexto satisfaz todas as propriedades esperadas de uma janela em geral, reconhecemos esta situação como um exemplo de especialização.

Generalização: Usar Herança por generalização é, de certa forma, o oposto de especialização. Aqui, uma subclasse estende o comportamento da classe pai para criar um tipo mais geral de objeto. Generalização é frequentemente aplicada quando construímos uma base de classes existentes que não desejamos modificar ou não podemos modificar. Como exemplo, considere um sistema gráfico no qual uma classe Janela tenha sido definida para exibir desenhos com um fundo preto-e-branco. Nós poderíamos criar uma subclasse JanelaColorida que permitisse uma cor de fundo diferente de preto-e-branco. Isto poderia ser feito com a adição de um campo para armazenar a cor de fundo e por sobrescrever o código herdado de Janela que especifica a cor de fundo.

Limitação: Limitação ocorre quando o comportamento da subclasse é menor ou mais restritivo do que o comportamento da classe pai. Assim como generalização, limitação ocorre mais frequentemente quando um programador está construindo sobre uma base de classes existentes que não deveria, ou não pode, ser modificada. Por exemplo, uma biblioteca existente de classes oferece uma fila de duas extremidades (ou uma estrutura de dados Deque). Elementos podem ser adicionados ou removidos de qualquer uma das extremidades, mas o programador deseja escrever uma classe Pilha, que possui a propriedade que os elementos podem ser adicionados ou removidos de apenas uma extremidade da pilha. Então, o programador pode definir uma classe Pilha como uma subclasse da classe Deque e modificar ou sobrescrever métodos existentes e indesejáveis de modo que eles produzam uma mensagem de erro se forem usados. Estes métodos sobrescrevem métodos existentes e eliminam a funcionalidade deles, o que caracteriza a Herança por limitação.

Especificação: Um uso frequente de Herança é para garantir que classes mantenham uma certa interface comum, isto é, implementem os mesmos métodos. A classe pai pode ser uma combinação de operações implementadas e operações que são deixadas para ser implementadas pelas classes filhas. Frequentemente, não há mudança de interface de nenhuma ordem entre a classe pai e a classe filha: a classe filha meramente implementa o comportamento descrito, mas não implementado, pela classe pai. Especificação é de fato um caso especial de especialização, exceto que as subclasses não são refinamentos de um tipo existente, mas sim realizações de uma especificação abstrata e incompleta. Em tais casos, a classe pai é algumas vezes conhecida como classe de especificação abstrata.



Combinação: Uma situação comum é uma subclasse representar uma combinação de características de duas ou mais classes pai. Um monitor de ensino pode possuir características tanto de um professor quanto de um estudante e pode, portanto, comportar-se como ambos. A capacidade de uma classe herdar de duas ou mais classes pais é conhecida como Herança múltipla. Esta forma de Herança será objeto de estudo de uma aula futura devido a sua sutileza e complexidade.

Contração: é uma variante de herança onde a subclasse elimina métodos da superclasse com o objetivo de criar uma "classe mais simples". A eliminação pode ocorrer pela redefinição de métodos com corpo vazio. O problema com este mecanismo é que ele viola o princípio da substituição, pois a subclasse já não pode mais ser utilizada em todos os pontos onde a superclasse poderia ser utilizada.

Se a contração parece ser uma solução adequada em uma hierarquia de classes, provavelmente a hierarquia deve ser re-analisada para detecção de inconsistências (problema pássaros-pinguins). De modo geral, o mecanismo de contração deve ser evitado.

1.5.2.2 Benefícios e Custos do Uso de Herança

Alguns dos benefícios alcançados com o uso de herança são:

Reusabilidade de código: Quando comportamento é herdado de uma outra classe, o código que fornece aquele comportamento não precisa ser reescrito. Isto parece óbvio, mas as implicações são importantes. Muitos programadores gastam muito do tempo deles reescrevendo código que eles escreveram muitas vezes antes. Por exemplo, para encontrar um padrão em uma cadeia ou inserir um novo elemento em uma tabela. Com técnicas de orientação a objetos, estas funções podem ser escritas apenas uma vez e reutilizadas.

Compartilhamento de código: Ocorre em muitos níveis com técnicas orientadas a objetos. Uma forma de compartilhamento se dá quando muitos usuários ou projetos podem usar as mesmas classes. Uma outra ocorre quando duas ou mais classes desenvolvidas por um único programador como parte de um projeto herda de uma única classe pai. Quando isto acontece, dois ou mais tipos de objetos compartilharão o código que eles herdaram.

Consistência de interface: Quando duas ou mais classes herdam da mesma superclasse, podemos estar certos de que o comportamento que elas herdaram será o mesmo em todas as classes. Logo, é mais fácil garantir que interfaces para objetos semelhantes sejam, de fato, semelhantes e que o usuário não se depare com um conjunto confuso de objetos que são quase os mesmos mas se comportam e interagem de forma bem diferente.

Componentes de software: Herança permite que programadores construam componentes de software reutilizáveis. O objetivo é permitir o desenvolvimento de aplicações novas que requeiram pouca ou nenhuma condição. Várias bibliotecas de classes reutilizáveis já se encontram disponíveis comercialmente e podemos esperar muito mais com o passar do tempo.



Prototipação rápida: Quando um sistema de software é grandemente construído com base em componentes reutilizáveis, o tempo de desenvolvimento pode ser concentrado em entender a parte nova do sistema. Logo, sistemas de software podem ser gerados mais facilmente e mais rapidamente, levando a um estilo de programação conhecido como prototipação rápida ou programação exploratória.

Ocultamento de informação: Um programador que reutiliza um componente de software necessita apenas entender a natureza do componente e sua interface. Não há a necessidade do programador ter informação detalhada a respeito de fatos como, por exemplo, as técnicas usadas para implementar o componente. Logo, a interconexão entre sistemas de software é reduzida e, conseqüentemente, a complexidade também é reduzida.

Embora os benefícios de herança em programação orientada a objetos sejam grandes, herança, assim como outros mecanismos, não possui custo zero. Por esta razão, devemos considerar o custo de técnicas orientadas a objetos e, em particular, o custo de herança:

Velocidade de execução: Dificilmente, ferramentas de software de propósito geral são tão rápidas quanto aquelas desenvolvidas cuidadosamente para um propósito específico. Logo, métodos herdados, que devem lidar com subclasses arbitrárias são frequentemente mais lentos do que código especializado. Ainda assim, a preocupação com eficiência é geralmente desnecessária. Primeiro, a diferença é frequentemente pequena. Segundo, a redução em velocidade de execução pode ser balanceada por um aumento da velocidade de desenvolvimento de software. Finalmente, a maioria dos programadores não possui a mínima idéia de como o tempo de execução está sendo usado no programa. É muito melhor desenvolver um sistema que funcione, monitorá-lo para descobrir onde o tempo de execução está sendo usado e melhorar aquelas seções, do que gastar uma quantidade desordenada de tempo se preocupando com eficiência no início do projeto.

Tamanho do programa: O uso de qualquer biblioteca de software em geral acarreta o aumento de tamanho do programa, o que não acontece com sistemas construídos através de um projeto específico. Embora este gasto possa ser substancial, quando o custo de memória for reduzido a níveis insignificantes. Conter custos de desenvolvimento e produzir código de alta qualidade e livre de erro são preocupações que se tornaram muito mais importantes do que limitar o tamanho dos programas.

Overhead de envio de mensagens: Muito do que tem sido feito do fato que envio de mensagens é por natureza uma operação mais custosa do que chamada de procedimento. Assim como velocidade de execução, entretanto, a preocupação neste caso também é supervalorizada, pois a diferença nos custos de envio de mensagens e chamada de procedimento é marginal. Talvez, duas ou três instruções adicionais de linguagem de montagem e um gasto adicional de 10 por cento do tempo total. Este custo, como muitos outros, pode ser balanceado pelos benefícios das técnicas de orientação a objetos.

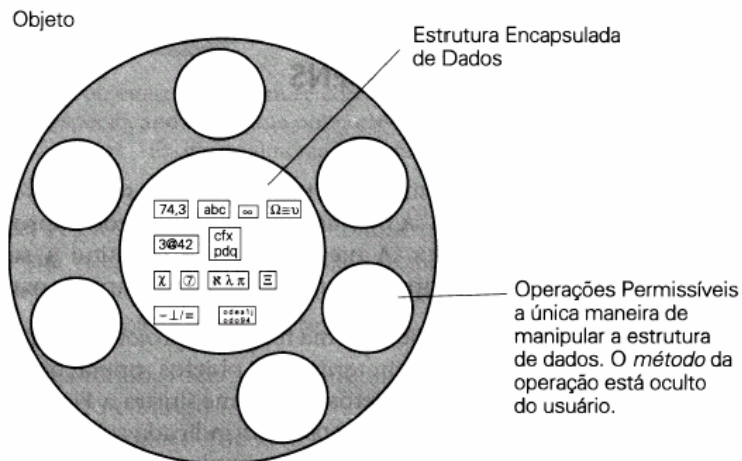
Complexidade do programa: Embora a programação orientada a objetos seja tida como uma solução para a complexidade do software, o uso demasiado de herança pode simplesmente substituir uma forma de complexidade por outra. Entender o fluxo de controle de um programa que utiliza herança pode requerer várias varreduras no grafo de herança. Isto é conhecido como o problema do iô - iô.

1.5.3 Encapsulamento

Encapsulação é o processo no qual se baseia a formação de objetos. Um objeto encapsulado é muito similar ao conceito de Tipos Abstratos de Dados, suportado em outras linguagens. Encapsulação envolve o uso de classes, e sem encapsulação não há programação orientada a objetos. Há mais em programação orientada a objetos do que encapsulação, mas este é um princípio fundamental.

A idéia por trás de encapsulação é proteger a informação -- código, dados -- por trás de uma barreira, de forma a evitar que programadores usando aquele objeto venham porventura corromper esta informação devido a pequenos enganos a que todos estão sujeitos. Este é o mecanismo conhecido como ocultamento da informação. Outra contribuição é a possibilidade de isolar erros em seções pequenas de código, o que facilita encontrá-los e corrigi-los.

Segundo **[Martin]** : “É um envoltório protetor que oculta os detalhes de implementação de um objeto, tornando seus dados acessíveis somente às operações lá colocadas para mediar seu acesso. Frequentemente o termo é usado de forma intercambiável com ocultação de informações.”



A figura 3 também mostra o que é encapsulamento.



1.5.4 Polimorfismo

O termo polimórfico tem origem grega e significa literalmente "muitas formas". Em linguagens de programação um objeto polimórfico é qualquer entidade como uma variável ou argumento de função que pode armazenar valores de tipos diferentes durante o curso de execução de um programa. Funções polimórficas são aquelas que possuem argumentos polimórficos.

Em linguagens orientadas a objetos, polimorfismo é um resultado natural do relacionamento, é um e dos mecanismos de passagem de mensagem, herança e substituição. Um dos pontos fortes da abordagem da programação orientada a objetos é que estes dispositivos podem ser combinados de várias formas, produzindo um número de técnicas para compartilhar e reusar código.

Há vários tipos de polimorfismo, os quais podem ser classificados em duas categorias, denominadas ad hoc e universal. Na primeira, não existe um modo único e sistemático de determinar o tipo de resultado de uma função em termos dos tipos dos seus argumentos de entrada. Esta é uma forma limitada de polimorfismo, pois trabalha com um número limitado de tipos. Na segunda, temos uma forma sistemática e única de trabalhar com um número potencialmente infinito de tipos. Sobrecarga e coerção são formas de polimorfismo ad hoc, enquanto polimorfismo puro e polimorfismo parametrizado ou genérico são formas de polimorfismo universal.

1.5.4.1 Polimorfismo Ad Hoc

1.5.4.1.1 SOBRECARGA

O polimorfismo de sobrecarga ocorre quando um nome de função está associado a mais de dois corpos de função, ou seja, quando um nome de uma função for usado mais de uma vez com diferentes tipos de parâmetro. Por exemplo, uma função soma pode ser sobrecarregada para operar com dois parâmetros inteiros e com dois parâmetros reais. Isto é, definimos duas funções soma com tipos de parâmetros de entrada diferentes. A informação sobre o tipo de parâmetro é utilizada para selecionar a função apropriada.



1.5.4.1.2 COERÇÃO

Algumas linguagens de programação permitem que haja uma conversão implícita entre certos tipos de dados. Estas conversões implícitas são realizadas automaticamente pelo compilador e são denominadas de coerção. Em geral, coerções são limitadas a situações onde não há perda de informações, como na conversão de um valor inteiro para o valor real correspondente.

Coerção proporciona um meio de contornar a rigidez de tipos monomórficos. Se um contexto particular demanda um determinado tipo e um tipo diferente é passado, então a linguagem verifica se existe uma coerção adequada. Por exemplo, se o método `exibe` com parâmetro inteiro, da classe `A` definida na seleção anterior, não existisse, ainda seria possível responder à mensagem.

```
a.exibe(10);
```

pois o método `exibe` com parâmetro real seria invocado e o argumento inteiro 10 seria coargido para o real 10:0, desde que esta conversão é sem perdas e realizada automaticamente pelo compilador Java.

Agora, considere a definição em Java, dada a seguir, de uma classe `Soma`:

```
public class Soma {  
    public int soma (int x, int y)  
    {return x + y;}  
    public double soma (double x, double y)  
    {return x + y;}  
}
```

Então, na sequência de envio de mensagens a seguir

```
Soma op = Soma();  
op.soma(7.5,4.3);  
op.soma(7.5,4);  
op.soma(7,4.3);  
op.soma(7,4);
```

temos que o método `soma` com parâmetros reais será invocado para os três primeiros envios da mensagem `soma`. Aqui, notamos a presença tanto de sobrecarga quanto de coerção, pois no segundo e no terceiro envios da mensagem `soma`, o valor inteiro foi coargido para o real correspondente. Já no primeiro e no último envio da mensagem `soma`, temos apenas sobrecarga.



Uma situação embaraçosa pode ocorrer, no entanto, se tivermos uma situação como a seguinte:

```
public class SomaPai {  
    public int soma (int x, int y)  
    { return x + y;}  
}  
  
public class SomaFilha  
    public double soma (double x, double y)  
    {return x + y;}  
}  
...
```

```
SomaFilha op = SomaFilha();  
op.soma(7,4);
```

Aqui, talvez fôssemos levados a imaginar que o método soma a ser invocado seria aquele da classe pai, SomaPai. Acontece que na busca por um método para acoplar a mensagem, primeiro examina-se os métodos de SomaFilha. Apesar de não haver um método soma com os dois argumentos inteiros em SomaFilha, uma coerção é possível, tal que o método soma de SomaFilha é chamado.

1.5.4.2 Polimorfismo Universal

1.5.4.2.1 POLIMORFISMO PURO

Muitos autores reservam o termo polimorfismo puro (ou de inclusão) para situações onde uma função pode ser usada com uma variedade de argumentos. Note que, neste caso, os parâmetros da função são polimórficos e não o nome da função, como é o caso com sobrecarga.

A capacidade para criar funções polimórficas é uma das técnicas mais potentes da programação orientada a objetos, pois esta técnica permite que código seja escrito apenas uma vez, em um nível mais alto de abstração, e seja ajustado para se adequar a uma variedade de situações. Usualmente, o programador efetua esse ajuste através do envio de outras mensagens para o receptor. Estas mensagens subsequentes não estão frequentemente relacionadas com a classe no nível do método polimórfico; ao invés disso, são métodos virtuais definidos nas classes de nível inferior.



1.5.4.2.2 GENÉRICOS

Uma outra forma de polimorfismo é fornecida pela facilidade conhecida como um genérico (ou template, em C++). Genéricos oferecem uma maneira de parametrizar uma classe ou uma função através do uso de um tipo, assim como parâmetros normais para uma função oferecem uma forma de definir um algoritmo abstrato sem identificar valores específicos. Para ilustrar este conceito, façamos uma referência a um problema com linguagens estruturadas fortemente tipadas, que é a impossibilidade de criar um tipo tal como Lista Ligada de X, onde X é um tipo desconhecido. Genéricos oferecem esta habilidade.

Com genéricos, uma variável é definida como um parâmetro de tipo. este parâmetro pode então ser usado dentro da definição de classe como se ele fosse um tipo, embora nenhuma propriedade do tipo seja conhecida quando a descrição da classe está sendo analisada pelo compilador. em algum ponto mais adiante, o parâmetro de tipo é igualado a um tipo específico e um valor pode então ser declarado. por exemplo, uma lista ligada pode ser declarada em c++ da seguinte forma:

```
template <class T>
class Lista {
public:
void inclui(T);
T primElemento();
T valor;
Lista<T>* proxElemento;
};
```

Neste exemplo, T está sendo usado como um parâmetro de tipo. Cada instância da classe Lista armazena um valor de tipo T e um apontador para o próximo elemento. A função-membro inclui adiciona um novo membro à lista, enquanto o valor da cabeça da lista é retornado pela função primElemento.

Para criar uma instância da classe, o usuário deve fornecer um valor de tipo para o parâmetro T. O trecho a seguir declara uma lista de valores inteiros e uma lista de valores de ponto flutuante:

```
Lista<int> aLista;
Lista<double> bLista;
```



1.6 BENEFÍCIOS DA OO

Listaremos a seguir os principais ganhos, para o desenvolvedor, caso este troque a Metodologia Tradicional pela Orientada ao Objeto.

- **Exatidão:** Devido a característica do desenvolvimento estruturado, onde elaborase um projeto e depois faz-se os programas, podemos ter no final um sistema que não atenda perfeitamente seus objetivos depois de implementado. No desenvolvimento OOP, devido ao fato deste ser feito de maneira quase que interativa com o usuário, este risco é significativamente diminuído. A pouca quantidade de código programável também reduz os problemas inerentes as mudanças das especificações durante o desenvolvimento do projeto.
- **Potencialidade:** Definimos potencialidade a forma como o programa reage aos erros imprevistos como uma falha na impressora, ou a um disco cheio. Tanto maior for a potencialidade, maior a capacidade do programa em causar o menor estrago possível aos dados e evitar uma saída drástica do sistema.
- **Extensibilidade:** Dizemos que quanto maior for a extensibilidade do software, maior será sua capacidade em adequar-se as especificações definidas pelos analistas.
- **Reutilização:** A capacidade de se otimizar a produtividade do programador depende diretamente da maneira como o software disponibiliza a reutilização do código gerado. De fato, a maioria dos programadores profissionais, já reutiliza código anteriormente gerado, porém a perfeita reutilização consiste na utilização COMPLETA de um código gerado para algum sistema SEM qualquer outra adaptação prévia.



2. INTRODUÇÃO A JAVA

2.1 CARACTERÍSTICAS DO JAVA

Parecida com C, C++:

Java tem a aparência de C ou de C++, embora a filosofia da linguagem seja diferente. Por este motivo estaremos frequentemente fazendo comparações alguma destas linguagens. O leitor que programa em qualquer uma delas, ou em uma linguagem orientada a objetos, se sentirá mais a vontade e se tornará um bom programador Java em menos tempo.

Java também possui características herdadas de muitas outras linguagens de programação: Objective-C, Smalltalk, Eiffel, Modula-3, etc. Muitas das características desta linguagem não são totalmente novas. Java é uma feliz união de tecnologias testadas por vários centros de pesquisa e desenvolvimento de software.

Compilada:

Um programa em Java é compilado para o chamado “byte-code”, que é próximo as instruções de máquina, mas não de uma máquina real. O “byte-code” é um código de uma máquina virtual idealizada pelos criadores da linguagem. Por isso Java pode ser mais rápida do que se fosse simplesmente interpretada.

Portável:

Java foi criada para ser portátil. O “byte-code” gerado pelo compilador para a sua aplicação específica pode ser transportado entre plataformas distintas que suportam Java (Solaris 2.3®, Windows-NT®, Windows-95®, Mac/Os etc) . Não é necessário recompilar um programa para que ele rode numa máquina e sistema diferente, ao contrário do que acontece por exemplo com programas escritos em C e outras linguagens. Esta portabilidade é importante para a criação de aplicações para a heterogênea internet. Em Java um inteiro por exemplo, tem sempre 32 bits, independentemente da arquitetura. O próprio compilador Java é escrito em Java, de modo que ele é portátil para qualquer sistema que possua o interpretador de “byte-codes”. Um exemplo de programa escrito em Java é o browser hotjava.

Orientada a Objetos:

A portabilidade é uma das características que se inclui nos objetivos almejados por uma linguagem orientada a objetos. Em Java ela foi obtida de maneira inovadora com relação ao grupo atual de linguagens orientadas a objetos.

Java suporta herança, mas não herança múltipla. A ausência de herança múltipla pode ser compensada pelo uso de herança e interfaces, onde uma classe herda o com-



portamento de sua superclasse além de oferecer uma implementação para uma ou mais interfaces.

Java permite a criação de classes abstratas. Outra característica importante em linguagens orientadas a objetos é a segurança. Dada a sua importância o tópico foi escrito a parte.

Segura:

A presença de coleta automática de lixo, evita erros comuns que os programadores cometem quando são obrigados a gerenciar diretamente a memória (C, C++, Pascal). A eliminação do uso de ponteiros, em favor do uso de vetores, objetos e outras estruturas substitutivas traz benefícios em termos de segurança. O programador é proibido de obter acesso a memória que não pertence ao seu programa, além de não ter chances de cometer erros comuns tais como “reference aliasing” e uso indevido de aritmética de ponteiros. Estas medidas são particularmente úteis quando pensarmos em aplicações comerciais desenvolvidas para a internet.

Ser “strongly typed” também é uma vantagem em termos de segurança, que está aliada a eliminação de conversões implícitas de tipos de C++.

A presença de mecanismos de tratamento de exceções torna as aplicações mais robustas, não permitindo que elas abortem, mesmo quando rodando sob condições anormais. O tratamento de exceções será útil na segunda parte para modelar situações tais como falhas de transmissão e formatos incompatíveis de arquivos.

Suporta concorrência:

A linguagem permite a criação de maneira fácil, de vários “threads” de execução.

Este tópico será útil quando você estudar animações, e é particularmente poderoso nos ambientes em que aplicações Java são suportadas, ambientes estes que geralmente podem mapear os threads da linguagem em processamento paralelo real.

Eficiente:

Como Java foi criada para ser usada em computadores pequenos, ela exige pouco espaço, pouca memória. Java é muito mais eficiente que grande parte das linguagens de “scripting” existentes, embora seja cerca de 20 vezes mais lenta que C, o que não é um marco definitivo. Com a evolução da linguagem, serão criados geradores de “byte-codes” cada vez mais otimizados que trarão as marcas de performance da linguagem mais próximas das de C++ e C. Além disso um dia Java permitirá a possibilidade de gerar código executável de uma particular arquitetura “on the fly”, tudo a partir do “byte-code”.

Suporte para programação de sistemas distribuídos:

Java fornece facilidades para programação com sockets, remote method call, tcp-ip, etc. Estes tópicos não serão abordados nesta apostila.



2.2 ONDE ENCONTRAR O JAVA E COMO INSTALAR

O Java é uma linguagem de propriedade da Sun Microsystems® e pode ser obtida no site da Sun através do endereço <http://java.sun.com/j2se/1.4/download.html>.

A Sun disponibiliza a tecnologia Java padrão para três plataformas, UNIX/Linux, Windows e Mac OS, sem custos adicionais, em um Kit de desenvolvimento padrão, o SDK (Software Development Kit), que anteriormente era denominado de JDK (Java Development Kit).

Após obtermos o Kit para a plataforma de desenvolvimento desejada, o SDK pode ser instalado facilmente através do executável respectivo para cada plataforma. Após a instalação, o compilador e outras ferramentas do Kit ainda não estarão à disposição para todo o sistema operacional, a não ser que chamemos diretamente o compilador do diretório em que foi instalado. Dessa forma, a fim de facilitar nosso trabalho de desenvolvimento, precisamos configurar as variáveis de ambiente da plataforma.

No Windows®, a variável de ambiente que possui os caminhos dos programas e utilitários usados pelo sistema operacional é a variável PATH. Após a instalação do SDK, o programa jamais inclui o caminho do compilador na variável. Para verificar se o caminho do compilador está na variável, basta acionar o comando PATH no prompt do sistema.

Caso o caminho não esteja na variável, a fim de colocá-lo permanentemente nesta e poupar trabalho, precisamos adicionar a seguinte linha no arquivo AUTOEXEC.BAT e depois reiniciando o computador para que o comando seja executado:

```
SET PATH=C:\j2sdk1.4.0_03\bin;%PATH%  
SET CLASSPATH = C:\j2sdk1.4.0_03\lib  
SET INCLUDE = C:\j2sdk1.4.0_03\include
```

Feito isso, já podemos chamar o compilador e outros programas do Kit de qualquer diretório do sistema sem necessitar informar o diretório onde está o compilador.

Programas do JDK (*Java Development Kit*)

javac: compila programas fonte e gera arquivos de classe no formato de *bytecode*.

java: interpreta aplicações escritas em Java.

jdb: faz depuração da execução de aplicações e *applets* Java.

javap: desassemblador de *bytecodes*.

javadoc: gerador de documentação HTML a partir de programas fonte Java.

appletviewer : carregador de *applets* sem usar web browsers.

Como Compilar: `javac <nomedofonte.java>` | `javac HelloWorld.java` (em geral, é criado um arquivo ".class" para cada classe)

Execução: `java <nomedaclasse>` | `java HelloWorld`

O JKD (*Java Development Kit*) contém vários pacotes que já vem com muitas bibliotecas prontas para usar e os pacotes do JDK são:



UNIVERSIDADE DO AMAZONAS
FACULDADE DE TECNOLOGIA
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS



java.lang: Contém as classes relacionadas ao funcionamento básico do ambiente Java: envoltórios (wrappers) para os tipos de dados básicos e strings, gerenciamento de threads, operações matemáticas, exceções, a classe System e a classe básica Object.

java.io: Contém classes para o controle de operações de arquivos e fluxos de entrada e saída.

java.util: Contém classes para a manipulação de datas, estrutura básica de dados (tabelas hash e pilhas), análise de strings, etc.

java.applet: Contém a classe Applet que une o applet ao ambiente funcional correspondente (por exemplo, o de um browser), uma classe para explorar as características do navegador (browser), etc.

java.awt: O pacote AWT (Abstract Window Toolkit) fornece ferramentas simples e avançadas para controlar a interface visual de um applet: botões, campos de texto, componentes de menu, gráficos, fontes, eventos, etc.

java.awt.image: É um sub-pacote do java.awt, bastante complexo e muito eficiente na utilização de imagens.

java.awt.peer: Utilizado para interligar os itens do AWT à plataforma básica.

java.awt.net: Contém as classes necessárias para programas de rede: sockets para cliente e servidor, referência a URLs, etc.

2.3 A TECNOLOGIA JAVA

A tecnologia Java consiste em uma linguagem de programação e uma plataforma.

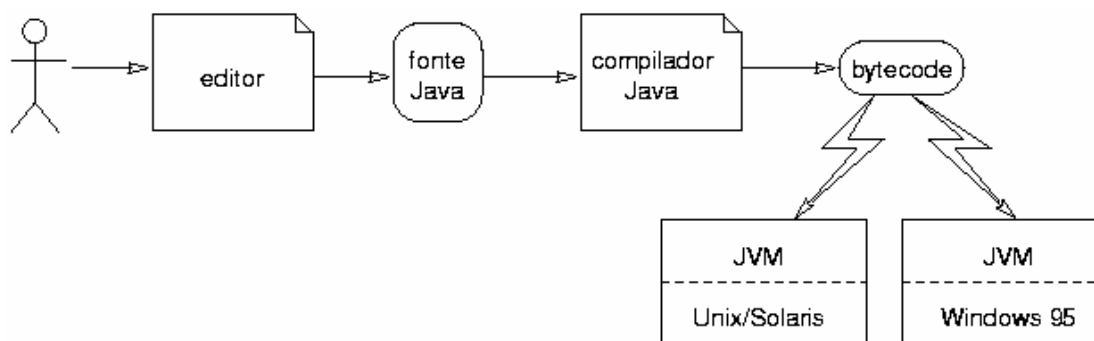
Como linguagem programação, Java é uma linguagem de alto nível e possui as seguintes características:

- Simples
- Orientada a objetos
- Distribuída
- Interpretada
- Robusta
- Segura
- Arquitetura neutra
- Portável
- De alta performance
- Multitarefa
- Dinâmica

Na maioria das linguagens existentes, ou você compila ou interpreta o programa.

Um programa em Java é tanto compilado como interpretado. O compilador apenas traduz o programa para um código intermediário chamado Java bytecodes – um código interpretado independente de plataforma. Por sua vez o interpretador Java verifica e executa cada instrução em bytecode no computador. A compilação ocorre somente uma vez, a interpretação ocorre toda vez que for necessária.

Para facilitar o entendimento, podemos imaginar que a execução dos bytecodes aconteceria da mesma forma que a execução de um programa escrito em Java ocorresse em um computador que somente entendesse Java. Embora esse computador fisicamente não exista, ele pode existir logicamente, simulando um verdadeiro computador que entende Java: a máquina virtual Java – Java Virtual Machine (JVM). Todo interpretador que executa applets, seja uma ferramenta de desenvolvimento, seja um navegador, é uma implementação da JVM.



A Máquina Virtual Java (JVM) é uma máquina de computação abstrata e um ambiente de execução independente de plataforma. Programas escritos em Java e que utili-



zema as funcionalidades definidas pelas APIs dos pacotes da plataforma Java executam nessa máquina virtual.

Assim se resume filosofia Java: “Escreva uma vez, execute em qualquer lugar”. Isso significa que podemos compilar os programas em qualquer plataforma que desenvolva em Java e executar o bytecode em qualquer plataforma que implemente a JVM.

As plataformas existentes são freqüentemente uma combinação de hardware e software. Como plataforma, Java difere das outras por ser uma plataforma somente software que é executada sobre as plataformas baseadas em hardware.

A plataforma Java é constituída de dois componentes básicos:

1. A Máquina Virtual Java – Java Virtual Machine (JVM) ;
2. A Interface Programada Java de Aplicação – Java Application Programming Interface (API).

Como discutiremos anteriormente sobre JVM, voltaremos nossa atenção somente para a API Java.

A API Java é uma vasta coleção de componentes de software “prontos para usar” que dão suporte a recursos do Java, como a interface gráfica para usuários – Graphical User Interface (GUI). Esses recursos são agrupados em bibliotecas chamadas pacotes (packages) e enumeram classes e interfaces.

Dessa forma um programa escrito em Java apenas precisa de bibliotecas, a API, para conseguir a utilidade desejada, e de uma máquina virtual capaz de executar as instruções Java, isolando o contato do programa com o hardware e por conseguinte facilitando a portabilidade e segurança.

Embora Java tenha vantagens sobre outras linguagens que geram códigos nativos, isto é, códigos para uma plataforma específica, o desempenho de um código Java é um pouco mais lento que das outras, porém tão eficiente quanto. Foram realizados estudos que demonstraram que o desempenho de um programa escrito em C e compilado para uma plataforma específica é vinte vezes melhor que o Java. Entretanto, a Sun e seus parceiros, auxiliados pela evolução tecnológica, unem esforços contínuos para desenvolver compiladores inteligentes, interpretadores mais eficientes e otimizadores de bytecode cujos resultados consistem em aproximar o desempenho do Java aos dos demais códigos nativos.



3. PROGRAMAÇÃO EM JAVA

3.1 TIPOS PRIMITIVOS

Os tipos primitivos existentes em java estão listados abaixo, sendo que as estas variáveis não precisam ser alocadas após serem declaradas em Java.

Nomes dos Tipos	Tam.(Bits)	Faixa de Valores
Long	64	-9.223.372.036.854.775.808 a 9.223.372.036.854.775.807
Int	32	-2.147.483.648 a 2.147.483.647
Short	16	-32.768 a 32.767
Byte	8	-128 a 127
Double	64	1,7e-308 a 1,7e+308
Float	32	3,4e-038 a 3,4e+038
Char	16	0 a 65536 (não assume valores negativos)
Boolean	1	True ou False

Obs.: Java utiliza o Unicode para representar os caracteres. O Unicode define um conjunto internacional de caracteres, capaz de representar os caracteres de todas as línguas humanas. Ele reúne dezenas de conjuntos de caracteres, como latino, grego, árabe, cirílico, hebraico, katakana, hangul e muitos outros. Para possibilitar isso, são necessários 16 bits.

É por isso que o tipo char de Java tem 16 bits. O alcance de char é de 0 a 65.536. char não pode assumir valores negativos.

Valores literais de caracteres podem também ser representados por seqüências de escape, como em '\n' (o caracter newline).

Variáveis booleanas e variáveis inteiras, ao contrário do que ocorre em C e C++, não são compatíveis em Java. Assim, não faz sentido atribuir uma variável booleana a uma variável inteira ou usar um valor inteiro como uma condição de um teste.

A lista de seqüências de escape reconhecidas em Java é:

Seqüência	Caracter
\b	Backspace
\t	Tabulação horizontal
\n	Newline
\f	form feed
\r	Carriage return
\"	Aspas
\'	aspas simples
\\	Contrabarra
\xxx	o caracter com código de valor octal xxx, que pode assumir valores entre 000 e 377
\uxxxx	o caracter Unicode com código de valor hexadecimal xxxx, onde xxxx pode assumir valores entre 0000 e ffff



3.1.1 Classes wrapper (Classes embrulho)

A plataforma Java disponibiliza diversas classes que permitem representar, como objetos, valores primitivos. Tratam-se das classes: Boolean, Char, Byte, Short, Integer, Long, Float e Double.

Estas classes são designadas por wrappers porque os seus objetos limitam-se a servir de embrulho a valores primitivos. Por exemplo, um objeto da classe Integer contém uma única variável de instância, de tipo int, onde se guarda um inteiro.

Estas classes embrulho contêm alguns métodos relativamente úteis, mas repare que não disponibilizam as operações associadas aos tipos primitivos. Por exemplo, não é possível adicionar diretamente dois objetos da classe Integer.

Exemplo de criação de objeto da classe Integer:

```
Int iobj = new Int(21056) ;
```

Exemplo de acesso ao valor guardado dentro dum objeto da classe Integer:

```
int i = iobj.intValue() ;
```

3.1.1.1 Classe Boolean

A classe java.lang.Boolean oferece facilidades para a manipulação de valores do tipo primitivo boolean.

Por exemplo, o método `valueOf()` permite converter uma string para um valor booleano. Se o argumento do método é a string "true" (independente de estar em letras maiúsculas ou minúsculas), esse método retorna o valor booleano verdadeiro (true):

Exemplo: `Boolean.valueOf("True")` returns true.

Exemplo: `Boolean.valueOf("yes")` returns false.



3.1.1.2 Classe Character

A classe Character oferece facilidades para a manipulação de variáveis do tipo character.

A maior parte das funcionalidades é definida em métodos de classe (static). Por exemplo, dado que uma variável foi declarada como character, ela pode ser utilizada como argumento nesses métodos para verificar que tipo de valor está associado à variável, como `isLetter()` e `isUpperCase()`, ou realizar modificações nessa representação, como `toUpperCase()` ou `digit()`, que converte um character que representa um dígito para um valor inteiro correspondente.

Outra funcionalidade importante desta classe é a possibilidade de encapsular uma variável desse tipo como sendo um objeto. Um objeto Character pode ser construído tendo como argumento para o construtor a variável do tipo char, que pode ser então utilizada onde objetos são esperados. Para recuperar a variável do objeto, o método `charValue()` é aplicado.

Obs.: Para mais informações:

<http://java.sun.com/j2se/1.4.2/docs/api/java/lang/Character.html>

3.1.1.3 Classes para manipular valores numéricos inteiros

Para cada tipo numérico inteiro, Java define uma classe correspondente com facilidades para manipulação de seus valores. Assim, são definidas as classes Byte, Short, Integer e Long.

Em cada uma dessas classes são definidas constantes que especificam qual os limites de valores representáveis em uma variável do tipo correspondente, tais como `Integer.MIN_VALUE` e `Long.MAX_VALUE`. Cada classe oferece também, através de métodos da classe, funcionalidades para converter uma representação de um valor da forma string para a forma numérica, como por exemplo `parseInt()`.

Outra funcionalidade importante destas classes é a possibilidade de encapsular uma variável de cada um desses tipos como sendo um objeto da classe correspondente. Por exemplo, um objeto Integer pode ser construído tendo como argumento para o construtor a variável do tipo int, que pode ser então utilizada onde objetos são esperados. Para recuperar a variável int do objeto Integer, o método `intValue()` é aplicado.



3.1.1.4 Classes para manipular valores numéricos reais

Java oferece duas classes com facilidades para manipular valores numéricos reais, Float e Double.

Para cada uma das classes, são definidas constantes que representam os mínimo e máximo valores representáveis (por exemplo, Float.MIN_VALUE e Double.MAX_VALUE), assim como as representações internas para os valores infinitos negativo e positivo (Float.NEGATIVE_INFINITY, Double.POSITIVE_INFINITY) e a representação para Not A Number (Double.NaN), um valor especial definido no padrão de representação de valores inteiros em computadores.

Há funcionalidades definidas como métodos da classe, como para testar se o valor de uma variável corresponde a uma das situações especiais de representação (por exemplo, isNaN()) e para conversão de representação, como parseDouble(). Há também funcionalidades para converter entre as representações binárias (internas) de um número real e seu valor, como floatToIntBits() e longBitsToDouble().

Outra funcionalidade importante destas classes é a possibilidade de encapsular uma variável de cada um desses tipos como sendo um objeto da classe correspondente. Por exemplo, um objeto Double pode ser construído tendo como argumento para o construtor a variável do tipo double, que pode ser então utilizada onde objetos são esperados. Para recuperar a variável double do objeto Double, o método doubleValue() é aplicado.

3.1.1.6 Classe String

Ao contrário do que ocorre em C e C++, strings em Java não são tratadas como seqüências de caracteres terminadas por NUL. São objetos, instâncias da classe java.lang.String.

Uma string pode ser criada como em:

```
String s = "abc";
```

O operador '+' pode ser utilizado concatenar strings:

```
System.out.println("String s: " + s);
```

É possível concatenar valores de outros tipos a strings:

```
int umInteiro;  
System.out.println("Valor: " + umInteiro);
```

onde implicitamente ocorre uma conversão da forma

```
System.out.println("Valor: " + Integer.toString(umInteiro));
```

A classe String define uma série de funcionalidades para manipular strings, tais como:



Obter o número de caracteres em uma string:

```
int length();
```

Concatenar strings:

```
String concat(String outro);
```

Comparar strings:

```
boolean equals(String outro);  
boolean equalsIgnoreCase(String outro);  
int compareTo(String outro);
```

Obs.: não podemos esquecer que Java é sensitive case ou seja ele faz diferenciação entre caracteres maiusculo e minusculo, para comparar strings sem considerar maiusculas e minusculas é só utilizar o comando “equalsIgnoreCase”.

Extrair caracteres e substrings:

```
int charAt(int posição);  
String substring(int pos_inicial);  
String substring(int pos_inicial, int pos_final);
```

Localizar substrings:

```
int indexOf(String substring);  
int indexOf(String substring, int pos_inicial);  
int lastIndexOf(String substring);  
int lastIndexOf(String substring, int pos_inicial);  
boolean startsWith(String prefixo);  
boolean endsWith(String sufixo);
```

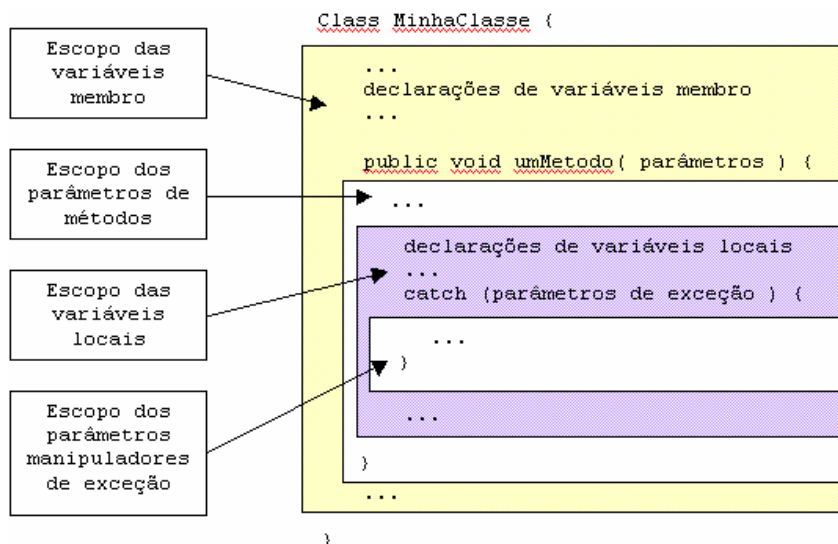
Obs.: Maiores informações podem ser consultadas no help disponibilizada pela SUN© MICROSYSTEMS no site :

<http://java.sun.com/j2se/1.4.1/docs/api/java/lang/String.html>.

3.2 Escopo

Escopo é a região na qual uma variável pode ser referenciada pelo seu nome. Em segundo lugar, o escopo determina o quando o sistema criará a variável e o momento de sua destruição. Seu conceito difere de visibilidade, uma vez que esta determina se a variável pode ser usada fora da classe em que foi criada.

O local em que declaramos a variável determina o seu escopo que está entre as seguintes categorias:



Uma variável membro é uma variável que está declarada dentro de uma classe ou objeto, contudo está fora de qualquer método ou construtor. O escopo de uma variável membro corresponde a definição por inteiro de uma classe. Deve ser devidamente declarada e inicializada antes de ser usada.

As variáveis locais podem ser declaradas no interior de um bloco de código. Em regra, o escopo de uma variável local corresponde ao bloco de código em que está inserida, ou seja, desde sua declaração até o “fecha chaves” - }.

Parâmetros são argumentos formais para métodos ou construtores e são usados na passagem valores para o interior dos métodos. O escopo é todo a definição do método a qual pertence.

Parâmetros manipuladores de exceção são similares a parâmetros de métodos, exceto no que diz respeito a funcionalidade, pois são argumentos para manipular exceções. O escopo é delimitado pelo bloco que se segue ao catch.



3.3 OPERADORES EM JAVA

3.3.1 Operadores Básicos

Operador	Utilização	Descrição
+	op1 + op2	Adiciona op1 a op2
-	op1 - op2	Subtrai op2 de op1
*	op1 * op2	Multiplica op1 por op2
/	op1 / op2	Divide op1 por op2
%	op1 % op2	Retorna o resto da divisão inteira de op1 por op2

3.3.2 Operadores Unários

O operador que trabalha como um operando é denominado operador unário. Os operadores unários suportam notação posfixa ou prefixa. Nesta, o operador aparece antes do operando. Naquela, o operador aparece depois. Observe o exemplo a seguir:

```
operador operando; //notação prefixa  
operando operador; //notação posfixa
```

Operador	Utilização	Descrição
++	op1++ ou ++op1	Incrementa em uma unidade o valor do operando.
--	op1-- ou --op1	Decrementa em uma unidade o valor do operando.

Os demais operadores têm notação infixa, ou seja, os operadores estão situados entre os operandos.

Obs.: Há diferença entre a notação prefixa e a posfixa pois na notação prefixa é primeiro realizado a operação e depois é utilizada o valor do operando.

Ex.:

fazendo a = 15;
utilizando a notação prefixa: (++a) + a
é realizado primeiro a soma ao valor da variável "a" que fica 16 e é feito uma soma de : 16 + 16.
utilizando notação posfixa:
(a++) + a : a soma a ser realizada é : 15+16 e não 16 + 16.

3.3.3 Operadores de Atribuição

O operador binário = atribui o valor da expressão do lado direito à variável à esquerda do operador. Os tipos da expressão e da variável devem ser compatíveis. Os operadores listados a seguir são que facilitam certas operações matemáticas:

Operador	Utilização	Equivalente a
+=	op1 += op2	op1 = op1 + op2
-=	op1 -= op2	op1 = op1 - op2
*=	op1 *= op2	op1 = op1 * op2
/=	op1 /= op2	op1 = op1 / op2
%=	op1 %= op2	op1 = op1 % op2

3.3.4 Operadores Bit-a-Bit

Operador	Utilização	Descrição
>>	op1 >> op2	Shift à direita
<<	op1 << op2	Shift à esquerda
>>>	op1 >>> op2	Shift à direita (sem sinal)
&	op1 & op2	And
	op1 op2	Or
^	op1 ^ op2	Xor
~	~op	Complemento

Operadores que facilitam a manipulação bit-a-bit

Operador	Utilização	Equivalente a
&=	op1 &= op2	op1 = op1 & op2
=	op1 = op2	op1 = op1 op2
^=	op1 ^= op2	op1 = op1 ^ op2
<<=	op1 <<= op2	op1 = op1 << op2
>>=	op1 >>= op2	op1 = op1 >> op2
>>>=	op1 >>>= op2	op1 = op1 >>> op2



3.3.5 Operadores relacionais e condicionais

Operadores Relacionais:

Operador	Utilização	Retorna true se
>	Op1 > op2	op1 é maior que op2.
>=	Op1 >= op2	op1 é maior ou igual a op2
<	Op1 < op2	op1 é menor que op2.
<=	Op1 <= op2	op1 é menor ou igual a op2
==	Op1 == op2	op1 é igual a op2
!=	Op1 != op2	op1 é diferente de op2

Operadores Condicionais:

Operador	Utilização	Retorna true se
&&	op1 && op2	Op1 e op2 são true; avalia condicionalmente op2
	op1 op2	Op1 ou op2 é true; avalia condicionalmente op2
!	! op	Inverte o valor booleano de op

3.3.6 Operador de Type Cast

O operador de type cast em Java funciona igualmente em C/C++ o, sendo que a sintaxe continua a mesma:

Variável = ((Tipo)Operação);

Sendo que o tipo entre parenteses deve ser do mesmo tipo da variável.

Ex.:

```
int c;  
c = ((int)8/3)  
// O resultado que deveria ser 2,67 é convertido para 2
```



3.4 ESTRUTURAS DE CONTROLE EM JAVA

3.4.1 If (ou Se) em Java

O IF em java é o que verifica se o resultado lógico de uma comparação em Java é true (verdadeiro) ou false (Falso). É equivalente ao “se” condicional do algoritmo.

Sua sintaxe é:

```
if (condição)
{
    <comandos>; //Executa estes comandos se o resultado da condição for verdadeiro
}
else
{
    <comandos>; //Executa estes comandos se o resultado da condição for falsa
}
```

3.4.2 O comando Switch

O comando switch funciona como se fosse um conjunto de if e else's aninhados de maneira a realizar uma única tarefa apartir de uma condição. Sendo que o operador para se verificar a condição deve ser do tipo char ou int, devendo-se colocar a palavra chave “break;” depois de feita cada operação.

Sintaxe:

```
switch (condição)
{
    case opção1: { comandos; }; break;
    case opção2: { comandos; }; break;
    case opção3: { comandos; }; break;
    case opção4: { comandos; }; break;
    default: { comandos; };
}
```

Sendo que a clausula “default” é opcional, mais deve ser utilizada sempre que nenhuma das opções anteriores tenham sido executadas.



3.5 LAÇOS DE REPETIÇÃO EM JAVA

3.5.1 Laço For

O laço de repetição for é utilizado para laços de repetição definidos ou seja em que o número de repetições é conhecido.

Se você conhece C ou C++, verá que o for em Java funciona da mesma forma que nessas linguagens.

A forma mais simples do for é a seguinte:

```
for(inicialização; condição; iteração)
{
    comando;
}
```

Geralmente a porção de inicialização ajusta a variável de controle do for para um valor inicial. A condição é uma expressão booleana que testa a variável de controle do loop. Se o resultado do teste for verdadeiro (true), o for continua a ser executado. Se for falso (false), o for é encerrado. A expressão de iteração determina como a variável de controle do for é alterada cada vez que o for é executado.

3.5.2 Laço While

Esta estrutura faz com que a condição seja avaliada em primeiro lugar. Se a condição é verdadeira o bloco é executado uma vez e a condição é avaliada novamente.

Caso a condição seja falsa a repetição é terminada sem a execução do bloco.

Sintaxe:

```
while(condição)
{
    bloco
}
```

onde: condição é uma expressão lógica.

Bloco: é um conjunto de instruções.

3.5.3 Laço Do ... While

Esta estrutura faz com que o bloco de instruções seja executado pelo menos uma vez. Após a execução do bloco, a condição é avaliada. Se a condição é verdadeira o bloco é executado outra vez, caso contrário a repetição é terminada.

```
Sintaxe: do{
        bloco;
    } while(condição);
```

onde:

condição: é uma expressão lógica.

Bloco: é um conjunto de instruções.



3.5.4 Comandos de salto

Os comandos de salto são usados para transferir a execução para outra parte do programa. Java oferece três comandos de salto:

- break
- continue
- return

3.5.4.1 O Comando Break

Inicialmente, examinaremos o comando break. Vimos na lição 3.4.2 como break pode ser usado para terminar uma sequência de comandos dentro de uma construção switch.

Nesta lição veremos como break pode ser usado para sair de um laço de repetição.

Usando break podemos forçar o encerramento imediato de um laço, contornando a expressão condicional e qualquer código remanescente que esteja dentro do corpo do laço. Quando um comando break é encontrado dentro de um laço, o laço é terminado e o controle do programa é retomado no próximo comando após o laço.

Além do laço for, o comando break pode ser usado em qualquer dos laços de Java, break pode também ser usado para encerrar laços intencionalmente infinitos.

Quando é usado dentro de laços aninhados, o comando break somente encerra o laço mais interno.

Eis alguns aspectos importantes a serem lembrados com respeito ao uso de break:

- É possível ter mais de um comando break em um único loop. Contudo, é preciso ter cuidado. O uso excessivo de comandos break tende a desestruturar o código.
- O comando break que encerra uma construção switch afeta somente o comando switch, e nenhum outro loop que o contenha, se houver.
- Em geral, break não deve ser usado como terminação normal de um loop. A expressão condicional do loop é que deve ser usada para isso. O comando break deve ser usado para cancelar um loop somente quando algum tipo de situação especial ocorrer.

Dissemos que o comando break pode também funcionar como uma forma civilizada do comando goto. Java não tem o comando goto, porque ele representa uma forma de dar um salto de maneira arbitrária e não-estruturada. Isso geralmente torna o código que contém comandos goto difícil de entender e de manter. Além disso, dificulta o trabalho de otimização do código feito pelo compilador.



3.5.4.2 O Comando Continue

O continue faz com que o fluxo de controle vá para a condição de terminação do laço. Assim, a condição de terminação é reavaliada neste ponto e o laço continua ou não dependendo dos resultados do teste. Em laços do tipo for e while, isto significa que o controle retorna ao “topo” do laço. Em laços do-while, o controle retorna para o “fundo” do laço.

```
for (int i=-10; i < 10; i++) {  
    if (i == 0) { continue; }  
    System.out.println(100/i);  
}
```

Quando a condição é verdadeira continue faz com que o laço teste a condição de parada e adiciona mais um ao contador evitando que um erro seja cometido, pois não existe um número que possa ser dividido por zero.

3.5.4.3 O comando Return

O return é utilizado para sair do método atual e retornar para o ponto imediatamente subsequente a chamada do método. Existem duas formas de return: uma retorna um valor e a outra não. Para retornar um valor, simplesmente coloque um valor (ou expressão que calcula o valor) depois do return.

O valor retornado deve ser compatível com o tipo do parâmetro de retorno declarado pelo método.

Quando um método é declarado como void, deve se utilizar a versão de return sem valor.

3.6 COMENTÁRIOS

Comentários em Java podem ser expressos em três formatos distintos. Na primeira forma, uma “barra dupla” // marca o início do comentário, que se estende até o fim da linha. A segunda forma é o comentário no estilo C tradicional, onde o par de símbolos /* marca o início de comentário, que pode se estender por diversas linhas até encontrar o par de símbolos */, que delimita o fim do comentário.

O outro estilo de comentário é específico de Java e está associado à geração automática de documentação do software desenvolvido. Nesse caso, o início de comentário é delimitado pelas seqüências de início /** e de término */. O texto entre essas seqüências será utilizado pela ferramenta javadoc para gerar a documentação do código em formato hipertexto.



3.7 CLASSES EM JAVA

3.7.1 Classes

A definição de um modelo conceitual para o domínio da aplicação, contemplando as classes relevantes e suas associações, é um dos principais resultados das etapas de análise e projeto orientado a objetos. A adoção de uma linguagem de modelagem, tal como o diagrama de classes UML, permite expressar esse resultado de maneira organizada e padronizada.

Uma classe é um gabarito para a definição de objetos. Através da definição de uma classe, descreve-se que propriedades - ou atributos - o objeto terá.

Além da especificação de atributos, a definição de uma classe descreve também qual o comportamento de objetos da classe, ou seja, que funcionalidades podem ser aplicadas a objetos da classe. Essas funcionalidades são descritas através de métodos. Um método nada mais é que o equivalente a um procedimento ou função, com a restrição que ele manipula apenas suas variáveis locais e os atributos que foram definidos para um objeto desse tipo.

Uma vez que estejam definidas quais serão as classes que irão compor uma aplicação, assim como qual deve ser sua estrutura interna e comportamento, é possível criar essas classes em Java.

3.7.2 DEFINIÇÃO DE CLASSES EM JAVA

Em Java, classes são definidas através do uso da palavra-chave `class`. Para definir uma classe, utiliza-se a construção:

```
[modif] class NomeDaClasse
{
    // corpo da classe...
}
```

A primeira linha é um comando que inicia a declaração da classe. Após a palavra-chave `class`, segue-se o nome da classe, que deve ser um identificador válido para a linguagem. O modificador `modif` é opcional; se presente, pode ser uma combinação de `public` e `abstract` ou `final`.

A definição da classe propriamente dita está entre as chaves `{` e `}`, que delimitam blocos na linguagem Java. Este corpo da classe usualmente obedece à seguinte seqüência de definição:

As variáveis de classe, iniciando pelas `public`, seguidos pelas `protected`, pelas com visibilidade padrão (sem modificador) e finalmente pelas `private`.

Os atributos (ou variáveis de instância) dos objetos dessa classe, seguindo a mesma ordenação definida para as variáveis de classe.

Os construtores de objetos dessa classe.

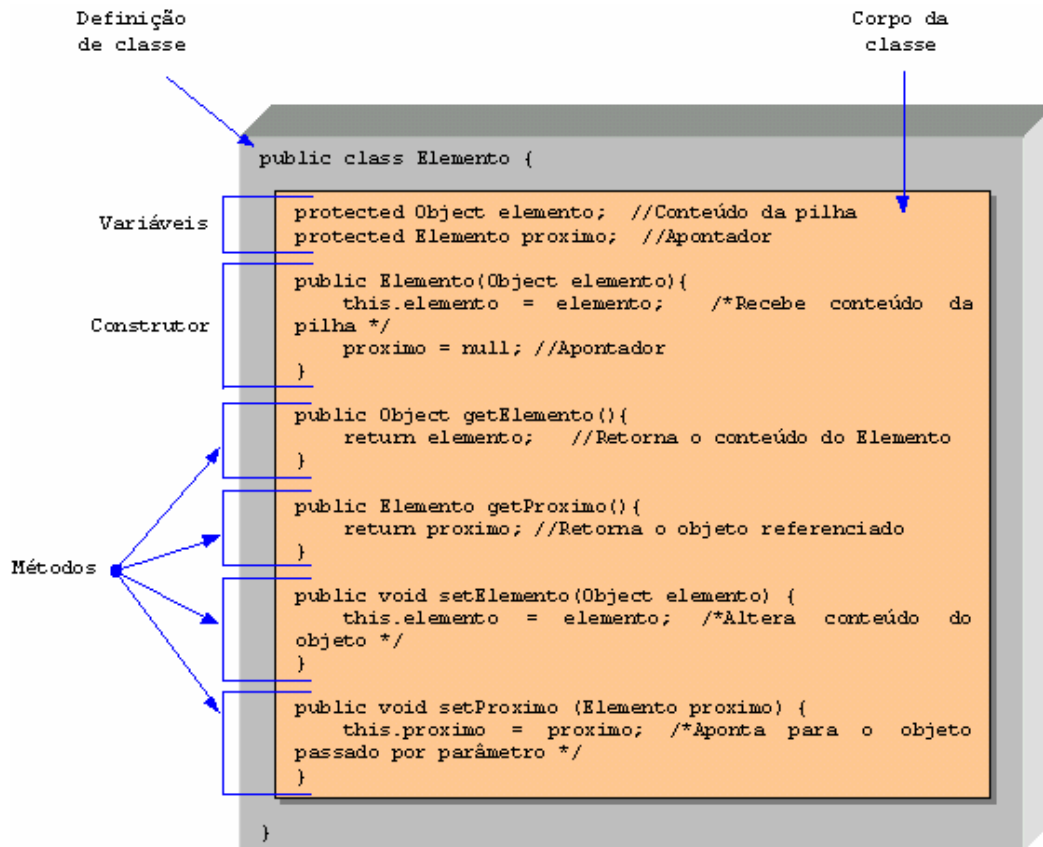
Os métodos da classe, geralmente agrupados por funcionalidade.

Toda classe pode também ter um método main associado, que será utilizado pelo interpretador Java para dar início à execução de uma aplicação.

Java também oferece outra estrutura, denominada interface, com sintaxe similar à de classes mas contendo apenas a especificação da funcionalidade que uma classe deve conter, sem determinar como essa funcionalidade deve ser implementada.

Propriedades de uma classe (meta-informação) podem ser obtidas através das funcionalidades oferecidas na classe `java.lang.Class`.

Exemplo do escopo de uma Classe em Java:





Palavras reservadas para classes

Palavra reservada	Descrição
Public	Acesso livre para qualquer outra classe
Abstract	Classe abstrata, não pode ser instanciada
Final	A classe não pode ter subclasses
Class <i>nomeDaClasse</i>	Nome da classe
Extends <i>Superclasse</i>	Indica herança de classe
Implements <i>Interfaces</i>	Interfaces implementadas pela classe

3.7.2.1 Palavra Reservada public

Uma classe definida como pública pode ser utilizada por qualquer objeto de qualquer pacote. Em Java, uma unidade de compilação (um arquivo fonte com extensão .java) pode ter no máximo uma classe pública, cujo nome deve ser o mesmo do arquivo (sem a extensão). As demais classes na unidade de compilação, não públicas, são consideradas classes de suporte para a classe pública e têm a visibilidade padrão.

Um atributo público de uma classe pode ser diretamente acessado e manipulado por objetos de outras classes.

Um método público de uma classe pode ser aplicado a um objeto dessa classe a partir de qualquer outro objeto de outra classe. O conjunto de métodos públicos de uma classe determina o que pode ser feito com objetos da classe, ou seja, determina o seu comportamento.

3.7.2.2 Palavra Reservada **protected**

A palavra-chave **protected** restringe a visibilidade do membro modificado, atributo ou método, de forma que classes não-relacionadas não possam acessá-lo. Objetos da própria classe, de classes derivadas desta e de classes do mesmo pacote têm acesso a membros **protected**.

3.7.2.3 Palavra Reservada **private**

A palavra-chave **private** restringe a visibilidade do membro modificado, método ou atributo, exclusivamente a objetos da própria classe que contém sua definição.



3.7.2.4 Classes abstratas e finais

//Melhorar o conceito de classe abstrata.

Uma classe abstrata não pode ser instanciada, ou seja, não há objetos que possam ser construídos diretamente de sua definição. Por exemplo, a compilação do seguinte trecho de código

```
abstract class AbsClass {  
    public static void main(String[] args) {  
        AbsClass obj = new AbsClass();  
    }  
}
```

geraria a seguinte mensagem de erro:

```
AbsClass.java:3: class AbsClass is an abstract class.  
It can't be instantiated. AbsClass obj = new AbsClass();  
                                     ^
```

1 error

Em geral, classes abstratas definem um conjunto de funcionalidades das quais pelo menos uma está especificada mas não está definida — ou seja, contém pelo menos um método abstrato, como em

```
abstract class AbsClass {  
    public abstract int umMetodo();  
}
```

Um método abstrato não cria uma definição, mas apenas uma declaração de um método que deverá ser implementado em uma classe derivada. Se esse método não for implementado na classe derivada, esta permanece como uma classe abstrata mesmo que não tenha sido assim declarada explicitamente.

Assim, para que uma classe derivada de uma classe abstrata possa gerar objetos, os métodos abstratos devem ser definidos em classes derivadas:

```
class ConcClass extends AbsClass {  
    public int umMetodo() {  
        return 0;  
    }  
}
```

Uma classe final, por outro lado, indica uma classe que não pode ser estendida. Assim, a compilação do arquivo `Reeleicao.java` com o seguinte conteúdo:



```
final class Mandato {
```

```
    public class Reeleicao extends Mandato {  
    }  
}
```

ocasionaria um erro de compilação:

caolho:Exemplos[39] javac Reeleicao.java

Reeleicao.java:4: Can't subclass final classes: class Mandato

```
public class Reeleicao extends Mandato {  
    ^
```

1 error

A palavra-chave final pode também ser aplicada a métodos e a atributos de uma classe. Um método final não pode ser redefinido em classes derivadas.

Um atributo final não pode ter seu valor modificado, ou seja, define valores constantes. Apenas valores de tipos primitivos podem ser utilizados para definir constantes.

O valor do atributo deve ser definido no momento da declaração, pois não é permitida nenhuma atribuição em outro momento.

A utilização de final para uma referência a objetos é permitida. Como no caso de constantes, a definição do valor (ou seja, a criação do objeto) também deve ser especificada no momento da declaração. No entanto, é preciso ressaltar que o conteúdo do objeto em geral pode ser modificado — apenas a referência é fixa. O mesmo é válido para arranjos.

A partir de Java 1.1, é possível ter atributos de uma classe que sejam final mas não recebem valor na declaração, mas sim nos construtores da classe. (A inicialização deve obrigatoriamente ocorrer em uma das duas formas.) São os chamados blank finals, que introduzem um maior grau de flexibilidade na definição de constantes para objetos de uma classe, uma vez que essas podem depender de parâmetros passados para o construtor. Argumentos de um método que não devem ser modificados podem ser declarados como final, também, na própria lista de parâmetros.



3.7.3 Atributos

A definição de atributos de uma classe Java reflete de forma quase direta a informação que estaria contida na representação da classe em um diagrama UML. Para tanto, a sintaxe utilizada para definir um atributo de um objeto é:

[modificador] tipo nome [= default];

onde :

[*modificador*] : (opcional) é uma combinação de public, protected ou private; final; e static.

Tipo: deve ser um dos tipos primitivos da linguagem Java ou o nome de uma classe ou interface;

Nome: deve ser um identificador válido;

Default: (opcional) é a especificação de um valor inicial para a variável.

A tabela a seguir mostra as possíveis formas de declarações de atributos seguidas de uma breve descrição:

Palavra reservada	Descrição
Nível de acesso: public, private, protected, package	Especifica o nível de acesso da variável membro em relação a outras classes.
Static	Declara uma variável de classe.
Final	Indica que o valor de uma variável membro é constante e não poderá ser alterado.
Transient	Marcador ainda não totalmente especificado na linguagem, utilizado freqüentemente em serialização.
Volatile	Utilizados apenas por programadores experientes no intuito de prevenir que o compilador faça otimizações na variável membro.

3.7.3.1 Membros de Classe static

Cada objeto de uma classe tem sua própria cópia de todas as variáveis de instância da classe. Em certos casos, apenas uma cópia de uma variável particular deve ser compartilhada por todos os objetos de uma mesma classe. Utiliza-se uma variável de classe static por essas e outras razões. A variável de classe static representa informações de escopo da classe, todos os objetos da classe compartilham a mesma parte dos dados. A declaração de um membro static inicia com a palavra-chave static. Embora as variáveis de classe static possam se parecer com variáveis globais, as variáveis de classe static tem escopo de classe. Na prática utiliza-se variáveis static quando se quer compartilhar um(as) variável(eis) entre vários objetos do mesmo tipo.



3.7.4 Métodos

Em Java, a definição de um método em uma classe é expressa dentro do corpo da classe como um bloco na forma:

```
[modificador] tipo nome(argumentos)
{ corpo do método }
```

Onde o nome (um identificador válido) e o tipo dos argumentos constituem a assinatura do método. O tipo é um indicador do tipo de retorno do método, que pode ser um dos tipos primitivos da linguagem, um tipo de objeto (nome de classe ou interface), ou void se não houver nenhum valor de retorno.

O modificador (opcional) é uma combinação de um especificador de visibilidade (public, protected ou private), de um especificador de redefinição (abstract ou final) e de um especificador de escopo (static).

Métodos são essencialmente procedimentos que podem manipular atributos de objetos para os quais o método foi definido. Além dos atributos de objetos, métodos podem definir e manipular variáveis locais; também podem receber parâmetros por valor através da lista de argumentos.

Uma boa prática de programação é manter a funcionalidade de um método simples, desempenhando uma única tarefa. O nome do método deve refletir de modo adequado a tarefa realizada. Se a funcionalidade do método for simples, será fácil encontrar um nome adequado para o método.

Como ocorre para a definição de atributos, a definição de métodos reflete de forma quase direta a informação que estaria presente em um diagrama de classes UML, a não ser por uma diferença vital: o corpo do método.

Métodos de mesmo nome podem co-existir em uma mesma classe desde que a lista de argumentos seja distinta, usando o mecanismo de sobrecarga.



Palavras reservadas para a criação de Métodos

Palavra reservada	Descrição
<i>Nível de acesso:</i> public, private, protected, package	Especifica o nível de acesso do método em relação a outras classes.
Static	Declara um método de classe.
Abstract	Métodos abstratos são membros de classes abstratas e não possuem implementação.
Final	Indica que o método não poderá ser sobrescrito por outra classe.
Native	Se você possui bibliotecas de funções escritas em outra linguagem, como C, você pode utilizar essas funções com Java através desta palavra reservada. Para mais detalhes, procure por JNI.
<i>Tipo de retorno</i>	Todo método em Java necessita retornar um resultado. Assim devemos colocar o tipo de retorno. Caso o método não deva retornar um tipo deve-se utilizar a palavra reservada <i>void</i> .
<i>Nome do método</i>	Identificador válido em Java para o método.
<i>(lista de parâmetros)</i>	Lista de parâmetros que deve ser passada para o método.
Throws exceções	Se o seu método previne erros determinados, você deve indicar quais as exceções previstas e quais os tipos.

3.7.6 Construtores

Um construtor é um (pseudo-)método especial, definido para cada classe. O corpo desse método determina as atividades associadas à inicialização de cada objeto criado. Assim, o construtor é apenas invocado no momento da criação do objeto através do operador new.

A assinatura de um construtor diferencia-se das assinaturas dos outros métodos por não ter nenhum tipo de retorno (nem mesmo void). Além disto, o nome do construtor deve ser o próprio nome da classe.

O construtor pode receber argumentos, como qualquer método. Usando o mecanismo de sobrecarga, mais de um construtor pode ser definido para uma classe.

Toda classe tem pelo menos um construtor sempre definido. Se nenhum construtor for explicitamente definido pelo programador da classe, um construtor default, que não recebe argumentos, é criado pelo compilador Java. No entanto, se o programador da classe criar pelo menos um método construtor, o construtor default não será criado automaticamente -- se ele o desejar, deverá criar um construtor sem argumentos explicitamente.

No momento em que um construtor é invocado, a seguinte sequência de ações é executada para a criação de um objeto:

O espaço para o objeto é alocado e seu conteúdo é inicializado (bitwise) com zeros.



O construtor da classe base é invocado.

Os membros da classe são inicializados para o objeto, seguindo a ordem em que foram declarados na classe.

O restante do corpo do construtor é executado.

Seguir essa sequência é uma necessidade de forma a garantir que, quando o corpo de um construtor esteja sendo executado, o objeto já terá à disposição as funcionalidades mínimas necessárias, quais sejam aquelas definidas por seus ancestrais. O primeiro passo garante que nenhum campo do objeto terá um valor arbitrário, que possa tornar erros de não inicialização difíceis de detectar.

3.7.7 Interface

Uma interface Java é uma classe abstrata para a qual todos os métodos são implicitamente `abstract` e `public`, e todos os atributos são implicitamente `static` e `final`. Em outros termos, uma interface Java aproxima-se da especificação de uma "classe abstrata pura".

A sintaxe para a declaração de uma interface é similar àquela para a definição de classes, porém seu corpo define apenas assinaturas de métodos e constantes.

Uma interface estabelece uma espécie de contrato que é obedecido por uma classe. Quando uma classe implementa uma interface, garante-se que todas as funcionalidades especificadas pela interface serão oferecidas pela classe.

3.7.7.1 Declaração de interface

A sintaxe para a definição de uma interface Java equivale àquela da definição de uma classe, apenas usando a palavra chave `interface` ao invés da palavra chave `class`.

Por exemplo, para definir uma `Interface1` que declara um método `void met1()` a sintaxe é:

```
interface Interface1
{ void met1(); }
```

A diferença entre uma classe abstrata e uma interface Java é que a interface obrigatoriamente não tem um "corpo" associado. Para que uma classe seja abstrata basta que ela seja assim declarada, mas a classe pode incluir atributos de objetos e definição de métodos, públicos ou não. Na interface, apenas métodos públicos podem ser declarados -- mas não definidos. Da mesma forma, não é possível definir atributos -- apenas constantes públicas.

Enquanto uma classe abstrata é "estendida" (palavra chave `extends`) por classes derivadas, uma interface Java é "implementada" (palavra chave `implements`) por outras classes.



3.7.7.2 Definição de constantes

Outro uso de interfaces Java é para a definição de constantes que devem ser compartilhadas por diversas classes. Neste caso, a recomendação é implementar interfaces sem métodos, pois as classes que implementarem tais interfaces não precisam tipicamente redefinir nenhum método:

```
interface Coins {  
    int  
    PENNY = 1, NICKEL = 5, DIME = 10, QUARTER = 25, DOLAR = 100;  
}  
class CokeMachine implements Coins {  
    int price = 3*QUARTER;  
    // ...  
}
```

3.7.7.3 Implementação de interfaces

Uma vez que interfaces estejam definidas, como são usadas?

Interfaces são especificações que são ou serão implementadas através de classes Java. Por exemplo, uma interface *Abc* definida como

```
interface Abc {  
    void a(int i);  
    int b(String s);  
    String c();  
}
```

indica que qualquer classe implementando essa especificação oferece pelo menos os três métodos, exatamente com as assinaturas descritas. Não é necessário saber como os métodos estão sendo implementados.

Uma classe *Xyz* que implementa a especificação de uma interface *Abc* é declarada com a sintaxe:

```
class Xyz implements Abc {  
    // declarações, outros métodos  
    void a(int valor) {  
        ...  
    }  
  
    int b(String nome) {  
        ...  
    }  
}
```



```
String c() {  
    ...  
}  
}
```

É na classe que o corpo de cada um dos métodos da interface é efetivamente especificado, determinando como ocorre a implementação.

3.7.8 O MÉTODO MAIN

Toda classe pode ter um método main, que determina o ponto de início de execução de qualquer aplicação Java. Ao contrário do que acontece em C e C++, onde apenas uma função main deve ser definida para a aplicação como um todo, toda e qualquer classe Java pode ter um método main definido. Apenas no momento da interpretação o main a ser executado é definido através do primeiro argumento (o nome da classe) para o programa interpretador.

O método main é um método associado à classe e não a um objeto específico da classe -- assim, ele é definido como um método estático. Adicionalmente, deve ser um método público para permitir sua execução a partir da máquina virtual Java. Não tem valor de retorno, mas recebe como argumento um arranjo de strings que corresponde aos parâmetros que podem ser passados para a aplicação a partir da linha de comando. Essas características determinam a assinatura do método.

3.7.8.1 Valor de saída do main

Como o método main é do tipo void, ele não tem valor de retorno. No entanto, assim como programas desenvolvidos em outras linguagens, é preciso algumas vezes obter uma indicação se o programa executou com sucesso ou não. Isto é principalmente útil quando o programa é invocado no contexto de um script do sistema operacional.

Em Java, o mecanismo para fornecer essa indicação é o método `System.exit(int)`.

A invocação desse método provoca o fim imediato da execução do interpretador Java. Tipicamente, o argumento de `exit()` obedece à convenção de que '0' indica execução com sucesso, enquanto um valor diferente de 0 indica a ocorrência de algum problema.



3.7.8.2 Argumento do main

O método main recebe como argumento um parâmetro do tipo arranjo de objetos String. Cada elemento desse arranjo corresponde a um argumento passado para o interpretador Java na linha de comando que o invocou. Por exemplo, se a linha de comando é

```
java Xyz abc 123 def
```

o método main(String[] args) da classe Xyz vai receber, nessa execução, um arranjo de três elementos na variável args com os seguintes conteúdos:

- em args[0], o objeto String com conteúdo "abc";
- em args[1], o objeto String com conteúdo "123";
- em args[2], o objeto String com conteúdo "def".

3.7.8.3 Assinatura do método main

A assinatura do método main é

```
public static void main(String[] args)
```

ou

```
static public void main(String[] args)
```

O nome do parâmetro (args) obviamente poderia ser diferente, mas os demais termos da assinatura devem obedecer ao formato especificado. Se a máquina virtual Java do interpretador não encontrar um método com essa assinatura para a classe especificada, uma exceção será gerada em tempo de execução:

"Exception in thread "main" java.lang.NoSuchMethodError: main"



3.7.9 Controlando o nível de acesso dos membros das classes

Uma das vantagens de escrever classes é que podemos determinar o nível de acesso de seus métodos e variáveis membro em relação a outras classes e objetos. Por que isso é importante? Vamos supor que estejamos projetando uma classe que valide a senha de um usuário dado a própria senha e a conta do usuário. A classe pode guardar o valor da senha do usuário mas não pode mostrar para outra classe. Assim a variável dessa classe deveria ter sua visibilidade restrita ao máximo. Existe ainda, em nossa classe, um método que valida a senha do usuário. Esse método chama um outro método da própria classe que consulta o banco de dados que envia a senha criptografada. O método, ao receber o valor, descriptografa e retorna a senha do usuário. Por fim, o método que valida senha verifica se são iguais e retorna um resultado booleano. Caso o método responsável pela retorno da senha descriptografada tivesse visibilidade ilimitada, qualquer programador poderia se aproveitar desse método para descobrir a senha de todos os usuários de nosso sistema!

A linguagem Java permite que utilizemos qualificadores para determinar o nível de acesso de métodos ou variáveis membro:

Qualificador	classe	subclasse	Pacote	o mundo
Public	X	X	X	X
Private	X			
Protected	X	X	X	
Package	X		X	

3.8 OBJETOS EM JAVA

É através de objetos que (praticamente) todo o processamento ocorre em aplicações desenvolvidas com linguagens de programação orientadas a objetos. O uso racional de objetos, obedecendo aos bons princípios associados à sua definição conforme estabelecido no paradigma de desenvolvimento orientado a objetos, é chave para o desenvolvimento de bons sistemas de software.

Toda linguagem de programação orientada a objetos oferece mecanismos para definir os tipos de objetos para cada aplicação através do conceito de classes. Objetos são instâncias de classes; essas instâncias precisam ser criadas para que, através de sua manipulação, possam realizar seu trabalho. Após a conclusão de suas atividades, objetos podem ser removidos.

Arranjos de tipos primitivos ou de objetos são criados e manipulados de forma análoga a objetos.



O barato de uma linguagem orientada a objetos é que podemos criar classes como modelos e atribuímos a outra classe o método main para trabalhar com as classes criadas. Assim, a classe que possui o método main é a classe de interface com o usuário, e outras classes que são instanciadas pela classe de interface são os objetos de negócio ou aplicação.

3.8.1 Propriedades de objetos

Quando um objeto for ser utilizado em alguma aplicação (por outro objeto, normalmente), a única informação necessária que deve ser conhecida é "o que podemos fazer com ele?", ou seja, qual o conjunto de operações associadas àquele objeto. As técnicas de programação orientada a objetos recomendam que a estrutura interna de um objeto e a implementação de seus métodos devem ser tão privativos como possível.

Há dois conceitos básicos associados a essa visão de um objeto. O primeiro está associado ao fato de que cada objeto é uma unidade que contém, internamente, toda a informação necessária para sua manipulação. Este é o princípio da encapsulação -- cada componente de um programa deve agregar toda a informação relevante para sua manipulação como uma unidade (uma cápsula).

O segundo conceito fundamental associado a um bom objeto é o ocultamento da informação. Em outras palavras, quem usa o objeto não precisa nem deve conhecer como o objeto mantém sua informação ou como uma operação é implementada -- cada componente de um programa deve manter oculta, sob sua guarda, esses aspectos relativos a decisões de projeto.

Na orientação a objetos, cada objeto deve ser manipulado exclusivamente através dos métodos públicos do objeto, dos quais apenas a assinatura deve ser revelada. O conjunto de assinaturas desses métodos constitui a interface operacional da classe. O programador trabalha em um nível alto de abstração, sem preocupação com os detalhes internos da classe, simplificando a construção de programas que incorporam funcionalidades complexas, tais como interfaces gráficas e aplicações distribuídas.



3.8.2 Como criar um objeto em Java

Uma vez que a classe a partir da qual deseja-se criar o objeto exista, a criação do objeto dá-se através da aplicação do operador new:

```
new NomeDaClasse();
```

Essa expressão invoca o construtor da classe, um (pseudo-)método especial presente em todas as classes.

A aplicação do operador new ao construtor da classe retorna uma referência para o objeto. Para que o objeto possa ser efetivamente manipulado, essa referência deve ser armazenada por quem determinou a criação do objeto:

<pre>NomeDaClasse umaRef; umaRef = new NomeDaClasse();</pre>	<pre>NomeDaClasse umaRef = new NomeDaClasse();</pre>
--	--

Nesse exemplo, umaRef é uma variável que guarda uma referência para um objeto do tipo NomeDaClasse. As duas formas apresentadas são equivalentes.

3.8.3 Referência para o objeto

Objetos nunca são manipulados diretamente, mas sempre através de uma variável que contém uma referência para o objeto. Internamente, uma referência conterá o endereço para a área de memória que contém o objeto, mas isso é irrelevante sob o ponto de vista do programador.

Em função dessa característica na manipulação de objetos, a atenção deve ser redobrada quando a aplicação precisa comparar objetos; a cópia de objetos também é afetada por esses mesmos aspectos.

Deve-se observar que o escopo da referência ao objeto encerra-se com o fim do bloco no qual a referência é definida, mas a remoção do objeto só ocorre através do processo de coleta de lixo.

Uma referência especial é criada para que cada objeto possa poder referenciar a si próprio no corpo de seus métodos; esta referência é apontada pela variável implícita this.



3.8.4 Operador this

Quando um método é aplicado a um objeto, de algum modo deve ser sinalizado ao método a qual objeto a invocação está referindo-se. Por exemplo,

```
String s1 = "java";
```

```
String s2 = "linguagem de programação orientada a objetos";
```

```
int x1 = s1.length(); // 4
```

```
int x2 = s2.length(); // 44
```

Nesse exemplo, o método aplicado às duas strings é o mesmo -- length(). Como o método sabe qual o objeto que ele deve "olhar" para saber o que fazer?

Por trás da resposta a essa pergunta está a presença de um "parâmetro secreto", que é passado a todo método, que é uma referência ao objeto que está realizando a invocação. Em termos de uma linguagem procedimental, isso seria o equivalente a ter para as duas últimas linhas do exemplo acima:

```
int x1 = length(s1);
```

```
int x2 = length(s2);
```

Se necessário, o próprio método pode ter acesso a essa referência em seu código.

Essa referência é passada através da palavra-chave this, reservada para esse propósito.

Exemplos de uso de this

A palavra-chave this é utilizada principalmente em dois contextos:

Diferenciar atributos de objetos de parâmetros ou variáveis locais de mesmo nome;

Acessar o método construtor a partir de outros construtores.

Esse exemplo ilustra esses dois usos:

```
class ThisSample {  
    int x;  
    int y;  
  
    // exemplo do primeiro caso:  
    public ThisSample(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
    // exemplo do segundo caso:
```



```
public ThisSample( ) {  
    this(1, 1);  
}  
}
```

3.8.5 Manipulação de objetos

Quando declara-se uma variável cujo tipo é o nome de uma classe, como em :

```
String nome;
```

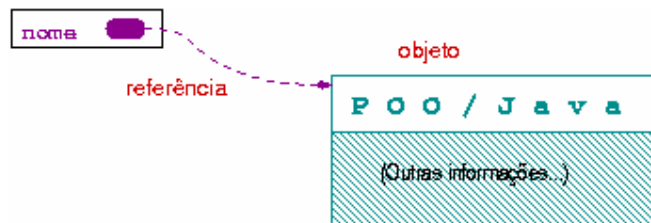
não está se criando um objeto dessa classe, mas simplesmente uma referência para um objeto da classe String, a qual inicialmente não faz referência a nenhum objeto válido:



Quando um objeto dessa classe é criado, obtém-se uma referência válida, que é armazenada na variável cujo tipo é o nome da classe do objeto. Por exemplo, quando cria-se uma string como em:

```
nome = new String("POO/Java");
```

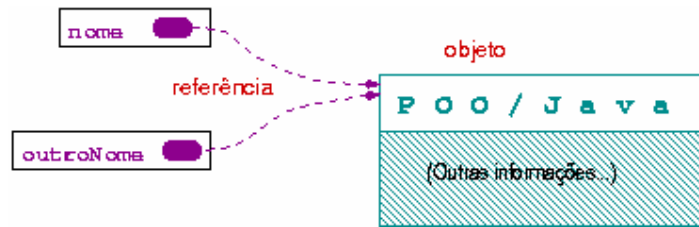
nome é uma variável que armazena uma referência para um objeto específico da classe String -- o objeto cujo conteúdo é "POO/Java":



É importante ressaltar que a variável nome mantém apenas a referência para o objeto e não o objeto em si. Assim, uma atribuição como

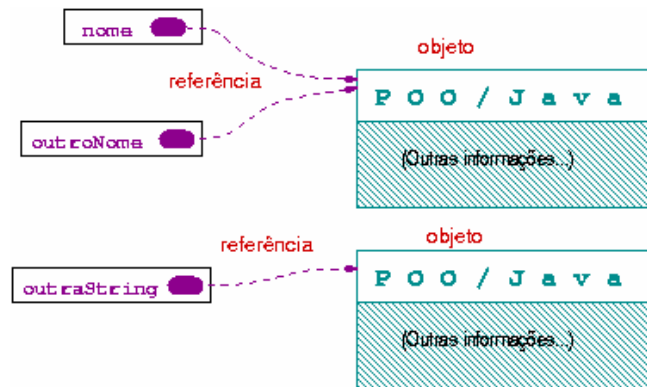
```
String outroNome = nome;
```

não cria outro objeto, mas simplesmente uma outra referência para o mesmo objeto:



O único modo de aplicar os métodos a um objeto é através de uma referência ao objeto. Seguindo com o mesmo exemplo, para criar um novo objeto como mesmo conteúdo do objeto existente, o método clone() pode ser aplicado a este:

```
String outraString = nome.clone();
```

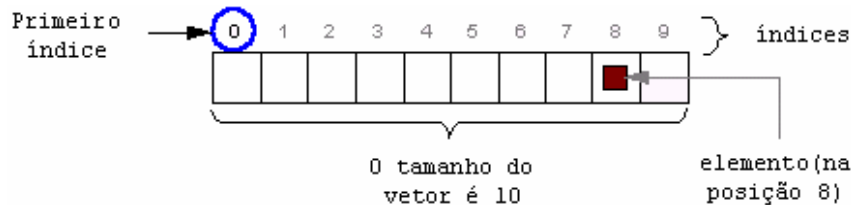


Em Java, a palavra-chave this é utilizada por um objeto para fazer a referência a si próprio.

3.9 VETORES EM JAVA

Vetores são importantes estruturas de dados em qualquer linguagem de programação. Consistem em uma estrutura de tamanho fixo que armazena múltiplos valores de um mesmo tipo. Assim podemos agrupar valores de mesmo tipo em um vetor. Vetores são suportados diretamente pela linguagem Java, pois não há uma classe Vetor. No entanto, são extensões implícitas da classe Object, logo podemos declarar uma variável do tipo Object e referenciar um vetor a ela.

Vetores são estruturas que guardam múltiplos valores de um mesmo tipo. Em Java, seu tamanho é determinado no momento de sua criação, ou seja, em tempo de execução. Após a criação, o vetor torna-se uma estrutura de tamanho fixo.



Sendo que assim como em C/C++, a primeira posição de memória sempre começa do 0.

3.9.1 Criação de Vetores em Java

Como declarações de variáveis de outros tipos, uma declaração de vetor tem dois elementos: o tipo do vetor e o nome do vetor. O tipo do vetor é definido da seguinte forma `tipo[]`, em que `tipo` indica o tipo do vetor e `[]` indica que é um vetor. Vale ressaltar que todos os valores de um vetor são do mesmo tipo do vetor. No exemplo acima, declaramos um vetor de inteiros cujo nome é `umVetor`. A seguir podemos observar outras declarações de vetores de outros tipos:

```
float[] umVetorDeFloats; // ou float umVetorDeFloats[];
boolean[] umVetorDeBooleans; // ou boolean umVetorDeBooleans[];
Object[] umVetorDeObjetos; // ou Object umVetorDeObjetos[];
String[] umVetorDeStrings; // ou String umVetorDeStrings[];
```

Após declaração é requerido espaço de memória para que a variável seja usada.

As variáveis vetores não fogem à regra, pois a declaração não implica em alocação de memória para armazenamento de elementos, assim, vetores, no mínimo, devem



ser criados e, se possível, inicializados antes de seu uso, embora a inicialização não seja obrigatória.

Podemos criar vetores explicitamente com o operador **new** conforme o exemplo em estudo que cria espaço de memória suficiente para dez inteiros e o referencia através do identificador `umVetor`:

```
int umVetor = new int[10]; // cria um vetor de inteiros
```

Em regra, devemos criar vetores através do operador **new**, indicando, após o operador, o tipo, e, entre os colchetes, indicando também o número de elementos que o vetor pode conter. Caso o vetor não seja criado antes de ser usado, o compilador Java lançará um erro.

Depois da instrução acima, alocamos o espaço de memória para usar o vetor. Assim podemos acessar os elementos do vetor e trabalhar com os valores nele armazenados.

Para referenciar um elemento de um vetor, tanto para armazenar como para recuperar o valor, devemos utilizar o nome do vetor e logo após, entre colchetes, indicar um índice válido para a posição que se encontra o elemento desejado. Por comodidade, Java permite que esse índice possa ser determinado com o resultado de uma expressão.

Para você saber qual o tamanho de um array [`arraySize`], utilizamos a seguinte propriedade: `length`

Ex:

```
for (int i = 0; i < meuArray.length; i++)  
{  
    meuArray[i] = i;  
    System.out.print(meuArray[i] + " ");  
}
```

3.9.1.1 Copiando vetores

Podemos copiar um vetor para outro usando o método `arraycopy` da classe `System`, que realiza tal operação de forma bastante eficiente. O método `arraycopy` necessita de cinco parâmetros passados na seguinte ordem: o objeto alvo da cópia, ou seja, o vetor a ser copiado; um índice do vetor a ser copiado que indica a partir de qual elemento será copiado; o objeto destino, isto é, o vetor que receberá os elementos copiados; um índice do vetor destino que indica a partir de qual posição o vetor receberá os elementos copiados; e, por último, um inteiro que define quantos elementos devem ser copiados do vetor alvo para o destino.

O exemplo abaixo utiliza o método `arraycopy`:

```
public class CopiandoVetores {
```



```
public static void main(String[] args) {  
  
    char[] vetorAlvo = { 'm', 'e', 'u', 'p', 'r', 'o', 'g', 'r', 'a', 'm', 'a', 'e', 'm', 'j', 'a', 'v', 'a'};  
    char[] vetorDestino = new char[8];  
  
    System.arraycopy(vetorAlvo, 3, vetorDestino, 0, 8);  
    System.out.println(new String(vetorDestino));  
}  
}
```

3.9.2 Matrizes em Java

Em java as matrizes seguem o mesmo conceito de vetores, ou seja uma matriz em java nada mais é do que um conjunto de vetores, que segue os mesmos parametros de um vetor como descrito anteriormente.

3.9.2.1 Criação de Matrizes

- Matrizes Bi-dimensionais

A criação de matrizes bi-dimensionais segue normal como a criação de um vetor adicionando-se apenas mais um par de colchetes.

Ex.: `int[][] mat = new int[5][5];`

- Matrizes Tri-dimensionais

Para se criar uma matriz tri-dimensional é só adicionar mais uma par de colchetes.

Ex.: `int mat_Tri[][][] = new int[5][5][5];`



3.9.3 Vetores e Matrizes de Objetos em Java

Em java a criação de vetores e Matrizes de objetos segue igual a criação de vetor e matrizes já descrito anteriormente, sendo na criação deve se referenciar o nome da classe após o operador new.

```
Ex.: Classe1 vet = new Classe1[10];  
ou  
Classe1 mat = new Classe1[10][10];
```

Obs.: É necessário frisar que em java a criação de um vetor ou uma matriz não implica na criação automática do vetor ou da matriz na memória, pois é necessário alocar cada posição do vetor ou da matriz objeto dinamicamente como podemos ver a seguir.

Ex.:

// Para vetores

```
Classe[] vet = new Classe[10];  
for(int o= 0;o<10;o++)  
{ vet[o] = new Classe(); }  
ou
```

// Para Matrizes

```
Classe[][] vet = new Classe[10][10];  
for(int o= 0;o<10;o++)  
{  
    for (int i= 0; i<10;i++)  
    { mat[o][i] = new Classe(); }  
}
```

3.9.4 Passando Vetores para métodos

Para passar um vetor para um método, especifique o nome do array sem nenhum colchete na hora de passar o vetor. Para um método receber um vetor através de uma chamada de métodos, a lista de parâmetros do método deve especificar um vetor como parâmetro.

```
Ex: void modificavetor(int b[]); // assinatura do método  
    int a = new int[20];  
    modificavetor(a); //chamada do método
```



3.10 HERANÇA

O conceito de encapsular estrutura e comportamento em um tipo não é exclusivo da orientação a objetos; particularmente, a programação por tipos abstratos de dados segue esse mesmo conceito. O que torna a orientação a objetos única é o conceito de herança.

Herança é um mecanismo que permite que características comuns a diversas classes sejam fatoradas em uma classe base, ou superclasse. A partir de uma classe base, outras classes podem ser especificadas. Cada classe derivada ou subclasse apresenta as características (estrutura e métodos) da classe base e acrescenta a elas o que for definido de particularidade para ela.

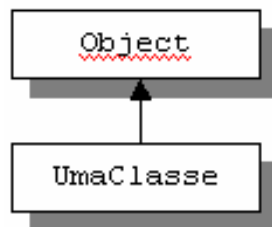
Sendo uma linguagem de programação orientada a objetos, Java oferece mecanismos para definir classes derivadas a partir de classes existentes. É fundamental que se tenha uma boa compreensão sobre como objetos de classes derivadas são criados e manipulados, assim como das restrições de acesso que podem se aplicar a membros de classes derivadas. Também importante para uma completa compreensão da utilização desse mecanismo em Java é a compreensão de como relacionam-se interfaces e herança.

Herança é sempre utilizada em Java, mesmo que não explicitamente. Quando uma classe é criada e não há nenhuma referência à sua superclasse, implicitamente a classe criada é derivada diretamente da classe Object. É por esse motivo que todos os objetos podem invocar os métodos da classe Object, tais como equals() e toString().

Todas as classes que criamos em Java provêm de uma classe maior, a superclasse de todas as classes, a classe Object.

A classe Object define e implementa os comportamentos que toda a classe precisa em Java. Assim, quando declaramos uma classe qualquer estamos implicitamente realizando a herança da classe Object, como ilustra o exemplo:

```
public class UmaClasse {  
    public UmaClasse(){  
        System.out.println("Uma nova classe foi criada! ");  
    }  
}
```



Sintaxe



A forma básica de herança em Java é a extensão simples entre uma superclasse e sua classe derivada. Para tanto, utiliza-se na definição da classe derivada a palavra-chave *extends* seguida pelo nome da superclasse.

Assim, definir uma classe Ponto2D como neste exemplo :

```
class Ponto2D {
    private int x;
    private int y;

    public Ponto2D(int x, int y) {
        this.x = x;
        this.y = y;
    }
    public Ponto2D( ) {
        this(0,0);
    }

    public double distancia(Ponto2D p) {
        double distX = p.x - x;
        double distY = p.y - y;
        return(Math.sqrt(distX*distX + distY*distY));
    }
}

class Ponto3D extends Ponto2D {
    private int z;

    public Ponto3D(int x, int y, int z) {
        super(x, y);
        this.z = z;
    }
    public Ponto3D( ) {
        z = 0;
    }
    public static void main(String[] args) {
        Ponto2D ref2 = new Ponto2D();
        Ponto2D p2 = new Ponto2D(1,1);
        System.out.println("Distancia2: " + p2.distancia(ref2));
        Ponto3D p3 = new Ponto3D(1,2,3);
        System.out.println("Distancia3: " + p3.distancia(ref2));
    }
}
```

Este exemplo ilustra como uma classe Ponto2D pode ser utilizada como base para a definição de outra classe derivada Ponto3D através do mecanismo de extensão simples.



3.10.1 Construção de objetos derivados

Durante a construção de um objeto de uma classe derivada, o construtor de sua superclasse é executado (implicitamente ou explicitamente) antes de executar o corpo de seu construtor.

Assim, ao se construir um objeto de uma classe derivada, o método construtor da superclasse será inicialmente invocado. Este, por sua vez, invocará o construtor de sua superclasse, até que o construtor da classe raiz de toda a hierarquia de objetos -- a classe Object -- seja invocado. Como Object não tem uma superclasse, seu construtor é executado e a execução retorna para o construtor de sua classe derivada. Então executa-se o restante do construtor de sua classe derivada e a execução retorna para o construtor de sua classe derivada e assim sucessivamente, até que finalmente o restante do construtor da classe para a qual foi solicitada a criação de um objeto seja executada.

3.10.2 A palavra-chave super

Java não oferece o mecanismo de herança múltipla, ou seja, não é possível criar uma classe derivada com mais de uma classe base. Por esse motivo, é simples fazer uma referência da classe derivada para sua superclasse; o mecanismo para tal é o uso da palavra-chave super.

Construtores da superclasse podem ser explicitamente invocados usando o método super(). No exemplo do Ponto3D, isso é feito para o construtor da classe derivada com argumentos:

```
public Ponto3D(int x, int y, int z) {  
    super(x, y);  
    this.z = z;  
}
```

A expressão super(x, y); na primeira linha do construtor está invocando o construtor da classe base, Ponto2D, que recebe dois argumentos. O efeito dessa invocação é iniciar os valores dos atributos x e y do objeto com os valores dos parâmetros x e y recebidos pelo método construtor.

A invocação do método super(), se presente, deve estar na primeira linha. Implicitamente, o compilador faz a invocação do construtor super() default (sem argumentos) para cada construtor definido. Por esse motivo, o segundo construtor da classe Ponto3D não faz a invocação desse método explicitamente. Assim,



```
public Ponto3D( ) {  
    z = 0;  
}
```

equivale a

```
public Ponto3D( ) {  
    super();  
    z = 0;  
}
```

Esse é um dos motivos pelo qual é sempre interessante ter o construtor default definido. Entretanto, a invocação direta pode ser interessante quando se deseja invocar algum construtor que não o default, como no exemplo anterior.

Outro uso dessa palavra-chave é como prefixo para referenciar métodos da super-classe.

3.10.3 Restrições de acesso

Neste ponto será reforçado o conceito explicado no ponto 3.8.9

O exemplo da classe Ponto3D ilustra a forma básica de herança -- uma vez que uma classe tenha sido definida como derivada de outra, ela preserva da superclasse os atributos (no caso, os valores das coordenadas x e y) e seus métodos (no caso, o método distancia(Ponto2D) aplicado a um Ponto3D).

No entanto, deve ser observado que nenhum método definido para a classe Ponto3D pode ter acesso aos atributos x e y, pois esses são declarados como private da classe Ponto2D. De fato, caso tente-se definir naquela classe um método para calcular a distancia entre dois pontos no espaço, tal como:

```
public double distancia(Ponto3D p) {  
    double distX = p.x - x;  
    double distY = p.y - y;  
    double distZ = p.z - z;  
    return(Math.sqrt(distX*distX + distY*distY + distZ*distZ));  
}
```

os seguintes erros seriam detectados pelo compilador Java:

Ponto3Derr.java:35: No variable x defined in class Ponto3D.

```
double distX = p.x - x;  
                  ^
```

Ponto3Derr.java:35: Undefined variable: x

```
double distX = p.x - x;  
                  ^
```

Ponto3Derr.java:36: No variable y defined in class Ponto3D.



```
double distY = p.y - y;  
      ^
```

Ponto3Derr.java:36: Undefined variable: y

```
double distY = p.y - y;  
      ^
```

ou seja, mesmo embora a classe Ponto3D tenha os atributos x e y, métodos dessa classe não podem manipulá-los diretamente.

Uma alternativa para permitir que classes derivadas possam manipular atributos da superclasse é declarar tais atributos como `protected` ao invés de `private`. Assim, objetos de classes derivadas têm acesso tanto de leitura como de escrita para esses atributos:

```
public class Ponto2D {  
    protected int x;  
    protected int y;  
    ... }  
}
```

Caso deseje-se dar acesso apenas de leitura a esses atributos, uma outra alternativa é usar um método acessor para cada atributo, ou seja, um método que simplesmente retorna o valor do atributo. Nesse exemplo, a classe Ponto2D definiria dois métodos,

```
public int getX() {  
    return x; }  
  
public int getY() {  
    return y; }  
}
```

que permitem que qualquer objeto de qualquer classe consiga ler os valores de x e y -- mas não alterá-los. Assim, a classe Ponto3D poderia definir o método para a distância no espaço como

```
public double distancia(Ponto3D p) {  
    double distX = p.getX() - this.getX();  
    double distY = p.getY() - this.getY();  
    double distZ = p.z - z;  
    return(Math.sqrt(distX*distX + distY*distY + distZ*distZ));  
}
```

que compilaria e executaria sem erros.

Pode-se também permitir acesso de apenas leitura para as classes derivadas (ou do mesmo pacote) mas não para as demais classes -- nesse caso, os métodos acessores seriam declarados como `protected`, como em

```
protected int getX() {  
    return x;  
}  
protected int getY() {
```



```
}  
    return y;  
}
```

O método `distancia(Ponto3D)` não precisaria sofrer nenhuma modificação, mas outras classes não-derivadas de `Ponto2D` não teriam mais como invocar esses métodos acessores.

3.10.4 Herança e interfaces

Assim como ocorre com classes, interfaces Java também podem ser derivadas de outras interfaces -- a interface derivada declara métodos que devem ser adicionalmente definidos por classes que implementem a interface. Uma interface derivada pode estender diversas interfaces:

```
interface S1 { ... }  
interface S2 { ... }  
interface D extends S1, S2 { ... }
```

Enquanto Java limita que uma classe pode estender apenas uma superclasse, a linguagem não impõe limites com relação a quantas interfaces podem ser estendidas por outra interface ou implementadas por uma classe.

Quando mais de uma interface está sendo implementada, a sintaxe de declaração da classe especifica que as interfaces implementadas são separadas por vírgulas:

```
class Derivada extends SuperClasse  
    implements Interface1, Interface2 {  
    ...  
}
```

Neste exemplo, a classe `Derivada` deve implementar todos os métodos da `Interface1` e todos os métodos da `Interface2`. Adicionalmente, se `SuperClasse` também tiver métodos abstratos, esses também deverão ser implementados por métodos da classe `Derivada`.

3.10.5 Interfaces

Em java, as heranças só podem ocorrer de forma restrita, isto é, uma superclasse pode ter várias subclasses, mas uma subclasse só pode ter uma superclasse. Desta forma uma subclasse herda somente atributos e comportamentos da classe pai ou de seu ancestral e mudanças de comportamentos de outras classes devem ser implementadas



na própria classe que, por sua vez, pode ou não transmitir os novos comportamentos para seus descendentes.

Entretanto, algumas vezes seria conveniente que herdássemos comportamentos de duas ou mais classes. Essa técnica, denominada herança múltipla, não é permitida em java, uma vez que traz muitos problemas nas linguagens que a implementaram.

Assim em Java foi criada uma forma mais elegante e menos complicada de adicionar comportamentos a classes já herdadas, as interfaces.

Interface são classes abstratas que só possuem métodos abstratos.

3.10.5.1 Definindo Interfaces

Interfaces são na realidade um protocolo de comportamentos que podem ser implementados por qualquer classe em qualquer hierarquia. As interfaces apenas definem os comportamentos que serão implementados nas classes e as classes que implementarem as interfaces aceitam que deverão implementar os métodos definidos.

Por exemplo suponhamos que queremos utilizar um comportamento potencial que possa ser utilizado por qualquer classe. Assim um desses comportamentos poderia ser:

```
public class aeroporto {  
    ...  
    public void prepararPouso(Voador v){  
        System.out.println("Aterrisagem permitida!");  
        v.pousar();  
    }  
    ...  
}
```

Nossa classe deve aceitar qualquer objeto voador que possa atender nosso protocolo, que é o comportamento pousar. Daí podemos definir uma interface que para as classes que desejam pousar em nosso aeroporto.

```
public interface Voador {  
    public void pousar();  
}
```

A interface acima define o protocolo de comportamentos, mas não o implementa. A implementação deve ser realizada pelas classes que desejarem pousar no aeroporto.

As interfaces só podem ser definida como public ou *package*(este tópico será vista mais a frente). Este último caso se configura quando a interface vier sem o modificador



public. Também deve ter pelo menos um nome precedido de seu tipo (interface). E, ainda pode herdar outras interfaces já definidas através da palavra reservada `extends`.

O corpo da interface só pode ter definições de métodos a serem implementados e constantes que são implicitamente rotuladas pelos modificadores `public`, `static` e `final`. As constates não podem ter qualquer outro modificador sem ser os citados, isto é, os modificadores `private` e `protected` não são permitidos em interfaces.

3.10.5.2 Implementado interfaces

Para implementar uma ou mais interfaces previamente definida basta que utilizemos a palavra reservada `implements` e as interfaces que desejamos implementar. Assim, se quisermos implementar um ponto voador para pousar em nosso aeroporto bastar definirmos a nova classe dizendo que seguiremos o protocolo de comportamento da interface.

```
public class PontoVoador extends Ponto2D implements Voador{  
    public void pousar() {  
        System.out.ponto("Aterrisagem bem sucedida!");  
    }  
}
```

Portanto, podemos instanciar qualquer classe que implementa o comportamento e passarmos o parâmetro que o aeroporto aceitará o pouso do objeto. Isso representa grandes vantagens para a comunicação entre classes uma vez que a classe pode estar esperando comportamento conforme o protocolo. Também podemos tirar vantagens da interface uma vez que a implemetação pode ser adicionada a qualquer classe em qualquer nível da árvore. Talvez a única desvantagem das interfaces é que elas não deveriam ser modificadas uma vez que todas as classes que implementarem a interface deverão atender às modificações realizadas.

3.11 POLIMORFISMO EM JAVA

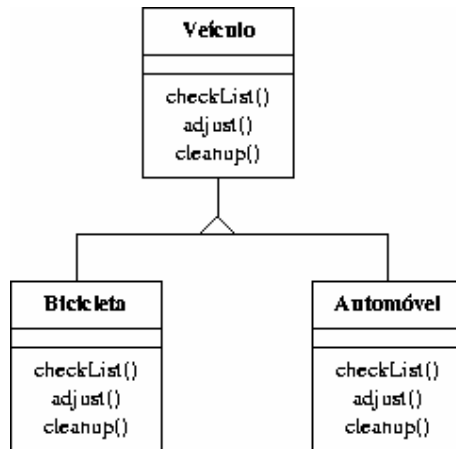
Java suporta tanto uma hierarquia de subclasses (através da palavra-chave `extends`) quanto uma hierarquia de subtipos separada (através do uso de interfaces). Variáveis podem ser declaradas através de uma classe ou de uma interface. Todas as variáveis são polimórficas, pois uma variável declarada como um objeto de uma classe pode armazenar valores de qualquer subclasse, e uma variável declarada como uma interface pode armazenar valores de qualquer classe que implemente a interface.

Sobrecarga ocorre em Java quando métodos em uma mesma classe ou hierarquia de classes compartilham o mesmo nome e tipo de retorno. Métodos com a mesma assi-

natura (nome e lista de argumentos idênticos) podem ocorrer desde que estejam em classe distintas na hierarquia de classes. Se métodos em uma mesma classe possuem o mesmo nome, eles devem possuir uma lista de argumentos distinta daquelas dos demais métodos de mesmo nome, seja pelo tipo ou ordem dos argumentos. Finalmente, não é possível termos métodos com a mesma assinatura e tipo de retorno distinto em uma mesma classe ou hierarquia de classes.

Java não possui a capacidade para definir tipos genéricos. Esta capacidade está disponível na linguagem C++ e é exaustivamente explorada em sua biblioteca de containers, a Standard Template Library (STL).

Considere uma classe Veículo com duas classes derivadas, Automóvel e Bicicleta:



Essas classes têm três métodos, definidos para veículos de forma geral e redefinidos mais especificamente para automóveis e bicicletas:

checkList(), para verificar o que precisa ser analisado no veículo;
adjust(), para realizar os reparos e a manutenção necessária; e
cleanup(), para realizar procedimentos de limpeza do veículo.

A aplicação Oficina define um objeto que recebe objetos da classe Veículo. Para cada veículo recebido, a oficina executa na sequência os três métodos da classe Veículo. No entanto, não há como saber no momento da programação se a Oficina estará recebendo um automóvel ou uma bicicleta -- assim, o momento de decisão sobre qual método será aplicado só ocorrerá durante a execução do programa.

```
import java.util.*;

class Veiculo {
    public Veiculo() {
        System.out.print("Veiculo ");
    }
    public void checkList() {
```




```
        System.out.println("Veiculo.checkList");
    }
    public void adjust() {
        System.out.println("Veiculo.adjust");
    }
    public void cleanup() {
        System.out.println("Veiculo.cleanup");
    }
}
```

```
class Automovel extends Veiculo {
    public Automovel() {
        System.out.println("Automovel");
    }
    public void checkList() {
        System.out.println("Automovel.checkList");
    }
    public void adjust() {
        System.out.println("Automovel.adjust");
    }
    public void cleanup() {
        System.out.println("Automovel.cleanup");
    }
}
```

```
class Bicicleta extends Veiculo {
    public Bicicleta() {
        System.out.println("Bicicleta");
    }
    public void checkList() {
        System.out.println("Bicicleta.checkList");
    }
    public void adjust() {
        System.out.println("Bicicleta.adjust");
    }
    public void cleanup() {
        System.out.println("Bicicleta.cleanup");
    }
}
```

```
public class Oficina {
    Random r = new Random();

    public Veiculo proximo() {
        Veiculo v;
        int code = r.nextInt();
    }
}
```



```
        if (code%2 == 0)
            v = new Automovel();
        else
            v = new Bicicleta();

        return v;
    }

    public void manter(Veiculo v) {
        v.checkList();
        v.adjust();
        v.cleanup();
    }

    public static void main(String[] args) {
        Oficina o = new Oficina();
        Veiculo v;

        for (int i=0; i<4; ++i) {
            v = o.proximo();
            o.manter(v);
        }
    }
}
```

Este trecho de código ilustra a utilização da classe Oficina, considerando que os métodos acima foram definidos para Veículo e para todas as suas classes derivadas. Um possível resultado da interpretação dessa aplicação é:

```
Veiculo Bicicleta
Bicicleta.checkList
Bicicleta.adjust
Bicicleta.cleanup
Veiculo Bicicleta
Bicicleta.checkList
Bicicleta.adjust
Bicicleta.cleanup
Veiculo Automovel
Automovel.checkList
Automovel.adjust
Automovel.cleanup
Veiculo Automovel
Automovel.checkList
Automovel.adjust
Automovel.cleanup
```



Alguns pontos a destacar com relação a esse exemplo:

Upcasting

O método `Oficina.proximo()` realiza uma atribuição de um objeto `Automóvel` à variável (referência para objeto `Veículo`) `v` quando o valor do número aleatório gerado é par (o resto da divisão inteira por 2 é igual a 0). Essa atribuição de um objeto de uma classe mais especializada para uma referência de uma classe ancestral é denominada `upcast`.

Esse mesmo tipo de atribuição é realizado de `Bicicleta` para `veículo` quando o número aleatório gerado é ímpar.

Uso de métodos abstratos

Apesar de métodos da classe `Veículo` terem sido definidos, estes nunca são invocados nesse exemplo. (Se fossem, algo estaria errado.) Isso ilustra uma situação onde métodos abstratos poderiam ser utilizados, pois a definição do corpo desses métodos é de fato irrelevante. Ainda mais, se uma classe como essa só contém métodos abstratos, ela poderia ser implementada como uma interface Java.

3.11.1 Sobrescrevendo métodos

Vimos que uma subclasse pode esconder variáveis membro visíveis de sua superclasse. E com métodos, também podemos escondê-los? A resposta para questão é afirmativa.

A habilidade de uma subclasse esconder um comportamento denomina-se `sobrescrição de método`.

Por exemplo, todas as classes são descendentes de `Object` e por isso herdam seus métodos e atributos. Existe um método em `Object` denominado `equals(Object obj)` cuja função é comparar dois objetos. Sabemos que quando trabalhamos com Objetos trabalhamos com referências e quando passamos parâmetros, os passamos por valor. Dito isso, o parâmetro passado resulta no valor de uma referência que Java utiliza na comparação de objetos. Resumindo, a função mencionada faz a mesma coisa que o operador de comparação `=`.

Mas se ao invés de compararmos referências, quiséssemos comparar todos os atributos do objeto para verificar se são iguais ainda que suas referências apontem para objetos diferentes. A solução é sobrescrever o método para adequar à realidade:

```
public class Ponto2D {  
    int x, y;  
    public Ponto2D() {
```



```
x = 0;  
y = 0;  
}  
public boolean equals(Object obj){  
    Ponto2D temp = (Ponto2D) obj;  
    return( (x == temp.x) & (y == temp.y) );  
}  
}
```

Mesmo sobrescrevendo um método ainda podemos usa-lo na subclasse que o sobrescreveu utilizando a palavra reservada `super` para invocar o método original.

Métodos restritos por visibilidade à subclasse, declarados como finais e métodos de classe não podem ser sobrescritos. Métodos abstratos devem ser obrigatoriamente sobrescritos.

3.11.2 Sobrecarregando métodos

Um outro conceito de orientação a objetos muito aplicado nas linguagens de programação é o polimorfismo. O polimorfismo é realizado em Java através de sobrecarga.

Sobrecarregar um método é declarar um novo método com o mesmo de nome de um método existente com número de argumentos ou tipos de argumentos diferentes, mesmo que o método existente seja declarado na classe ou superclasse, se tratar de subclasse.

```
public class Ponto2D {  
  
    int x, y;  
  
    public Ponto2D() {  
        x = 0;  
        y = 0;  
    }  
  
    public boolean equals(Ponto2D obj){  
        return( (x == obj.x) & (y == obj.y) );  
    }  
}
```



3.12 PACOTES

No desenvolvimento de pequenas atividades ou aplicações, é viável manter o código da aplicação e suas classes associadas em um mesmo diretório de trabalho — em geral, o diretório corrente. No entanto, para grandes aplicações é preciso organizar as classes de maneira a evitar problemas com nomes duplicados de classes e permitir a localização do código da classe de forma eficiente. Em Java, a solução para esse problema está na organização de classes e interfaces em pacotes.

Um pacote é uma unidade de organização de código que congrega classes, interfaces e exceções relacionadas. O código-base de Java está todo estruturado em pacotes e as aplicações desenvolvidas em Java também devem ser assim organizadas.

Essencialmente, uma classe *Xyz* que pertence a um pacote *nome.do.pacote* tem um “nome completo” que é *nome.do.pacote.Xyz*. Assim, se outra aplicação tiver uma classe de mesmo nome não haverá conflitos de resolução, pois classes em pacotes diferentes têm nomes completos distintos.

A organização das classes em pacotes também serve como indicação para o compilador Java para encontrar o arquivo que contém o código da classe. O ambiente Java normalmente utiliza a especificação de uma variável de ambiente *CLASSPATH*, a qual define uma lista de diretórios que contém os arquivos de classes Java. No entanto, para não ter listas demasiadamente longas, os nomes dos pacotes definem subdiretórios de busca a partir dos diretórios em *CLASSPATH*.

Ao encontrar no código uma referência para a classe *Xyz*, o compilador deverá procurar o arquivo com o nome *Xyz.class*; como essa classe faz parte do pacote *nome.do.pacote*, ele irá procurar em algum subdiretório *nome/do/pacote*. Se o arquivo *Xyz.class* estiver no diretório */home/java/nome/do/pacote*, então o diretório */home/java* deve estar incluído no caminho de busca de classes definido por *CLASSPATH*.

Para indicar que as definições de um arquivo fonte Java fazem parte de um determinado pacote, *package nome.do.pacote*; Caso tal declaração não esteja presente, as classes farão parte do “pacote default”, que está mapeado para o diretório corrente.

Para referenciar uma classe de um pacote no código fonte, é possível sempre usar o “nome completo” da classe; no entanto, é possível também usar a declaração *import*.

Por exemplo, se no início do código estiver presente a declaração

```
import nome.do.pacote.Xyz;
```

então a classe *Xyz* pode ser referenciada sem o prefixo *nome.do.pacote* no restante do código.

Alternativamente, a declaração

```
import nome.do.pacote.*;
```

indica que quaisquer classes do pacote especificado podem ser referenciadas apenas pelo nome no restante do código fonte.



A única exceção para essa regra refere-se às classes do pacote `java.lang` — essas classes são consideradas essenciais para a interpretação de qualquer programa Java e, por este motivo, o correspondente `import` é implícito na definição de qualquer classe Java.

3.12.1 Pacotes e Diretórios

Para a implementação atual de Java, as unidades de compilação de um pacote devem estar no mesmo diretório do sistema operacional. De fato, o nome (identificador) do pacote deve ser parte do nome deste diretório. Usando o sistema UNIX como exemplo, podemos definir um pacote chamado `aulas.aula9` cujos arquivos estejam no diretório `/home/java/public_html/graduacao/aulas/aula9` assumindo que o compilador Java foi informado para procurar pacotes embaixo do diretório `/home/java/public_html/graduacao`.

Apesar de um pacote corresponder a vários arquivos no mesmo diretório, note que se o diretório contém subdiretórios, estes não correspondem a "subpacotes". Por exemplo, um pacote chamado `aulas.aula9.bancoar` não tem nenhuma relação direta com o pacote `aulas.aula9`, apesar dos arquivos do primeiro estarem num subdiretório do diretório onde estão os arquivos do segundo.

3.12.2 Ocultando Informações

Pacotes são especialmente úteis para ocultar informações. Com este intuito, Java permite que as definições de métodos, atributos, construtores e classes sejam acompanhadas de qualificadores especiais.

Se um atributo, método, construtor ou uma classe for declarado `public`, ele poderá ser utilizado em qualquer lugar.

Se um atributo, método ou construtor for declarado `private`, ele só poderá ser utilizado (referenciado) na definição da classe onde o mesmo é definido.

Se um atributo, método ou construtor for declarado `protected`, ele só poderá ser utilizado no pacote onde o mesmo é definido ou na definição de uma subclasse da classe onde o mesmo é definido.

Se uma atributo, método, construtor ou uma classe não for declarado com um dos qualificadores discutidos acima, ele só poderá ser utilizado no pacote onde o mesmo é definido.

Por exemplo, veja como usar os qualificadores acima:

```
public class Banco {  
    private Conta[]    contas;  
    ...  
}
```



```
public void cadastro(Conta c) {...}  
...  
protected Conta procura(int num) {...}  
...  
public Banco () {...}  
}
```

3.12.3 Importação de Pacotes

As declarações feitas em uma unidade de compilação são visíveis em qualquer outra unidade do mesmo pacote, a menos que elas sejam declaradas privadas, como veremos mais adiante.

Além disso, qualquer unidade de compilação de um pacote pode usar definições de outros pacotes, desde que elas sejam visíveis. Assim, se em uma unidade de compilação do pacote segundo.pacote for preciso usar o tipo (classe ou interface, juntamente com todas operações e atributos visíveis) Tipo definido no pacote primeiro.pacote temos que fazer as seguinte declarações:

```
package segundo.pacote;  
...  
import primeiro.pacote.Tipo;  
...
```

o que permite que o nome Tipo seja usado como sinônimo para primeiro.pacote.Tipo

dentro da unidade de compilação contendo as declarações acima. Dessa forma note que o pacote segundo.pacote não poderá ter a definição de um tipo com o nome Tipo.

Alternativamente, pode-se usar as declarações

```
package segundo.pacote;  
...  
import primeiro.pacote.*;  
...
```

para importar todos os tipos públicos declarados no pacote primeiro.pacote.

É importante notar que importação de pacotes não é transitiva nem distribui sobre as unidades de compilação de um pacote. Por exemplo, considere a seguinte unidade de compilação

```
package terceiro.pacote;  
...
```



```
import segundo.pacote.*;  
...
```

Se nesta unidade for preciso utilizar o tipo Tipo definido no pacote primeiro.pacote, temos que adicionar a seguinte declaração

```
...  
import primeiro.pacote.Tipo;  
...
```

à unidade, mesmo que primeiro.pacote.Tipo tenha sido importado por segundo.pacote. Além disso, se na unidade de compilação

```
package terceiro.pacote;  
...
```

precisa-se usar alguma definição do pacote segundo.pacote temos que incluir a seguinte declaração

```
...  
import segundo.pacote.*;  
...
```

mesmo que esta declaração já tenha sido incluída em outra unidade de compilação do pacote terceiro.pacote.

3.12.4 Nomeando Pacotes e Unidades de Compilação

Cada unidade de compilação só pode definir um tipo público. De fato, para a implementação atual de Java, as unidades de compilação devem ser nomeadas de acordo com o seguinte padrão: `NomedoTipoPúblico.java`.

Para facilitar a identificação universal de pacotes que podem ser usados remotamente, estes devem ser nomeados usando-se o endereço Internet de onde o pacote está localizado. Por exemplo, `BR.di.ufpe.segundo.pacote` deveria ser usado ao invés de simplesmente `segundo.pacote`.

3.13 EXCEÇÕES EM JAVA

Todo programa está sujeito a pontenciais problemas de execução, como problemas de hardware, de quebras de disco, de espaço insuficiente em memória, acesso indevido à memória, entre outros. Quando um desses potenciais problemas acontecem, java cria um objeto e manipula esse objeto para o sistema em tempo de execução. Esse obje-



to contém uma série de informações que podem ser úteis para que o sistema em execução possa continuar sem problemas. Assim, toda vez que ocorre um problema que pode ser previsto e manipulado no qual é criado um objeto de exceção denomina-se lançamento de exceção.

Uma exceção é um sinal que indica que algum tipo de condição excepcional ocorreu durante a execução do programa. Assim, exceções estão associadas a condições de erro que não tinham como ser verificadas durante a compilação do programa.

As duas atividades associadas à manipulação de uma exceção são:

geração

A sinalização de que uma condição excepcional (por exemplo, um erro) ocorreu, e

captura

A manipulação (tratamento) da situação excepcional, onde as ações necessárias para a recuperação da situação de erro são definidas.

Para cada exceção que pode ocorrer durante a execução do código, um bloco de ações de tratamento (um exception handler) deve ser especificado. O compilador Java verifica e força que toda exceção "não-trivial" tenha um bloco de tratamento associado.

O mecanismo de manipulação de exceções em Java, embora apresente suas particularidades, teve seu projeto inspirado no mecanismo equivalente de C++, que por sua vez foi inspirado em Ada. É um mecanismo adequado à manipulação de erros síncronos, para situações onde a recuperação do erro é possível.

A filosofia de tratamento de exceções consiste no seguinte: se não puder resolver o problema passe para quem possa!

Java possui diferentes tipos de exceções: de entrada e saída, de tempo de execução e exceções que podem ser criadas pelo programador. As exceções, em regra, são sempre verificadas, uma vez que é necessário informar ao programador problemas potenciais que podem ocorrer durante a execução de um programa permitindo ao programador escolher a melhor forma de tratá-lo. Por exemplo, caso o arquivo no programa acima não puder ser aberto ou não existir, o que o programa deveria fazer? Escolher outro arquivo para você?

Como podemos perceber, essa decisão não cabe à API java, ela é exclusiva do programador. Entretanto existem outros tipos de exceções que não são verificadas, são as exceções de tempo de execução derivadas da classe RuntimeException. Estas são exceções muito comuns a diversas operações como a de acesso a referências nulas.

Tais exceções, embora possam ser capturadas, não são verificadas pelo compilador, uma vez que o processo de verificação é muito custoso, o projeto da linguagem optou por ignorá-las em tempo de compilação.



3.13.1 Capturando e tratando exceções

A sinalização da exceção é propagada a partir do bloco de código onde ela ocorreu através de toda a pilha de invocações de métodos até que a exceção seja capturada por um bloco manipulador de exceção. Eventualmente, se tal bloco não existir em nenhum ponto da pilha de invocações de métodos, a sinalização da exceção atinge o método `main()`, fazendo com que o interpretador Java apresente uma mensagem de erro e aborte sua execução.

Blocos try-catch

A captura e o tratamento de exceções em Java se dá através da especificação de blocos `try`, `catch` e `finally`, definidos através destas mesmas palavras reservadas da linguagem. Um comando `try/catch/finally` obedece à seguinte sintaxe:

```
try {  
    // código que inclui comandos/invocações de métodos  
    // que podem gerar uma situação de exceção.  
}  
catch (XException x) {  
    // bloco de tratamento associado à condição de  
    // exceção XException ou a qualquer uma de suas  
    // subclasses, identificada aqui pelo objeto  
    // com referência x  
}  
catch (YException y) {  
    // bloco de tratamento para a situação de exceção  
    // YException ou a qualquer uma de suas subclasses  
}  
finally {  
    // bloco de código que sempre será executado após  
    // o bloco try, independentemente de sua conclusão  
    // ter ocorrido normalmente ou ter sido interrompida  
}
```

Os blocos não podem ser separados por outros comandos -- um erro de sintaxe seria detectado pelo compilador Java neste caso. Cada bloco `try` pode ser seguido por zero ou mais blocos `catch`, onde cada bloco `catch` refere-se a uma única exceção.

O bloco `finally` é sempre executado. Em geral, ele inclui comandos que liberam recursos que eventualmente possam ter sido alocados durante o processamento do bloco `try` e que podem ser liberados, independentemente de a execução ter encerrado com sucesso ou ter sido interrompida por uma condição de exceção. A presença desse bloco é opcional.



Exemplo

Para tornar o exemplo anterior funcional, é preciso tratar a exceção que pode ser gerada na execução da linha 6. Para tanto, uma possibilidade é substituir essa linha por

```
try {  
    System.in.read(tB);  
}  
catch (IOException e) {  
    System.err.println(e); }
```

Nesse caso, o tratamento dado para a ocorrência do erro foi simplesmente apresentar na saída padrão de erros uma mensagem associada à exceção ocorrida. O programa completo fica:

```
1 import java.io.*;  
2  
3 public class ConvInt2 {  
4     public String leLinha() {  
5         byte[] tB = new byte[20];  
6         try {  
7             System.in.read(tB);  
8         }  
9         catch (IOException e) {  
10             System.err.println(e);  
11         }  
12         String tS = new String(tB).trim();  
13         return tS;  
14     }  
15  
16     public int leInt() {  
17         String s = leLinha();  
18         return Integer.parseInt(s);  
19     }  
20  
21     public static void main(String[] args) {  
22         ConvInt2 ci = new ConvInt2();  
23         System.out.print("Entre inteiro: ");  
24         int valor = ci.leInt();  
25         System.out.println("Valor lido foi: " + valor);  
26     }  
27 }
```



3.13.2 Propagando exceções

Embora toda exceção deva ser tratada, nem sempre é possível tratar uma exceção no mesmo escopo do método cuja invocação gerou a exceção. Nessas situações, é possível propagar a exceção para um nível acima na pilha de invocações.

Para tanto, o método que está deixando de capturar e tratar a exceção faz uso da cláusula `throws` na sua declaração:

```
void métodoQueNãoTrataExceção() throws Exception {  
    invoca.métodoQuePodeGerarExceção();  
}
```

Nesse caso, `métodoQueNãoTrataExceção()` reconhece que em seu corpo há a possibilidade de haver a geração de uma exceção mas não se preocupa em realizar o tratamento dessa exceção em seu escopo. Ao contrário, ele repassa essa responsabilidade para o método anterior na pilha de chamadas.

Eventualmente, também o outro método pode repassar a exceção adiante. Porém, pelo menos no `main()` as exceções deverão ser tratadas.

Exemplo

A seguir, o exemplo da aquisição de um valor inteiro a partir da entrada padrão é reapresentado com o uso da cláusula `throws`. Nesse caso, `throws` tem que ser declarado em dois métodos. Primeiro no método `leLinha`, que invoca `read()` e não faz o tratamento de `IOException`. Segundo, no método `leInt`, que pode receber o `IOException` de `leLinha` mas também não faz seu tratamento. Para ilustrar, nesse segundo caso inclui-se também a declaração de que o método pode também gerar a exceção `NumberFormatException`.

```
1 import java.io.*;  
2  
3 public class ConvInt3 {  
4     public String leLinha()  
5         throws IOException {  
6         byte[] tB = new byte[20];  
7         System.in.read(tB);  
8         String taTudo = new String(tB);  
9         String taLimpo = taTudo.trim();  
10        return taLimpo;  
11    }  
12  
13    public int leInt()  
14        throws IOException, NumberFormatException {
```



```
15 String s = leLinha();
16 return Integer.parseInt(s);
17 }
18
19 public static void main(String[] args) {
20     ConvInt3 ci = new ConvInt3();
21     System.out.print("Entre inteiro: ");
22     try {
23         int valor = ci.leInt();
24         System.out.println("Valor lido foi: " + valor);
25     }
26     catch (IOException ioe) {
27         System.err.println("Erro de leitura");
28     }
29     catch (NumberFormatException nfe) {
30         System.err.println("Erro de formato");
31     }
32 }
33 }
```

3.13.3 A hierarquia de exceções

Uma exceção em Java é um objeto. A classe raiz de todas as exceções é `java.lang.Throwable`. Apenas objetos dessa classe ou de suas classes derivadas podem ser gerados, propagados e capturados através do mecanismo de tratamento de exceções.

A classe `Throwable` tem duas subclasses:

java.lang.Exception

É a raiz das classes derivadas de `Throwable` que indica situações que a aplicação pode querer capturar e realizar um tratamento que permita prosseguir com o processamento.

java.lang.Error

É a raiz das classes derivadas de `Throwable` que indica situações que a aplicação não deve tentar tratar. Usualmente indica situações anormais, que não deveriam ocorrer.

Exemplos de exceções já definidas em Java incluem:

java.lang.ArithmeticException

indica situações de erros em processamento aritmético, tal como uma divisão inteira por 0.

java.lang.NumberFormatException

LAB-DSP

© 2003

Luiz Nagata



Indica que tentou-se a conversão de uma string para um formato numérico, mas seu conteúdo não representava adequadamente um número para aquele formato. É uma subclasse de `java.lang.IllegalArgumentException`.

`java.lang.ArrayIndexOutOfBoundsException`

Indica a tentativa de acesso a um elemento de um arranjo fora de seus limites -- ou o índice era negativo ou era maior ou igual ao tamanho do arranjo. É uma subclasse de `java.lang.IndexOutOfBoundsException`, assim como a classe `java.lang.StringIndexOutOfBoundsException`.

`java.lang.NullPointerException`

Indica que a aplicação tentou usar null onde uma referência a um objeto era necessária -- invocando um método ou acessando um atributo, por exemplo.

`java.lang.ClassNotFoundException`

Indica que a aplicação tentou carregar uma classe mas não foi possível encontrá-la.

`java.io.IOException`

Indica a ocorrência de algum tipo de erro em operações de entrada e saída. É a raiz das classes `java.io.EOFException` (fim de arquivo ou stream), `java.io.FileNotFoundException` (arquivo especificado não foi encontrado) e `java.io.InterruptedIOException` (operação de entrada ou saída foi interrompida), entre outras.

Entre os erros definidos em Java, subclasses de `java.lang.Error`, estão `java.lang.StackOverflowError` e `java.lang.OutOfMemoryError`. São situações onde não é possível uma correção a partir de um tratamento realizado pelo próprio programa que está executando.

Como exceções são objetos, podem incluir atributos. A classe `Throwable` inclui o registro do estado atual da pilha de chamadas de métodos e uma string associada a uma mensagem de erro, que pode ser obtida para qualquer exceção através do método `Throwable.getMessage()`.

Outras exceções podem incluir ainda outras informações que possam vir a ser relevantes para o seu tratamento. Por exemplo, `java.io.InterruptedIOException` inclui um atributo inteiro, `bytesTransferred`, para indicar quantos bytes foram transferidos antes da interrupção da operação ocorrer.

3.13.4 Capturando várias exceções

Quando apresentou-se a estrutura do bloco try-catch, pode ter ficado a impressão que para cada exceção possivelmente gerada em um bloco try um bloco catch deveria ser declarado.



Graças ao polimorfismo, não é esse o caso. A cláusula catch sabe que deve receber como argumento um objeto da classe Throwable. Como um objeto da classe derivada "é-um" objeto da superclasse, qualquer objeto-exceção da classe especificada ou de suas classes derivadas podem ser capturadas por um catch.

Por exemplo, uma especificação

```
catch (Exception e) {...}
```

implica na captura de todas as exceções, uma vez que a classe Exception é a raiz de todas as exceções.

Em geral, se dois blocos catches envolvem duas classes relacionadas na hierarquia de exceções, a superclasse deve vir após a classe derivada na sequência de blocos catch. Portanto, se um bloco catch for incluído com argumento Exception, como acima, esse deve ser o último bloco na sequência de catches.

3.13.5 Gerando exceções

A sinalização da ocorrência de uma condição excepcional de execução se dá através do comando throw, como ilustrado em:

```
public double calculaDivisao (int numerador, int denominador)
    throws Exception {
    if (denominador == 0)
        throw new Exception("Dividiu por zero");
    return ((double) numerador) / denominador;
}
```

3.13.6 Definição de novas exceções

É possível para uma aplicação definir e sinalizar suas próprias exceções. Por exemplo, considere que fosse importante para uma aplicação diferenciar a condição de divisão inteira por zero de outras condições de exceções aritméticas. Neste caso, uma classe DivideByZeroException poderia ser criada:

```
public class DivideByZeroException
    extends ArithmeticException {
    public DivideByZeroException() {
        super("O denominador na divisão inteira tem valor zero");
    }
}
```



A classe contém apenas um construtor, que invoca o construtor da superclasse especificando uma mensagem descrevendo a exceção. Esta é a mensagem que será retornada pelo método `getMessage()` quando invocado sobre um objeto dessa classe.

A sinalização da exceção se dá através do comando `throw`:

```
// Método que usa divisão inteira e pode gerar a exceção:
public double calculaDivisao (int numerador, int denominador)
    throws DivideByZeroException {
    if (denominador == 0)
        throw new DivideByZeroException();

    return (double) numerador / denominador;
}
```

3.13.7 Repassando a exceção

É possível usar o comando `throws` para repassar a sinalização da condição de exceção adiante, como em

```
public void usaDivisao()
    throws DivideByZeroException {
/*
    ...
*/
    try {
        d = calculaDivisao(x, y);
    }
    catch (DivideByZeroException dbze) {
        d = 0;
        throw dbze;
    }
/*
    ...
*/
}
```

Nesse caso, a informação associada à exceção (como o seu ponto de origem, registrado internamente no atributo do objeto que contém a pilha de invocações) é preservada. Caso fosse desejado mudar essa informação, uma nova exceção poderia ser gerada ou, caso a exceção seja a mesma, o método `fillInStackTrace()` poderia ser utilizado, como em

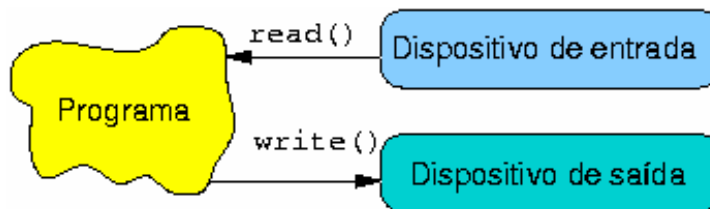
```
...
    throw dbze.fillInStackTrace();
...
```


3.14 ENTRADA E SAÍDA EM JAVA (I/O)

Freqüentemente o programador precisa trabalhar com informações externas que podem vir do disco rígido ou outro tipo de memória secundária, do teclado, rede, da memória ou de outro programa. Este capítulo trata das formas mais comuns de trabalhar com esses dados.

Por entrada e saída subentende-se o conjunto de mecanismos oferecidos para que um programa executando em um computador consiga respectivamente obter e fornecer informação de dispositivos externos ao ambiente de execução, composto pelo processador e memória principal.

De forma genérica, havendo um dispositivo de entrada de dados habilitado, o programa obtém dados deste dispositivo através de uma operação `read()`. Similarmente, um dado pode ser enviado para um dispositivo de saída habilitado através de uma operação `write()`.



A manipulação de entrada e saída de dados em Java é suportada através de classes do pacote *java.io*. Essas classes oferecem as funcionalidades para manipular a *entrada e saída de bytes*, adequadas para a transferência de dados binários, e para manipular a *entrada e saída de caracteres*, adequadas para a transferência de textos. Sendo Java uma linguagem de programação orientada a objetos, seria de se esperar que, além das funcionalidades usuais de entrada e saída, ela oferecesse também alguma funcionalidade para transferência de objetos. O mecanismo de *serialização* suporta essa funcionalidade.

Como a velocidade de operação de dispositivos de entrada e saída é várias ordens de grandeza mais lenta que a velocidade de processamento, *buffers* são tipicamente utilizados para melhorar a eficiência dessas operações de leitura e escrita. Outra funcionalidade associada à transferência de dados está relacionada à conversão de formatos, tipicamente entre texto e o formato interno de dados binários. Essa e outras funcionalidades são suportadas através do oferecimento de filtros que podem ser agregados aos objetos que correspondem aos mecanismos elementares de entrada e saída.



Um dispositivo de entrada e saída de vital importância é disco, manipulado pelo sistema operacional e por linguagens de programação através do conceito de arquivos. Dois dispositivos básicos de entrada e saída associados a qualquer processo são o teclado (entrada) e o monitor (saída), habilitados pelo sistema operacional no início da execução de qualquer programa como os *dispositivos padrões*.

Dispositivos padrões de entrada e saída

Os dispositivos padrões de entrada e saída de dados são abstrações de arquivos oferecidas pelos sistemas operacionais para interação básico com usuários. São tipicamente três dispositivos:

- **Entrada padrão** : Associada inicialmente ao dispositivo teclado, oferece uma interface equivalente a um arquivo seqüencial exclusivamente de leitura.
- **Saída padrão** : Associada inicialmente ao dispositivo monitor, oferece uma interface equivalente a um arquivo seqüencial exclusivamente de escrita.
- **Saída padrão de erros** : É inicialmente associada ao mesmo dispositivo da saída padrão.

Em Java, esses três dispositivos são representados respectivamente pelas variáveis in, out e err da classe System.

Dispositivos de I/O

- discos (arquivos)
- console
- conexão de rede

Formas de acesso

- seqüencial
- randômico
- binário
- por linhas
- por palavras

O sistema de I/O associa uma forma de acesso a um dispositivos

Para o acesso seqüencial de leitura/escrita, há dois grupos de classes:



- associados às fontes de dados (memória, disco, rede,...)
 - de bytes (Streams)
 - de caracteres Unicode (Readers)
- de processamento
 - buferização
 - filtro
 - objetos
 - impressão

E ainda existe o Acesso randômico a arquivos.

Obs.: Para maiores informações acesse o site:

<http://java.sun.com/j2se/1.4.1/docs/api/java/io/package-summary.html>

3.14.1 Streams

Uma stream pode ser definida como uma abstração entre o programador e o dispositivo que realmente está sendo acessado, que pode ser arquivo, memória, rede, impressora e etc. Por intermédio das streams é possível transportar dados como voz, vídeo, imagem, entre outros. As streams suportam leituras e escritas síncronas e assíncronas.

Streams implementam o conceito de cadeias unidirecionais de dados (FIFO, First-In, First-Out) apenas de escrita ou apenas de leitura. Assim, uma aplicação pode obter dados do início de um stream de entrada e pode enviar dados para o final de um stream de saída de dados, sempre seqüencialmente.

Streams em Java são suportados por classes do pacote `java.io`. Para a leitura seqüencial de bytes utiliza-se um objeto da classe `InputStream`, obtido como retorno do método `getInputStream`. Para transferência de texto, a ponte `InputStreamReader` pode ser utilizada para usar um `Reader`.

Similarmente, para a transferência de dados no sentido deste soquete para o outro extremo da conexão, utiliza-se um objeto da classe `OutputStream`. Para transferência de texto neste sentido, a ponte utilizada é `OutputStreamWriter`, permitindo a obtenção de um `Writer` associado a esse stream.

Streams lidam apenas com entrada ou saída de bytes.

Para podermos tratar com tipos mais complexos precisamos associar filtros a um stream



3.14.2 Definição de Arquivos

Um arquivo é uma abstração utilizada por sistemas operacionais para uniformizar a interação entre o ambiente de execução e os dispositivos externos a ele em um computador. Tipicamente, a interação de um programa com um dispositivo através de arquivos passa por três etapas:

abertura ou criação de um arquivo,
transferência de dados, e
fechamento do arquivo.

Em Java, a classe *File* permite representar arquivos nesse nível de abstração. Um dos construtores desta classe recebe como argumento uma string que pode identificar, por exemplo, o nome de um arquivo em disco. Os métodos desta classe permitem obter informações sobre o arquivo -- por exemplo, *exists()*, *canRead()*, *canWrite()*, *length()* e *lastModified()* -- e realizar operações sobre o arquivo como um todo, como em *delete()* e *deleteOnExit()*.

Observe que as funcionalidades de transferência seqüencial de dados a partir de ou para um arquivo não é suportada pela classe *File*, mas sim pelas classes *FileInputStream*, *FileOutputStream*, *FileReader* e *FileWriter*. Todas estas classes oferecem pelo menos um construtor que recebe como argumento um objeto da classe *File* e implementam os métodos básicos de transferência de dados suportados respectivamente por *InputStream*, *OutputStream*, *Reader* e *Writer*.

Para aplicações que necessitam manipular arquivos de forma não seqüencial (ou "direta"), envolvendo avanços ou retrocessos arbitrários na posição do arquivo onde ocorrerá a transferência, Java oferece a classe *RandomAccessFile*.

Para suportar essa funcionalidade, o pacote *java.io* define a classe *RandomAccessFile*. Um dos construtores dessa classe toma como argumentos duas strings, a primeira com o nome do arquivo e a segunda com o modo de operação ("r" para leitura apenas, "rw" para leitura e escrita).

Os métodos para a manipulação da posição corrente do ponteiro no arquivo são:

void seek(long pos)

Seleciona a posição (em relação ao início do arquivo) para a próxima operação de leitura ou escrita;

long getFilePointer()

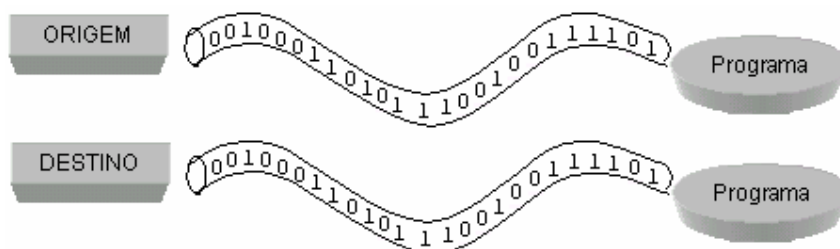
Retorna a posição atual do ponteiro do arquivo.

Além disso, o método *long length()* retorna o tamanho do arquivo em bytes.

Em java cada arquivo é tratado como um fluxo seqüencial de byte ou de dados.

3.14.3 O que são fluxos de dados?

Para trazer informações de outras origens que não estão no programa, assim como depositar informações para um destino, o programa abre um canal entre a origem ou destino e passa as informações seqüencialmente por esse canal. Esse sistema de canal, representado no esquema abaixo, em que os dados trafegam seqüencialmente recebe o nome de stream ou fluxo de dados.



Assim abstrai-se o procedimento de entrada e saída de dados de um programa através dos fluxos:

<i>Ler</i>	<i>Escreva</i>
Enquanto exista informação	Enquanto exista informação
Leia informação	Escreva informação
Feche o canal	Feche o canal

3.14.4 Entrada e saída de dados binários

Fontes de dados manipuladas como seqüências de bytes são tratadas em Java pela classe `InputStream` e suas classes derivadas. Similarmente, saídas de seqüências de bytes são tratadas por `OutputStream` e suas classes derivadas.

3.14.4.1 Entrada binária: `InputStream`

A classe abstrata `InputStream` oferece a funcionalidade básica para a leitura de um byte ou de uma seqüência de bytes a partir de alguma fonte. Os principais métodos dessa classe incluem: `read(...)`, `available()` e `close()`.

Os métodos `read(...)` oferecem a funcionalidade básica de leitura de bytes. O valor de retorno para esses métodos é um inteiro, que pode ser o byte lido (para o caso do método sem argumentos) ou o número de bytes lidos (para os métodos que fazem a leitura para um arranjo de bytes). Em qualquer caso, um valor de retorno -1 indica que o final do arquivo foi atingido.



Além dos métodos indicados, esta classe tem métodos para lidar com a posição de leitura no arquivo (`mark()` e `reset()`), se essa funcionalidade for suportada (o que pode ser consultado pelo método `markSupported()`).

3.14.4.2 Saída de dados binários: `OutputStream`

A classe abstrata `OutputStream` oferece a funcionalidade básica para a transferência sequencial de bytes para algum destino. Os métodos desta classe são: `write(...)`, `flush()` e `close()`.

Os métodos `write(...)` oferecem a funcionalidade básica para escrita de bytes para o destino. O método `flush()` força a saída para o stream destino de qualquer byte cuja escrita esteja pendente (por exemplo, armazenado em um buffer interno).

3.14.5 Entrada e Saída de caracteres

Aplicações típicas de entrada e saída envolvem a manipulação de arquivos contendo caracteres. Em Java, a especificação de funcionalidades para manipular esse tipo de arquivo estão contidas nas classes abstratas `Reader` (e suas classes derivadas) e `Writer` (e suas classes derivadas), respectivamente para leitura e para escrita de caracteres.

3.14.5.1 Entrada de caracteres: `Reader`

A classe abstrata `Reader` determina as funcionalidades comuns necessárias a todas as classes para a leitura sequencial de caracteres. Ela é similar à classe `InputStream`, que desempenha o mesmo papel na leitura sequencial de bytes.

Os principais métodos definidos na classe `Reader` incluem `read(...)`, para ler caracteres da fonte de dados, e `ready()`, que indica se a fonte está pronta para disponibilizar caracteres que serão lidos a seguir.

3.14.5.2 Saída de caracteres: `Writer`

A classe abstrata `Writer` é a raiz das classes de saída de dados do tipo caráter. Os seus principais métodos incluem `write(...)`, `close()` e `flush()`.



3.14.6 Transferência de objetos: Serialização

O processo de serialização de objetos permite converter a representação de um objeto em memória para uma seqüência de bytes que pode então ser enviada para um `ObjectOutputStream`, que por sua vez pode estar associado a um arquivo em disco ou a uma conexão de rede, por exemplo.

Por exemplo, para serializar um objeto da classe `java.util.HashMap`, armazenando seu conteúdo em um arquivo em disco, uma seqüência de passos poderia ser:

```
// criar o objeto
Map dados = new HashMap();
// associar informação ao objeto
dados.put(...);
...
// definir o nome do arquivo
String filename = ...;
// abrir o arquivo que irá receber o objeto serializado
File file = new File(filename);
if(!file.exists()){
    file.createNewFile();
}
OutputStream out = new FileOutputStream(file);
// associar ao arquivo o ObjectOutputStream
ObjectOutputStream s = new ObjectOutputStream(out);
// serializar e transferir o objeto
s.writeObject(dados);
```

O processo inverso, a desserialização, permite ler essa representação de um objeto a partir de um `ObjectInputStream` usando o método `readObject()`:

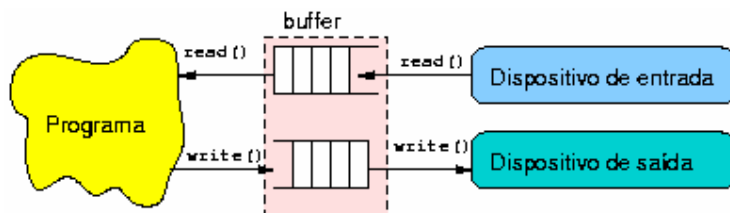
```
InputStream in = new FileInputStream(file);
ObjectInputStream s = new ObjectInputStream(in);
dados = (Map) s.readObject();
```

Serializable

Objetos de classes para os quais são previstas serializações e desserializações devem implementar a interface `java.io.Serializable`, que não especifica nenhum método -- é um exemplo de uso de uma interface marker, que serve apenas para registrar a semântica de serialização.

3.14.7 Buffers

Essencialmente, um buffer é uma entidade intermediária entre duas ou mais outras entidades que produzem e consomem elementos. Internamente, um buffer contém uma área de memória que é utilizada para armazenamento temporário de elementos que foram produzidos mas ainda não foram consumidos.



Essa capacidade de um buffer manter temporariamente parte dos dados produzidos mas não consumidos torna-o um mecanismo adequado para compatibilizar a operação de entidades operando em diferentes velocidades. Além de oferecer transparentemente as mesmas funcionalidades de leitura e escrita de dados, um buffer normalmente oferece operações adicionais que permitem consultar se há dados disponíveis para leitura no buffer de entrada ou para solicitar que os dados presentes no buffer de saída sejam enviados imediatamente para o dispositivo de saída.

Em Java, há quatro classes que adicionam as funcionalidades de buffering às classes fundamentais de entrada e saída. Para classes que manipulam bytes, as classes `BufferedInputStream` e `BufferedOutputStream` são oferecidas. Para a manipulação de entrada e saída de texto, são oferecidas as classes `BufferedReader` e `BufferedWriter`, que adiciona um método `void newLine()` para lidar com separadores de linha de maneira independente de plataforma.



3.14.8 Filtros

Um filtro de E/S é uma entidade que "envolve" um dispositivo de entrada ou saída agregando, em adição às funcionalidades básicas de transferência, alguma possibilidade de transformação dos dados sendo transferidos.

Em Java, há diversas classes que disponibilizam filtros que podem ser combinados de acordo com a necessidade da aplicação. Por exemplo, a classe `DataInputStream` é uma extensão de `InputStream` que implementa a interface `DataInput`, oferecendo um conjunto de métodos para ler grupos de bytes que serão interpretados como valores de variáveis dos tipos primitivos da linguagem Java. Similarmente, a classe `DataOutputStream` estende `OutputStream` e implementa `DataOutput`.

Outro filtro derivado de `OutputStream` é oferecido pela classe `PrintStream`, para converter tipos básicos para uma seqüência de bytes imprimível. Similarmente, a classe `PrintWriter` estende a classe `Writer` para converter tipos básicos em seqüências de caracteres. Essas duas classes oferecem os métodos `print()` e `println()` com assinaturas contemplando todos os tipos básicos da linguagem.

As funcionalidades básicas associadas a filtros de entrada e saída estão definidas nas classes `FilterInputStream`, `FilterOutputStream`, `FilterReader` e `FilterWriter`, das quais os filtros concretos são derivados.



3.14.9 Comandos de Java para Entrada e Saída

A tabela a seguir mostra as diversas subclasses de java.io e para que servem. Observe que existem diferentes classes que trabalham com tipos de dados diferentes com objetivos semelhantes.

E/S	Streams	Descrição
Memória	CharArrayReader CharArrayWriter ByteArrayInputStream ByteArrayOutputStream	Use essas classes para ler e escrever dados localizados em memória. Você pode usar os métodos dessas classes para ler e escrever vetores de dados.
	StringReader StringWriter	Use essas classes para ler e escrever caracteres de strings em memória. StringReader é usada para ler caracteres de strings em memória. StringWriter coleta caracteres escritos em um StringBuffer, os quais podem ser convertidos para String.
	StringBufferInputStream	StringBufferInputStream é similar a StringReader, exceto pelo motivo de ler bytes de um StringBuffer ao invés de caracteres.
Processos	PipeReader PipeWriter PipeInputStream PipeOutputStream	Implementa algoritmos para trabalhar com comunicação interprocessos, também conhecidos como <i>pipes</i> . São comumente utilizados como canal de comunicação entre <i>threads</i> .
Arquivos	FileReader FileWriter FileInputStream FileOutputStream	Esses fluxos são usados para ler de ou escrever em arquivos nativos do sistema operacional.
Cocatenação	SequenceInputStream	Cocatenam dados de múltiplos fluxos de dados de entrada em apenas um fluxo de entrada.
Serialização de Objetos	ObjectInputStream ObjectOutputStream	Usados para serializar objetos.



Conversão de Dados	DataInputStream DataOutputStream	Usados para ler e escrever dados primitivos em um formato independente de máquina.
Contadores	LineNumberRead LineNumberInputStream	Matêm o registro do número de linhas enquanto leêm.
Observadores	PushBackReader PushBackInputStream	Cada um desses <i>streams</i> possui um buffer adicional para decisão. Às vezes, quando estamos lendo dados de um buffer, é útil que esperemos alguns bytes ou caracteres até que possamos decidir como utilizar os dados do buffer.
Impressão	PrintWriter PrintStream	Contêm métodos convenientes para impressão. Esses são <i>streams</i> fáceis de manipular e podem ser frequentemente vistos em outros <i>streams</i> conectados com eles.
Armazenamento temporário	BufferedReader BufferedWriter BufferedInputStream BufferedOutputStream	Armazena os dados temporariamente quando leêm ou escrevem no intuito de reduzir acesso demasiado a fonte de dados original. Fluxos de dados que realizam armazenamento temporário são mais eficientes do que aqueles que não o fazem. Frequentemente são usados em conjunto com outros <i>streams</i> .
Filtro	FilterReader FilterWriter FilterInputStream FilterOutputStream	São classes abstratas que definem interfaces para canais de fluxo que precisam ser filtrados, nos quais o filtro de dados é na leitura ou escrita.
Conversores de Bytes e Caracteres	InputStreamReader OutputStreamWriter	Par de classes de fluxos de dados de leitura e escrita que formam uma ponte entre dados tipo byte e caracter ou vice-versa. Um <code>InputStreamReader</code> lê bytes de um <code>InputStream</code> e converte para caracteres, usando a decodificação de caracteres padrão ou uma decodificação fornecida pelo nome. Já um <code>OutputStreamWriter</code> converte caracteres para bytes, usando a decodificação de caracteres padrão ou uma codificação de caracter especificada pelo nome e escreve esses bytes para um <code>OutputStream</code>.



3.15 MULTIPROCESSAMENTO EM JAVA (THREADS)

Uma thread pode ser definida como “um fluxo de controle seqüencial isolado dentro de um programa”. Como um programa seqüencial qualquer, uma thread tem um começo, uma seqüência de comandos e um fim. Entretanto, uma thread em Java não é um programa, não executa sozinha; executa dentro de um programa.

Threads permitem que um programa simples possa executar várias tarefas diferentes ao mesmo tempo, independentemente uma das outras.

O próprio sistema runtime de Java utiliza threads para muitas coisas, e todas as bibliotecas de classes de Java são projetadas com a multithreading em mente. Na verdade, Java utiliza threads para tornar assíncrono todo o seu ambiente de execução. Isso ajuda a reduzir ineficiências, evitando o desperdício de ciclos da CPU.

Para entender melhor o ambiente multithreaded, vamos compará-lo com o ambiente de thread única. Sistemas de thread única utilizam um mecanismo chamado loop de eventos com polling. Neste modelo, uma única thread controla um loop infinito, checando uma fila única para decidir o próximo passo a ser feito. Quando esse mecanismo de polling retorna, digamos, o sinal de que um arquivo da rede está pronto para ser lido, o loop de eventos despacha o controle para o manipulador de eventos apropriado. Neste exemplo, o manipulador seria o método encarregado de ler o arquivo. Enquanto esse método não retornar, nada pode ser feito, e o sistema precisa ficar esperando. Isso é um desperdício de tempo da CPU. Pode também acontecer de uma parte do programa assumir o controle do sistema, impedindo que outros eventos sejam processados. Em geral, num ambiente de thread única, quando esta thread fica bloqueada, ou seja, quando a thread suspende a execução à espera de um recurso, todo o programa para de rodar.

A vantagem da multithreading de Java é a eliminação do mecanismo de loop principal e polling. Uma thread pode entrar em pausa sem parar as outras partes do programa. Por exemplo, o tempo ocioso criado quando uma thread lê dados de uma rede ou espera pela entrada do usuário pode ser utilizado por outra parte do programa. A multithreading permite que um loop de animação faça uma pequena parada entre uma imagem e outra, para evitar que a animação rode rápido demais, sem que isso cause a parada de todo o sistema. Quando uma thread é bloqueada num programa Java, somente aquela thread fica parada. Todas as outras threads continuam rodando.

Obs.: para maiores informações consulte o site::

<http://java.sun.com/j2se/1.4.1/docs/api/java/lang/Thread.html>

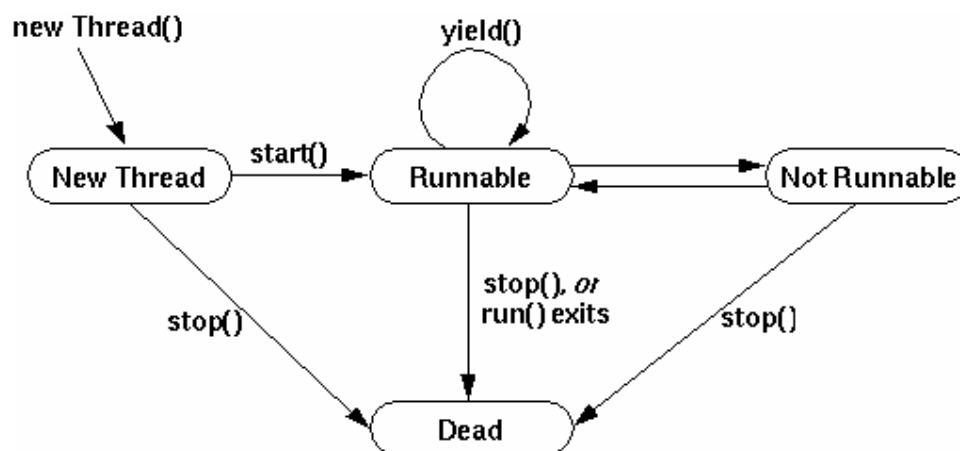
3.15.1 Estados de uma Thread

Uma thread pode se encontrar em vários estados. Uma thread pode estar rodando (running). Pode também estar pronta para rodar (ready to run), assim que conseguir tempo de CPU. Uma thread pode estar suspensa (suspended), ou seja, sua atividade pode estar parada temporariamente. Uma thread suspensa pode retomar sua atividade (resume) do ponto em que parou. Dizemos que uma thread está bloqueada (blocked) quando está à espera de um recurso. A qualquer momento, uma thread pode ser terminada (terminated), o que encerrará sua execução imediatamente. Uma vez terminada (terminated), uma thread não pode ser retomada (resumed).

A tabela abaixo resume os possíveis estados de uma thread:

Estados de uma thread	
Estado	Significado
Running	A thread está rodando
Ready to run	A thread está pronta para rodar
Suspended	A atividade da thread está parada temporariamente
Resume	O ato da thread retomar sua atividade
Blocked	A execução da thread está bloqueada à espera de um recurso
Terminated	A execução da thread foi encerrada, não podendo ser retomada

Os Estados e Métodos Básicos de um Thread :





3.15.2 Prioridades das threads

Java atribui uma *prioridade* a cada thread. Isso determina como a thread será tratada em relação a outras. A prioridade de uma thread é um número inteiro que especifica a prioridade da thread em relação às outras threads. Portanto, como valor absoluto, a prioridade não tem nenhum significado. Ou seja, ao rodar sozinha, uma thread com prioridade mais elevada não rodará mais rápido do que uma thread de prioridade mais baixa.

A prioridade é usada para decidir quando será feito o chaveamento de uma thread para outra. Esse procedimento é o que chamamos de chaveamento de contexto (*context switch*). As regras que determinam quando o chaveamento de contexto acontece são bastante simples.

Essas regras são mostradas na tabela abaixo:

Regras para o chaveamento de contexto
Uma thread pode abrir mão do controle espontaneamente. Isso é feito cedendo (<i>yielding</i>) o controle explicitamente, entrando em modo <i>sleep</i> ou em bloqueio à espera de I/O. Neste caso, todas as outras threads são examinadas e a thread de prioridade mais elevada que estiver pronta para rodar recebe o controle da CPU.
Uma thread pode ser deslocada (<i>preempted</i>) por outra thread de prioridade mais elevada. Neste caso, uma thread de prioridade mais baixa que não cede o controle é simplesmente deslocada por outra thread de prioridade mais elevada, não importa o que estiver fazendo. Em princípio, uma thread de prioridade mais elevada roda no momento que quiser. Este mecanismo é chamado de <i>multitarefa preemptiva (preemptive multitasking)</i> .

Nos casos em que duas threads com a mesma prioridade estão competindo por ciclos de CPU, a situação fica um pouco mais complicada. Em sistemas operacionais como o Windows 95/98, threads de mesma prioridade recebem fatias de tempo automaticamente, uma de cada vez. Em outros tipos de sistemas operacionais, como Sun Solaris, threads de mesma prioridade precisam ceder o controle voluntariamente umas às outras.

Se uma thread não fizer isso, as outras não poderão rodar.

Há um aspecto importante, que deve estar na mente do programador Java o tempo todo: na programação Java, diferenças na forma como os vários sistemas operacionais fazem o chaveamento de contexto podem ser uma fonte de problemas.

3.15.3 Sincronização

O uso de multithreading introduz um comportamento assíncrono nos programas.

Portanto, torna-se necessário um mecanismo para forçar a sincronização, quando for o caso. Por exemplo, se quisermos que duas threads se comuniquem compartilhando uma estrutura de dados complexa, como uma lista encadeada, é preciso ter uma forma de assegurar que não haverá conflito entre as threads. Ou seja, é preciso evitar que uma thread escreva dados ao mesmo tempo que outra thread está lendo esses dados. Para isso, Java implementa uma elegante inovação no modelo tradicional de sincronização



inter-processos, como veremos adiante. Esse modelo de sincronização é chamado monitor. O monitor é um mecanismo de controle definido originalmente por C.A.R. Hoare.

Podemos pensar no monitor como uma pequena caixa que, num determinado instante, somente pode conter uma thread de cada vez. Ou seja, depois que uma thread entra no monitor, todas as outras devem esperar até que a primeira saia. Desta forma, um monitor pode ser usado para proteger um recurso compartilhado, impedindo que ele seja manipulado por mais de uma thread de cada vez.

A maioria dos sistemas multithreaded expõe os monitores na forma de objetos que o programa deve explicitamente adquirir e manipular. É aqui que Java oferece uma solução mais elegante. Em Java, não existe uma classe "Monitor". Ao invés disso, cada objeto tem seu próprio monitor implícito, no qual a execução entra sempre que um dos métodos sincronizados (synchronized) do objeto é chamado. Uma vez que uma thread esteja dentro de um método sincronizado, nenhuma outra thread pode chamar nenhum outro método sincronizado do mesmo objeto. Isso possibilita a escrita de código multithreaded muito claro e conciso, porque o suporte à sincronização já vem embutido na linguagem.

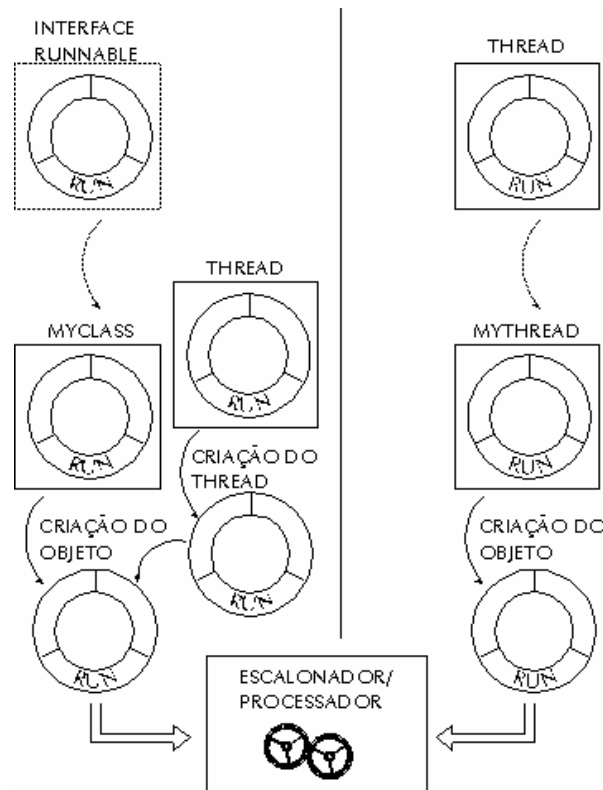
3.15.4 Mensagens

Estando o programa dividido em várias threads, precisamos definir uma forma como essas threads se comunicarão entre si. Na maioria das linguagens, o programador depende do sistema operacional para estabelecer a comunicação entre as threads. Naturalmente, isso acrescenta uma carga adicional de processamento. Java, por sua vez, oferece uma forma limpa e de baixo custo (em termos de processamento) para que duas ou mais threads conversem entre si. Isso é feito através de chamadas a métodos predefinidos, presentes em todos os objetos. O sistema de mensagens de Java permite que uma thread entre num método sincronizado de um objeto e depois espere até que alguma outra thread explicitamente a notifique para que saia.

3.15.5 A classe Thread e a interface Runnable

O sistema multithreading de Java é baseado na classe Thread e seus métodos, e também na interface Runnable. A classe Thread encapsula uma thread (linha) de execução. Como não podemos nos referir diretamente ao elemento impalpável que é o estado de uma thread em execução, lidamos com ele através de sua representação, que é a instância de Thread que gerou a thread. Para criar uma nova thread, um programa deve criar uma classe derivada de Thread, ou implementar a interface Runnable.

Duas formas de criar uma thread
1 – Criar uma classe derivada de Thread
2 – Implementar a interface Runnable



A classe `Thread` define diversos métodos que ajudam a gerenciar as threads. Eis os principais métodos de `Thread`:

Os principais métodos da classe <code>Thread</code>	
Método	O que faz
<code>getName()</code>	Obtém o nome de uma thread
<code>getPriority()</code>	Obtém a prioridade de uma thread
<code>isAlive()</code>	Determina se uma thread ainda está rodando
<code>join()</code>	Espera que uma thread termine
<code>resume()</code>	Retoma a execução de uma thread suspensa anteriormente
<code>run()</code>	Ponto de entrada para início da execução de uma thread
<code>sleep()</code>	Suspende uma thread por um período de tempo (em ms)
<code>start()</code>	Dá partida numa thread, chamando seu método <code>run()</code>
<code>suspend()</code>	Suspende uma thread

Os exemplos de programas Java mais simples utilizam apenas uma linha de execução (thread). Passaremos agora a explicar como usar `Thread` e `Runnable` para criar e gerenciar threads. Começaremos com uma thread que está presente em todos os programas Java: a thread main (principal).



3.15.6 A thread principal

Todo programa Java tem uma thread que começa a rodar no momento em que o programa é iniciado. Ela é conhecida como a thread principal (main) do programa, porque é aquela que começa junto com o programa. A thread principal é importante por dois motivos:

A importância da thread principal

É a thread a partir da qual as threads filhas (*child threads*) serão geradas

É a última thread a encerrar a execução. Quando a thread principal pára, o programa é encerrado

Embora a thread principal seja criada automaticamente quando o programa inicia, ela pode ser controlada através de um objeto Thread. Para isso, precisamos obter uma referência a ela, chamando o método `currentThread()`, um método public static da classe Thread. Eis sua forma geral:

```
static Thread currentThread()
```

Esse método retorna uma referência à thread na qual é chamado. De posse de uma referência à Thread principal, podemos controlá-la, como faríamos com qualquer outra thread. O exemplo abaixo ilustra essa técnica.

```
/**
 * Ilustração de acesso à thread principal.
 */
class AcessaThreadPrincipal {
    public static void main(String args[])
    {
        Thread thr = Thread.currentThread();
        System.out.println("Thread atual: " + thr);
        // Muda o nome da thread.
        thr.setName("Thread Principal");
        System.out.println("Novo nome: " + thr);
        // Utiliza sleep() para
        // suspender a thread (1 segundo).
        try
        {
            for(int i = 10; i > 0; i--)
            {
                System.out.println(i);
                Thread.sleep(1000);
            } // Fim de for.
        } // Fim de try.
        catch (InterruptedException e)
        {
            System.out.println("Thread principal interrompida");
        } // Fim de catch.
    } // Fim de main()
} // Fim de class AcessaThreadPrincipal.
```



Saída gerada por este programa:

```
Thread atual: Thread[main,5,main]
Novo nome: Thread[Thread Principal,5,main]
10
9
8
7
6
5
4
3
2
1
```

Neste programa, usamos o método `currentThread()` para obter uma referência à thread atual, que no caso é a thread principal (main thread). A referência obtida é armazenada na variável local `thr`. A seguir, o programa exibe informações sobre a thread. Depois, chama o método `setName()` para mudar o nome interno da thread. As informações sobre a thread são então reexibidas. A seguir, um loop faz a contagem decrescente de 10 a 1, fazendo uma pausa de 1 segundo entre cada dois números. A pausa é criada com o método `sleep()`. O argumento passado para `sleep()` especifica o período de espera em milissegundos. Observe o uso (obrigatório) dos blocos `try/catch` ao redor do loop. Isso é feito porque o método `sleep()` da classe `Thread` pode lançar uma exceção do tipo `InterruptedException`. Isso acontece se alguma outra thread tentar interromper a thread que está suspensa. Ou seja, digamos que uma thread está esperando (waiting), suspensa (sleeping), ou em pausa por algum outro motivo há um certo tempo. Se uma outra thread tentar interrompê-la usando o método `interrupt()`, uma exceção `InterruptedException` será lançada. Neste exemplo, se ocorrer uma exceção `InterruptedException`, uma mensagem simples será exibida. Em um programa do mundo real, é preciso lidar com cuidado com esse tipo de exceção.

Observe ainda a saída gerada quando `thr` é usada como argumento para `println()`.

Essa chamada exibe, pela ordem: o nome da thread, sua prioridade e o nome de seu grupo. Por default, o nome da thread principal é `main`. Sua prioridade é 5, que é o valor default, e o nome do grupo de threads ao qual ela pertence também é `main`. Um grupo de threads (thread group) é uma estrutura de dados que controla o estado de uma coleção de threads como um todo. Os grupos de threads são gerenciados pelo ambiente runtime, e não são discutidos em detalhes aqui. Voltando ao nosso exemplo, depois que o nome da thread é alterado, a thread `thr` é novamente exibida. Desta vez, o nome exibido é o novo nome.

Vamos examinar com mais detalhes os métodos definidos por `Thread` que são usados no programa. O método `sleep()` faz com que a thread dentro da qual é chamado tenha a execução suspensa pelo período especificado em milissegundos. Sua forma geral é mostrada abaixo:

`static void sleep(long milliseconds) throws InterruptedException`

O número de milissegundos da suspensão é especificado pelo parâmetro `milliseconds`. Este método pode lançar uma `InterruptedException`. O método `sleep()` tem também uma segunda forma, mostrada abaixo, que permite especificar o período em termos de milissegundos e nanossegundos:



static void sleep(long milliseconds, int nanoseconds) throws InterruptedException

Esta segunda forma só tem utilidade em ambientes que permitem temporização da ordem de nanossegundos.

Conforme mostramos no programa acima, podemos definir o nome de uma thread usando setName(). Podemos também usar um método chamado getName() para obter o nome de uma Thread. Este último método não é usado em nosso exemplo. Esses métodos são membros da classe Thread, declarados da seguinte forma:

```
final void setName(String nomeThread)  
final String getName()
```

O argumento nomeThread especifica o nome da thread.

3.15.7 Criando uma thread

Em Java, para criar uma thread precisamos criar uma instância de um objeto thread. Isso pode ser feito de duas maneiras:

Criando uma thread	
Implementando a interface	Runnable
Derivando da classe	Thread

Vamos examinar essas duas possibilidades.

3.15.7.1 Implementando Runnable

A forma mais fácil de criar uma thread é criar uma classe que implementa a interface Runnable. Runnable é a abstração de uma unidade de código executável. Podemos construir uma thread em qualquer objeto que implemente Runnable. Na verdade, para implementar Runnable, uma classe precisa implementar apenas um único método, chamado run(), declarado da seguinte forma:

```
public abstract void run()
```

Dentro do método run(), definimos o código que constitui a nova thread. É importante entender que run() pode chamar outros métodos, usar outras classes e declarar variáveis, da mesma forma que a thread principal. A única diferença é que run() representa o ponto de entrada para a nova thread, que roda simultaneamente dentro do programa. Esta thread será encerrada quando run() retornar.



Depois de criar uma classe que implementa Runnable, precisamos instanciar um objeto do tipo Thread dentro daquela classe. Thread define diversos construtores. Como exemplo, usaremos o construtor mostrado abaixo:

`Thread(Runnable objThread, String nomeThread)`

Neste construtor, objThread é uma instância de uma classe que implementa a interface Runnable. Isso define onde começa a execução da thread. O nome da nova thread é especificado por nomeThread.

Uma vez criada, a nova thread só começa a rodar quando seu método start() for chamado. O método start() é declarado na classe Thread. Essencialmente, o que start() faz é chamar run(). Eis a forma geral do método start():

`synchronized void start()`

Por enquanto, não se preocupe com o significado do modificador synchronized. Ele será descrito mais adiante.

O exemplo abaixo cria uma nova thread e dá partida em sua execução:

```
/**
 * Ilustra criação de uma Thread.
 */
class OutraThread implements Runnable {
    Thread thr;
    OutraThread()
    {
        // Cria a nova Thread.
        thr = new Thread(this, "Outra Thread");
        System.out.println("Thread filha: " + thr);
        // Dá partida na Thread.
        thr.start();
    } // Fim do construtor OutraThread().
    // Este é o ponto de entrada
    // da nova Thread.
    public void run()
    {
        try
        {
            for(int i = 5; i > 0; i--)
            {
                System.out.println("Thread filha: " + i);
                Thread.sleep(500);
            } // Fim de for.
        } // Fim de try.
        catch (InterruptedException e)
        {
            System.out.println("Thread filha interrompida.");
        } // Fim de catch.
        System.out.println("Saindo da Thread filha...");
    } // Fim de run()
} // Fim de class OutraThread.
class OutraThreadDemo
{
    public static void main(String args[])
    {
        // Cria a nova Thread.
```



```
new OutraThread();
try
{
    for(int i = 5; i > 0; i--)
    {
        System.out.println("Thread principal: " + i);
        Thread.sleep(1000);
    } // Fim de for.
} // Fim de try.
catch(InterruptedException e)
{
    System.out.println("Thread principal interrompida.");
} // Fim de catch.
System.out.println("Saindo da Thread principal...");
} // Fim de main()
} // Fim de class OutraThreadDemo.
```

Saída gerada por este programa:

```
Thread filha: Thread[Outra Thread,5,main]
Thread principal: 5
Thread filha: 5
Thread filha: 4
Thread principal: 4
Thread filha: 3
Thread filha: 2
Thread principal: 3
Thread filha: 1
Saindo da Thread filha...
Thread principal: 2
Thread principal: 1
Saindo da Thread principal...
```

Dentro do construtor `OutraThread()`, um novo objeto `Thread` é criado com o seguinte comando:

```
thr = new Thread(this, "Outra Thread");
```

A passagem de `this` como primeiro argumento indica que queremos que a nova `Thread` chame o método `run()` para o objeto atual (`this`). A seguir, o método `start()` é chamado, fazendo com que a execução da thread comece com uma chamada ao método `run()`. Isso faz com que o loop `for` da thread filha comece. Depois da chamada a `start()`, o construtor `OutraThread()` retorna para `main()`. Quando a thread principal é retomada, ela entra em seu próprio loop `for`. As duas threads continuam rodando, compartilhando a CPU, até que seus respectivos loops terminem.

Conforme dissemos anteriormente, em um programa multithreaded, a thread principal deve ser a última a terminar a execução. Se a thread principal terminar antes que uma thread filha seja completada, o sistema runtime Java pode travar. O programa anterior assegura que a thread principal seja a última a terminar fazendo com que ela seja suspensa por 1000 milissegundos entre as iterações, enquanto a thread filha é suspensa por apenas 500 milissegundos. Isso faz com que a thread filha termine antes da thread principal. É claro que existem maneiras melhores de assegurar que a thread principal seja a última a ser encerrada, conforme veremos em breve.



3.15.7.3 Derivando de Thread

A segunda forma de criar uma thread é criar uma classe derivada de Thread, e depois criar uma instância dessa classe. A classe derivada deve implementar o método run(), que será o ponto de entrada da nova thread. Deve também chamar o método start() para começar a execução da nova thread. O programa abaixo é uma nova versão do exemplo anterior, usando a técnica descrita acima.

```
class ThreadDeriv extends Thread {
    ThreadDeriv()
    {
        // Cria uma nova Thread.
        super("Nova Thread");
        System.out.println("Thread filha: " + this);
        // Dá partida na Thread.
        start();
    } // Fim do construtor ThreadDeriv()
    // Este método é o ponto de partida
    // para a execução da nova Thread.
    public void run()
    {
        try
        {
            for(int i = 5; i > 0; i--)
            {
                System.out.println("Thread filha: " + i);
                Thread.sleep(500);
            } // Fim de for.
        } // Fim de try.
        catch(InterruptedException e)
        {
            System.out.println("Thread filha interrompida.");
        } // Fim de catch.
        System.out.println("Saida da thread filha...");
    } // Fim de run()
} // Fim de class ThreadDeriv.

class ThreadDerivDemo {
    public static void main(String args[])
    {
        // Cria uma nova Thread.
        new ThreadDeriv();
        try
        {
            for(int i = 5; i > 0; i--)
            {
                System.out.println("Thread principal: " + i);
                Thread.sleep(1000);
            } // Fim de for.
        } // Fim de try.
        catch(InterruptedException e)
        {
            System.out.println("Thread principal interrompida.");
        } // Fim de catch.
        System.out.println("Saindo da Thread principal...");
    } // Fim de main()
} // Fim de class ThreadDerivDemo.
```



Saída gerada por este programa:

```
Thread filha: Thread[Nova Thread,5,main]
Thread principal: 5
Thread filha: 5
Thread filha: 4
Thread principal: 4
Thread filha: 3
Thread filha: 2
Thread principal: 3
Thread filha: 1
Saída da thread filha...
Thread principal: 2
Thread principal: 1
Saindo da Thread principal...
```

Podemos ver que a thread filha é criada por meio de uma instância da classe `ThreadDeriv`, que é derivada de `Thread`.

Observe a chamada a `super()` dentro de `ThreadDeriv`. Essa chamada invoca a seguinte versão do construtor de `Thread`:

```
public Thread(String nomeThread)
```

O argumento `nomeThread` especifica o nome da `Thread`.

3.15.7.4 Escolhendo o enfoque certo

Você deve estar se perguntando por que Java oferece duas formas de criar threads filhas, e qual desses enfoques é o melhor. As respostas a essas duas perguntas estão intimamente interligadas. A classe `Thread` define diversos métodos que podem ser superpostos (*overriden*) por uma classe derivada. De todos esses métodos, apenas um precisa ser obrigatoriamente superposto: é o método `run()`. É claro que o mesmo acontece quando implementamos a interface `Runnable`. Do ponto de vista filosófico da orientação a objetos, uma classe só deve ser estendida quando está sendo enriquecida ou modificada de alguma forma. Assim, se não houver necessidade de implementar nenhum dos outros métodos de `Thread`, provavelmente a melhor opção será implementar a interface `Runnable`. É claro que isso fica totalmente na mão do programador.

3.15.7.5 Criando várias threads

Até agora, nossos exemplos usaram somente duas threads: a thread principal e uma thread filha. Contudo, um programa pode gerar tantas threads quantas sejam necessárias. Por exemplo, o programa abaixo cria três threads filhas.

```
class ThreadFilha implements Runnable {
    // Nome da thread.
    String nome;
    Thread thr;
    // Construtor.
```



```
ThreadFilha(String nomeThread)
{
    nome = nomeThread;
    thr = new Thread(this, nome);
    System.out.println("Nova thread: " + thr);
    // Dá partida na thread.
    thr.start();
} // Fim do construtor ThreadFilha()
// Ponto de início de execução da thread.
public void run()
{
    try
    {
        for(int i = 5; i > 0; i--)
        {
            System.out.println(nome + ": " + i);
            Thread.sleep(1000);
        } // Fim de for.
    } // Fim de try.
    catch(InterruptedException e)
    {
        System.out.println("Thread " + nome + " interrompida!");
    } // Fim de catch.
    System.out.println("Saindo de " + nome + "...");
} // Fim de run()
} // Fim de class ThreadFilha.

class ThreadFilhaDemo {
    public static void main(String args[])
    {
        // Cria e dá partida em 3 threads.
        new ThreadFilha("ThreadFilha01");
        new ThreadFilha("ThreadFilha02");
        new ThreadFilha("ThreadFilha03");
        try
        {
            // Espera o encerramento das outras threads.
            Thread.sleep(10000);
        } // Fim de try.
        catch(InterruptedException e)
        {
            System.out.println("Thread principal interrompida!");
        } // Fim de catch.
        System.out.println("Saindo da thread principal...");
    } // Fim de main()
} // Fim de class ThreadFilhaDemo.
```

Saída gerada por este programa:

```
C:\$user\NovoCursoJava\Source>java ThreadFilhaDemo
Nova thread: Thread[ThreadFilha01,5,main]
Nova thread: Thread[ThreadFilha02,5,main]
ThreadFilha01: 5
Nova thread: Thread[ThreadFilha03,5,main]
ThreadFilha02: 5
ThreadFilha03: 5
ThreadFilha01: 4
ThreadFilha02: 4
ThreadFilha03: 4
ThreadFilha01: 3
ThreadFilha02: 3
```




```
ThreadFilha03: 3
ThreadFilha01: 2
ThreadFilha02: 2
ThreadFilha03: 2
ThreadFilha01: 1
ThreadFilha02: 1
ThreadFilha03: 1
Saíndo de ThreadFilha01...
Saíndo de ThreadFilha03...
Saíndo de ThreadFilha02...
Saíndo da thread principal...
```

Vemos que, uma vez iniciadas, as três threads filhas compartilham a CPU. Observe a chamada a `sleep(10000)` em `main()`. Isso faz com que a thread principal seja suspensa por 10 segundos, o que assegura que ela será a última a ser encerrada.

3.15.7.6 Os métodos `isAlive()` e `join()`

Já enfatizamos várias vezes que a thread principal deve ser a última a ser encerrada. Para assegurar isso nos exemplos anteriores, chamamos `sleep()` dentro de `main()`, sempre com um valor suficientemente longo para assegurar que todas as threads filhas terminassem antes da thread principal. É claro que esta solução não pode ser aplicada na maioria dos casos. Uma solução pode ser encontrada com a seguinte pergunta: existe uma maneira de uma thread saber se outra thread já terminou? Sim. A resposta está na classe `Thread`.

Existem duas maneiras de determinar se uma thread já terminou. Primeiro, podemos chamar o método `isAlive()` para aquela thread. O método `isAlive()`, definido na classe `Thread`, tem a seguinte forma geral:

```
final boolean isAlive() throws InterruptedException
```

O método `isAlive()` retorna `true` se a thread para a qual foi chamado ainda estiver rodando. Caso contrário, retorna `false`.

Porém existe um outro método usado com mais frequência para resolver esse problema. Trata-se do método `join()`, usado para esperar a finalização de uma thread. Eis sua forma geral:

```
final void join() throws InterruptedException
```

O método `join()` espera até que a thread para a qual foi chamado termine a execução. O nome desse método vem do conceito de que a thread chamadora espera até que a thread especificada junte-se (*join*) a ela. Existem outras formas do método `join()` que permitem definir o período de tempo máximo que desejamos esperar pela finalização da thread especificada.

O programa abaixo é uma versão melhorada do programa anterior. Neste exemplo, usamos `join()` para assegurar que a thread principal seja a última a ser encerrada. O exemplo demonstra também o método `isAlive()`.

```
class NovaThread implements Runnable {
    // Nome da Thread.
    String nome;
```



```
Thread thr;
NovaThread(String nomeThread)
{
    nome = nomeThread;
    thr = new Thread(this, nome);
    System.out.println("Nova thread: " + thr);
    // Dá partida na nova thread.
    thr.start();
} // Fim do construtor NovaThread()
// Ponto de entrada para início da
// execução da nova thread.
public void run()
{
    try
    {
        for(int i = 5; i > 0; i--)
        {
            System.out.println(nome + ": " + i);
            Thread.sleep(1000);
        } // Fim de for.
    } // Fim de try.
    catch(InterruptedException e)
    {
        System.out.println("Thread " + nome + " interrompida!");
    } // Fim de catch.
    System.out.println("Saindo de " + nome + "...");
} // Fim de run()
} // Fim de class NovaThread.

class ThreadJoinDemo {
    public static void main(String args[])
    {
        NovaThread nv01 = new NovaThread("Thread 01");
        NovaThread nv02 = new NovaThread("Thread 02");
        NovaThread nv03 = new NovaThread("Thread 03");
        System.out.println("Thread 01 rodando: "
            + nv01.thr.isAlive());
        System.out.println("Thread 02 rodando: "
            + nv02.thr.isAlive());
        System.out.println("Thread 03 rodando: "
            + nv03.thr.isAlive());
        // Espera encerramento das threads.
        try
        {
            System.out.println(
                "Esperando encerramento de threads filhas...");
            nv01.thr.join();
            nv02.thr.join();
            nv03.thr.join();
        } // Fim de try.
        catch(InterruptedException e)
        {
            System.out.println(
                "Thread principal interrompida!");
        } // Fim de catch.
        System.out.println("Thread 01 rodando: "
            + nv01.thr.isAlive());
        System.out.println("Thread 02 rodando: "
            + nv02.thr.isAlive());
        System.out.println("Thread 03 rodando: "
            + nv03.thr.isAlive());
    }
}
```



```
System.out.println("Thread principal terminando...");  
} // Fim de main()  
} // Fim de class ThreadJoinDemo.
```

Saída gerada por este programa:

```
Nova thread: Thread[Thread 01,5,main]  
Thread 01: 5  
Nova thread: Thread[Thread 02,5,main]  
Thread 02: 5  
Nova thread: Thread[Thread 03,5,main]  
Thread 03: 5  
Thread 01 rodando: true  
Thread 02 rodando: true  
Thread 03 rodando: true  
Esperando encerramento de threads filhas...  
Thread 01: 4  
Thread 02: 4  
Thread 03: 4  
Thread 01: 3  
Thread 02: 3  
Thread 03: 3  
Thread 01: 2  
Thread 02: 2  
Thread 03: 2  
Thread 01: 1  
Thread 02: 1  
Thread 03: 1  
Saindo de Thread 01...  
Saindo de Thread 02...  
Saindo de Thread 03...  
Thread 01 rodando: false  
Thread 02 rodando: false  
Thread 03 rodando: false  
Thread principal terminando...
```

Podemos ver que, quando a chamada a `join()` retorna, as threads já pararam de rodar.

3.15.7.7 Os métodos `suspend()` e `resume()`

Muitas vezes pode ser necessário suspender a execução de uma thread. Por exemplo, digamos que uma determinada thread é usada para exibir a hora. Se o usuário não quiser que o relógio seja exibido, pode suspender essa thread. Qualquer que seja o motivo, suspender uma thread é muito fácil. Depois de suspensão, uma thread pode também ser facilmente reiniciada. Os métodos que fazem isso são `suspend()` e `resume()`.

Eles são definidos por `Thread`, conforme os modelos abaixo:

```
final void suspend()  
final void resume()
```

O programa abaixo demonstra o uso desses métodos.

```
class SuspThread implements Runnable {  
    // Nome da thread.  
    String nome;  
    Thread thr;  
    // Construtor.
```



UNIVERSIDADE DO AMAZONAS
FACULDADE DE TECNOLOGIA
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS



```
SuspThread(String nomeThread)
{
    nome = nomeThread;
    thr = new Thread(this, nome);
    System.out.println("Thread filha: " + thr);
    // Inicia execução da thread.
    thr.start();
} // Fim do construtor SuspThread()
// Ponto de entrada da nova thread.

public void run()
{
    try
    {
        for(int i = 15; i > 0; i--)
        {
            System.out.println("Thread " + nome + ": " + i);
            Thread.sleep(200);
        } // Fim de for.
    } // Fim de try.
    catch(InterruptedException e)
    {
        System.out.println("Thread " +
            nome + " interrompida.");
    } // Fim de catch.
    System.out.println("Encerrando " +
        nome + "...");
} // Fim de run()
} // Fim de class SuspThread.

class SuspThreadDemo {
    public static void main(String args[])
    {
        SuspThread sp01 = new SuspThread("thread filha 01");
        SuspThread sp02 = new SuspThread("thread filha 02");
        try
        {
            // Thread principal sleep().
            Thread.sleep(1000);
            // Suspende thread sp01.
            sp01.thr.suspend();
            System.out.println("Suspendendo thread 01");
            // Thread principal sleep().
            Thread.sleep(1000);
            // Retoma thread sp01.
            sp01.thr.resume();
            System.out.println("Retomando thread 01");
            // Suspende thread sp02.
            sp02.thr.suspend();
            System.out.println("Suspendendo thread 02");
            // Thread principal sleep().
            Thread.sleep(1000);
            // Retoma thread sp02.
            sp02.thr.resume();
            System.out.println("Retomando thread 02");
        } // Fim de try.
        catch(InterruptedException e)
        {
            System.out.println(
                "Thread principal interrompida!!!");
            // Fim de catch.
        }
    }
}
```



UNIVERSIDADE DO AMAZONAS
FACULDADE DE TECNOLOGIA
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS



```
// Espera encerramento de threads.
try
{
    System.out.println(
        "Esperando encerramento de threads...");
    sp01.thr.join();
    sp02.thr.join();
} // Fim de try.
catch (InterruptedException e)
{
    System.out.println(
        "Thread principal interrompida!!!");
} // Fim de catch.
System.out.println(
    "Encerrando thread principal...");
} // Fim de main()
} // Fim de class SuspThreadDemo.
```

Saída gerada por este programa:

```
Thread filha: Thread[thread filha 01,5,main]
Thread thread filha 01: 15
Thread filha: Thread[thread filha 02,5,main]
Thread thread filha 02: 15
Thread thread filha 01: 14
Thread thread filha 02: 14
Thread thread filha 01: 13
Thread thread filha 02: 13
Thread thread filha 01: 12
Thread thread filha 02: 12
Suspendendo thread 01
Thread thread filha 02: 11
Retomando thread 01
Thread thread filha 01: 11
Suspendendo thread 02
Thread thread filha 01: 10
Thread thread filha 01: 9
Retomando thread 02
Thread thread filha 01: 8
Thread thread filha 02: 10
Esperando encerramento de threads...
Thread thread filha 01: 7
Thread thread filha 02: 9
Thread thread filha 01: 6
Thread thread filha 02: 8
Thread thread filha 01: 5
Thread thread filha 02: 7
Thread thread filha 01: 4
Thread thread filha 02: 6
Thread thread filha 01: 3
Thread thread filha 02: 5
Thread thread filha 01: 2
Thread thread filha 02: 4
Thread thread filha 01: 1
Thread thread filha 02: 3
Encerrando thread filha 01...
Thread thread filha 02: 2
Thread thread filha 02: 1
Encerrando thread filha 02...
Encerrando thread principal...
```



3.15.8 Definindo a prioridade das threads

O conceito de prioridade de threads é o que permite ao agendador de threads decidir quando cada uma das threads deve rodar. Em princípio, threads com prioridade mais elevada recebem mais tempo de CPU do que threads com prioridade mais baixa. Na prática, a quantidade de tempo de CPU que uma thread recebe depende de vários outros fatores, além de sua prioridade. Por exemplo, a forma como um sistema operacional implementa a multitarefa pode afetar a disponibilidade relativa do tempo de CPU. Uma thread de prioridade mais elevada pode também deslocar (*preempt*) uma thread com prioridade mais baixa.

Na teoria, threads com a mesma prioridade deveriam receber igual acesso à CPU.

Mas o programador precisa ser cauteloso. É preciso ter em mente que Java foi projetado para rodar numa variedade de sistemas diferentes. Alguns desses ambientes implementam a multitarefa de forma fundamentalmente diferente de outros. Por uma questão de segurança, o programador deve cuidar para que threads com a mesma prioridade cedam (*yield*) o controle de vez em quando. Isso assegura que todas as threads terão chance de rodar, mesmo em sistemas operacionais não-preemptivos. No mundo real, mesmo nos ambientes não-preemptivos todas as threads têm chance de rodar porque a maioria das threads inevitavelmente acaba chegando a alguma situação de bloqueio, como a espera por algum dispositivo de I/O. Quando isso acontece, a thread bloqueada é suspensa, permitindo que as outras threads rodem. Mas o melhor é não depender desse fato para assegurar uma execução tranqüila das múltiplas threads. Além disso, alguns tipos de tarefas utilizam intensamente a CPU. Threads desse tipo acabarão por dominar a CPU. Nesses casos, o melhor é ceder o controle de vez em quando para que as outras threads possam rodar.

Para definir a prioridade de uma thread, usamos o método `setPriority()`, definido na classe `Thread`. Eis sua forma geral:

```
final void setPriority(int nivel)
```

O parâmetro `nivel` especifica o valor da prioridade a ser aplicado à thread que o chama. O valor de `nivel` deve estar contido entre os valores `MIN_PRIORITY` e `MAX_PRIORITY`. Atualmente, esses valores são iguais a 1 e 10, respectivamente. Para retornar uma thread a sua prioridade default, devemos especificar o valor `NORM_PRIORITY`, atualmente igual a 5. Esses valores são definidos na forma de variáveis `final`, dentro da classe `Thread`.

Para obter o valor atual, dispomos do método `getPriority()`, também de `Thread`, mostrado abaixo:

```
final int getPriority()
```

Atualmente, as implementações de Java diferem muito em seu comportamento com relação ao agendamento. A versão de Java para Windows 95/98 funciona aproximadamente da forma que seria de se esperar. Contudo, a versão para Sun Solaris funciona de forma bem diferente. Geralmente, as inconsistências aparecem quando as threads



dependem do comportamento preemptivo do sistema operacional, ao invés de ceder o tempo da CPU de forma cooperativa. Portanto, para obter um comportamento coerente nas várias plataformas, devemos usar threads que voluntariamente cedam o controle da CPU.

O exemplo abaixo ilustra o uso de duas threads com prioridades diferentes. Esse é o tipo de programa que poderia exibir diferenças de comportamento nas duas plataformas que mencionamos acima. Para evitar isso, as threads filhas cedem o controle, a intervalos regulares. Uma das threads tem sua prioridade definida como sendo dois níveis acima da prioridade default, representada por `Thread.NORM_PRIORITY`. A outra thread tem sua prioridade definida como dois níveis abaixo de `Thread.NORM_PRIORITY`. No programa, as threads são iniciadas e rodam por 10 segundos. Cada thread executa um loop, contando o número de iterações. Cada vez que a contagem chega a um múltiplo de 1000, a thread cede voluntariamente o controle, chamando o método `yield()`. Depois de 10 segundos, a thread principal pára as duas threads filhas. O número de vezes que cada thread passou pelo loop é então exibido.

```
class PriorThread implements Runnable {
    int contador = 0;
    Thread thr;
    private boolean rodando = true;
    public PriorThread(int prior)
    {
        thr = new Thread(this);
        thr.setPriority(prior);
    } // Fim do construtor PriorThread()
    public void run()
    {
        while(rodando)
        {
            contador++;
            if((contador % 1000) == 0)
                thr.yield();
        } // Fim de while.
    } // Fim de run()
    public void start()
    {
        thr.start();
    } // Fim de start()
    public void stop()
    {
        rodando = false;
    } // Fim de stop()
} // Fim de class PriorThread.

class PriorThreadDemo {
    public static void main(String args[])
    {
        Thread.currentThread().setPriority(Thread.MAX_PRIORITY);
        PriorThread alta = new PriorThread(Thread.NORM_PRIORITY + 2);
        PriorThread baixa = new PriorThread(Thread.NORM_PRIORITY - 2);
        System.out.println("Aguarde...Rodando...");
        baixa.start();
        alta.start();
        try
        {
            Thread.sleep(10000);
        } // Fim de try.
    }
}
```



```
catch(InterruptedException e)
{
    System.out.println(
        "Thread principal interrompida!!!");
    // Fim de catch.
    baixa.stop();
    alta.stop();
    // Espera encerramento das threads filhas.
    try
    {
        alta.thr.join();
        baixa.thr.join();
    } // Fim de try.
    catch(InterruptedException e)
    {
        System.out.println(
            "Ocorreu InterruptedException!");
    } // Fim de catch.
    System.out.println(
        "Contagem de baixa: " + baixa.contador);
    System.out.println(
        "Contagem de alta: " + alta.contador);
} // Fim de main()
} // Fim de class PriorThreadDemo.
```

Saída gerada por este programa:

```
C:\$user\NovoCursoJava\Source>java PriorThreadDemo
Aguarde...Rodando...
Contagem de baixa: 19000
Contagem de alta: 58865000
```

A saída mostrada acima foi obtida com o programa rodando sob Windows 95. Ela indica a forma como a diferença nas prioridades afetam o chaveamento de contexto. É claro que os números exatos em cada execução do programa dependem de uma série de variáveis, como velocidade da CPU, quantidade de memória e sistema operacional.

Uma experiência interessante é remover as linhas:

```
if((contador % 1000) == 0)
    thr.yield();
```

Com isso, as threads deixarão de ceder o controle voluntariamente, e o programa passará a depender do sistema operacional para fazer o chaveamento entre as threads.

Atenção: esta alteração pode causar um comportamento imprevisível, dependendo do sistema operacional. A conclusão é que é importante ter em mente que não se deve escrever código que dependa do comportamento preemptivo do sistema operacional. Se você fizer isso, seu programa poderá rodar bem numa plataforma (exemplo: Windows 95/98) e ter problemas em outras plataformas (exemplo: Sun Solaris).

Provavelmente, no futuro, com o amadurecimento das implementações de Java esse problema deixará de existir.



3.15.9 Detalhes sobre a sincronização

Muitas vezes, durante a execução de um programa, duas ou mais threads precisam acessar um recurso compartilhado. Por isso, precisamos de uma forma de assegurar que esse recurso somente seja usado por uma thread de cada vez. Isso é conseguido por meio de um processo chamado sincronização. Veremos que Java oferece suporte a nível de linguagem para facilitar a sincronização.

Um conceito chave na sincronização é o **monitor**, às vezes chamado de semáforo (*semaphore*). Um monitor é um objeto que é usado como uma trava mutuamente exclusiva, ou *mutex*. Somente uma thread de cada vez pode estar de posse do monitor.

Quando uma thread obtém um recurso e bloqueia o acesso, impedindo que outras threads acessem esse recurso, dizemos que a thread *entrou* no monitor. Todas as outras threads que tentarem entrar nesse monitor que está bloqueado ficarão suspensas até que a primeira thread saia do monitor. Podemos também dizer que as outras threads estão *esperando* pelo monitor. A thread que está dentro do monitor pode re-entrar nele, se assim quiser.

Quem já trabalhou com sincronização usando outras linguagens, como C ou C++, sabe que isso pode ser um tanto complicado. Isso acontece porque a maioria das outras linguagens não oferece suporte a sincronização. Nesses casos, os programas precisam utilizar primitivas do sistema operacional para poder sincronizar as threads. Felizmente, como Java já oferece suporte a sincronização com elementos da própria linguagem, a maior parte da complexidade associada com a sincronização foi eliminada.

Existem duas maneiras de sincronizar o código. Ambas envolvem o uso da palavra-chave `synchronized`. A seguir, detalharemos essas duas formas de sincronização.

3.15.9.1 Usando métodos sincronizados

O motivo pelo qual é fácil usar sincronização em Java deve-se ao fato de que **cada objeto têm seu próprio monitor** implícito associado a ele. Para entrar no monitor de um objeto, **basta chamar um método que tenha sido criado com o modificador `synchronized`**. Enquanto uma thread está dentro de um método `synchronized`, todas as outras threads que tentarem chamar esse método (ou qualquer outro método `synchronized`) **da mesma instância** terão de esperar. Para sair do monitor, cedendo o controle do objeto para a próxima thread que está à espera, a thread que está de posse do monitor simplesmente **retorna do método `synchronized`**.

Para entender a necessidade da sincronização, começaremos com um exemplo simples que não a utiliza, mas que deveria. O programa abaixo tem três classes simples.

A primeira classe, cujo nome é `Chamada`, tem um único método, `chamado()`. O método `chamado()` recebe um único parâmetro do tipo `String`, chamado `mensagem`. O método



`chamado()` tenta exibir a `String` mensagem dentro de um par de colchetes `[ ]`. Observe que após a exibição do primeiro colchete `[` juntamente com a `String` mensagem, o método `chamado()` usa o método `Thread.sleep(1000)`, fazendo com que a thread atual entre em pausa por 1 segundo.

O construtor da segunda classe, cujo nome é `Chamadora`, recebe uma referência a uma instância da classe `Chamada` (atribuída à variável `alvo`), juntamente com uma `String` (atribuída à variável `mensagem`). Esse construtor também cria uma nova thread, que chamará o método `run()` deste objeto, via `start()`. Essa thread é imediatamente iniciada.

O método `run()` da classe `Chamadora` chama o método `chamado()` da instância `alvo` de `Chamada`, passando para ele a `String` `mensagem`. Finalmente, a classe `SemSynch` começa criando uma única instância de `Chamada`, e três instâncias de `Chamadora`, cada uma com uma string de mensagem diferente. A mesma instância de `Chamada` é passada para todas os objetos `Chamadora`.

```
class Chamada {
    void chamado(String mensagem)
    {
        System.out.print("[ " + mensagem);
        try
        {
            Thread.sleep(1000);
        } // Fim de try.
        catch (InterruptedException e)
        {
            System.out.println("Interrompida");
        } // Fim de catch.
        System.out.println("]");
    } // Fim de chamado()
} // Fim de class Chamada.

class Chamadora implements Runnable {
    String mensagem;
    Chamada alvo;
    Thread thr;
    public Chamadora(Chamada alv, String msg)
    {
        alvo = alv;
        mensagem = msg;
        thr = new Thread(this);
        thr.start();
    } // Fim do construtor Chamadora()
    public void run()
    {
        alvo.chamado(mensagem);
    } // Fim de run()
} // Fim de class Chamadora.

class SemSynch {
    public static void main(String args[])
    {
        Chamada alvoChamada = new Chamada();
        Chamadora ch1 = new Chamadora(alvoChamada, "Uma");
        Chamadora ch2 = new Chamadora(alvoChamada, "frase");
        Chamadora ch3 = new Chamadora(alvoChamada, "nao-sincronizada");
        // Espera encerramento das threads.
        try
            ch1.thr.join();
```



```
ch2.thr.join();
ch3.thr.join();
} // Fim de try.
catch (InterruptedException e)
{
    System.out.println("Interrompida");
} // Fim de catch.
} // Fim de main()
} // Fim de class SemSynch.
```

Saída gerada por este programa:

```
C:\$user\NovoCursoJava\Source>java SemSynch
[Uma[frase[nao-sincronizada] ] ]
```

Vemos que, ao chamar o método `sleep()`, o método `chamado()` permite que a execução passe para outra thread. Isso resulta na forma misturada como as três strings são exibidas. Neste programa, nada impede que as três threads chamem o mesmo método, no mesmo objeto, ao mesmo tempo. Isso é conhecido como *race condition* (*condição de corrida*), porque as três threads estão correndo umas contra as outras, competindo pelo método. Neste exemplo, usamos `sleep()` para tornar os efeitos bastante claros. Na maioria das situações do mundo real, uma *race condition* acontece de forma mais sutil e imprevisível, porque não há uma certeza sobre o momento em que acontecerá o chaveamento de contexto. Isso pode fazer com que o programa funcione corretamente algumas vezes, e desastrosamente em outras.

Para solucionar esse problema, precisamos serializar o acesso a `chamado()`. Ou seja, precisamos restringir o acesso a `chamado()` a uma thread de cada vez. Para isso, basta simplesmente iniciar a definição de `chamado()` com a palavra-chave `synchronized`, da forma mostrada abaixo. A definição do método `chamado()` como `synchronized` impedirá que outras threads entrem em `chamado()` enquanto uma thread estiver utilizando esse método.

```
class Chamada {
    synchronized void chamado(String mensagem)
    {
        ...
    }
}
```

A versão completa do programa que implementa esta modificação é mostrada abaixo (o programa foi rebatizado de `ComSynch.java`):

```
class Chamada {
    synchronized void chamado(String mensagem)
    {
        System.out.print "[" + mensagem);
        try
        {
            Thread.sleep(1000);
        } // Fim de try.
        catch (InterruptedException e)
        {
            System.out.println("Interrompida");
        } // Fim de catch.
        System.out.println("]");
    } // Fim de chamado()
} // Fim de class Chamada.
```



```
class Chamadora implements Runnable {
    String mensagem;
    Chamada alvo;
    Thread thr;
    public Chamadora(Chamada alv, String msg)
    {
        alvo = alv;
        mensagem = msg;
        thr = new Thread(this);
        thr.start();
    } // Fim do construtor Chamadora()
    public void run()
    {
        alvo.chamado(mensagem);
    } // Fim de run()
} // Fim de class Chamadora.

class ComSynch {
    public static void main(String args[])
    {
        Chamada alvoChamada = new Chamada();
        Chamadora chl = new Chamadora(alvoChamada, "Uma");
        Chamadora ch2 = new Chamadora(alvoChamada, "frase");
        Chamadora ch3 = new Chamadora(alvoChamada, "sincronizada");
        // Espera encerramento das threads.
        try
        {
            chl.thr.join();
            ch2.thr.join();
            ch3.thr.join();
        } // Fim de try.
        catch(InterruptedException e)
        {
            System.out.println("Interrompida");
        } // Fim de catch.
    } // Fim de main()
} // Fim de class ComSynch.
```

Saída gerada por este programa:

```
C:\$user\NovoCursoJava\Source>java ComSynch
[Uma]
[frase]
[sincronizada]
```

A conclusão importante aqui é que sempre que tivermos um ou mais métodos que manipulem o estado interno de um objeto numa situação multithreaded, devemos usar a palavra-chave `synchronized` para evitar a ocorrência de race conditions. Resumindo: sempre que uma thread entra num método `synchronized` de uma instância, nenhuma outra thread pode entrar em nenhum outro método `synchronized` **da mesma instância**.

Contudo, métodos não-`synchronized` daquela instância continuam disponíveis para serem chamados.



3.15.9.2 Outro uso de synchronized

Vimos que a criação de métodos `synchronized` dentro de classes é uma forma fácil e eficaz de obter a sincronização. Porém, isso não resolve todos os casos. Para entender por quê, considere a seguinte situação. Digamos que você precisa sincronizar o acesso a objetos de uma classe que **não foi criada** tendo em vista a execução multithreaded. Por exemplo, esta classe pode ter sido criada por outro programador, e você não tem acesso ao código fonte. Por isso, você não pode modificar a classe, declarando seus métodos como `synchronized`. Como assegurar o acesso sincronizado a objetos dessa classe?

Java oferece uma solução bastante simples para esse problema. Podemos **declarar um bloco** como `synchronized`. Em seguida, colocamos dentro do bloco `synchronized` as chamadas aos métodos da classe não-`synchronized`. Isso é feito da seguinte maneira:

```
synchronized(objeto)
{
    // Comandos a serem sincronizados.
} // Fim do bloco synchronized.
```

Observe que `objeto` é uma referência ao objeto que está sendo sincronizado.

Se houver necessidade de sincronizar apenas um comando, não é preciso colocar o bloco entre as chaves `{ e }`.

A presença de um bloco `synchronized` assegura que a chamada a um método que é membro de objeto somente acontecerá quando a thread atual conseguir entrar no monitor do objeto.

Eis uma nova versão do programa anterior. Aqui, usamos um bloco `synchronized` dentro do método `run()`.

```
class Chamada {
    // Aqui, o método chamado()
    // NÃO é synchronized.
    void chamado(String mensagem)
    {
        System.out.print("[ " + mensagem);
        try
        {
            Thread.sleep(1000);
        } // Fim de try.
        catch (InterruptedException e)
        {
            System.out.println("Interrompida");
        } // Fim de catch.
        System.out.println("]");
    } // Fim de chamado()
} // Fim de class Chamada.

class Chamadora implements Runnable {
    String mensagem;
    Chamada alvo;
    Thread thr;
    public Chamadora(Chamada alv, String msg)
    {
        alvo = alv;
        mensagem = msg;
    }
}
```



```
thr = new Thread(this);
thr.start();
} // Fim do construtor Chamadora()
public void run()
{
    // Este bloco sincroniza as chamadas
    // ao método chamado()
    // (não-synchronized).
    synchronized(alvo)
    {
        alvo.chamado(mensagem);
    } // Fim do bloco synchronized.
} // Fim de run()
} // Fim de class Chamadora.

class BlocoSynch {
    public static void main(String args[])
    {
        Chamada alvoChamada = new Chamada();
        Chamadora ch1 = new Chamadora(alvoChamada, "Uma");
        Chamadora ch2 = new Chamadora(alvoChamada, "frase");
        Chamadora ch3 = new Chamadora(alvoChamada, "sincronizada");
        // Espera encerramento das threads.
        try
        {
            ch1.thr.join();
            ch2.thr.join();
            ch3.thr.join();
        } // Fim de try.
        catch (InterruptedException e)
        {
            System.out.println("Interrompida");
        } // Fim de catch.
    } // Fim de main()
} // Fim de class BlocoSynch.
```

Saída gerada por este programa:

```
C:\$user\NovoCursoJava\Source>java BlocoSynch
[Uma]
[frase]
[sincronizada]
```

Neste exemplo, o método `chamado()` não é `synchronized`. Por isso, usamos um bloco `synchronized` dentro do método `run()` da classe `Chamadora`. Isso faz com que a saída seja exibida corretamente, conforme mostrado acima, porque cada thread espera que a outra termine seu trabalho antes de prosseguir.



3.15.10 Comunicação inter-threads

Vimos que a existência de monitores implícitos nos objetos Java é um recurso poderoso e fácil de usar. Porém existe um nível de controle mais refinado, que pode ser obtido pela comunicação entre threads. Veremos que esse é um recurso igualmente fácil de usar em Java.

Conforme dissemos anteriormente, o uso de multithreading permite a substituição de loops pela divisão das tarefas em unidades discretas e lógicas. As threads oferecem ainda um benefício adicional: elas acabam com a necessidade de polling. O polling é uma técnica que consiste em ficar repetidamente checando uma determinada condição.

Geralmente isso é conseguido por meio de um loop. Quando a condição torna-se verdadeira, a ação correspondente é efetuada. Isso é um desperdício de tempo de CPU.

Considere, por exemplo, um problema clássico de programação: as filas (*queues*).

Considere a situação em que uma thread produz dados que são consumidos por outra thread. Digamos que a thread produtora precisa esperar até que a thread consumidora termine de consumir os dados, antes de gerar mais dados. Num sistema de polling, a thread consumidora desperdiçaria preciosos ciclos de CPU, esperando que a thread produtora gerasse novos dados. Por sua vez, a thread produtora também desperdiçaria tempo de CPU, esperando que a thread consumidora consumisse os dados, e assim por diante. O resultado seria uma situação de muita ineficiência.

Para evitar a necessidade de polling, Java inclui um elegante mecanismo de comunicação inter-threads, usando os métodos `wait()`, `notify()` e `notifyAll()`. Esses métodos são implementados como métodos `final` na classe `Object`, de modo que estão disponíveis para todas as classes Java. Esses três métodos **somente podem ser chamados de dentro de um método `synchronized`**. Embora utilizem conceitos avançados do ponto de vista de Ciência da Computação, as regras para uso desses métodos são na verdade bastante simples:

Usando a comunicação inter-threads	
Método	Descrição
<code>final void wait()</code>	Manda a thread abrir mão do monitor e entrar em estado suspenso (<i>sleep</i>) até que alguma outra thread entre no mesmo monitor e chame o método <code>notify()</code> .
<code>final void notify()</code>	Retoma a primeira thread que chamou <code>wait()</code> para o mesmo objeto. A thread com prioridade mais elevada rodará primeiro.
<code>final void notifyAll()</code>	Retoma todas as threads que chamaram <code>wait()</code> para o mesmo objeto.

Esses métodos são declarados dentro da classe `Object`. Existem também outras formas de `wait()` que permitem especificar um período de espera.



O programa abaixo implementa uma solução simples, porém inadequada, para o problema produtora/consumidora. Consiste de quatro classes:

Classes do exemplo PCMain	
Classe	Descrição
Fila	A fila que estamos tentando sincronizar
Produtora	O objeto com thread que está produzindo itens para a fila
Consumidora	O objeto com thread que está consumindo itens da fila
PCMain	A classe que contém o método <code>main()</code> que lida com as três classes anteriores

Eis a listagem:

```
class Produtora implements Runnable {
    Fila fila;
    Produtora(Fila umaFila)
    {
        this.fila = umaFila;
        new Thread(this, "Produtora").start();
    } // Fim do construtor Produtora()
    public void run()
    {
        int i = 0;
        while(true)
        {
            fila.put(i++);
        } // Fim de while.
    } // Fim de run()
} // Fim de class Produtora.

class Consumidora implements Runnable {
    Fila fila;
    Consumidora(Fila umaFila)
    {
        this.fila = umaFila;
        new Thread(this, "Consumidora").start();
    } // Fim do construtor Consumidora()
    public void run()
    {
        while(true)
        {
            fila.get();
        } // Fim de while.
    } // Fim de run()
} // Fim de class Consumidora.

class Fila {
    int i;
    synchronized int get()
    {
        System.out.println("Got: " + i);
        return i;
    } // Fim de get()
    synchronized void put(int n)
    {
        this.i = n;
        System.out.println("Put: " + n);
    }
}
```




```
} // Fim de put()
} // Fim de class Fila.

class PCMain {
    public static void main(String args[])
    {
        Fila fila = new Fila();
        new Produtora(fila);
        new Consumidora(fila);
        System.out.println(
            "Pressione Ctrl-C para parar...");
    } // Fim de main()
} // Fim de class PCMain.
```

Saída gerada por este programa:

```
Got: 4
Put: 5
Put: 6
Got: 6
Got: 6
Got: 6
Got: 6
Put: 7
Put: 8
Got: 8
Put: 9
Put: 10
Put: 11
Put: 12
Got: 12
Got: 12
Got: 12
Got: 12
Got: 12
Put: 13
```

Podemos observar na amostra da saída gerada pelo programa que muitas vezes a thread consumidora consumiu várias vezes o mesmo valor (veja o valor 12). Por outro lado, em alguns momentos (veja os valores 9, 10 e 11) a thread produtora assumiu o controle produzindo vários valores sem permitir que a thread consumidora os consumisse.

A forma correta de escrever esse programa em Java seria usando o métodos `wait()` e `notify()` para sinalização de status entre as duas threads. A versão correta é mostrada abaixo:

```
// Implementação corrigida de thread
// produtora/consumidora.
class NovaFila
{
    int n;
    boolean vlrConsumido = false;
    synchronized int get()
    {
        if(!vlrConsumido)
            try
            {
                wait();
            } // Fim de try.
    }
}
```



UNIVERSIDADE DO AMAZONAS
FACULDADE DE TECNOLOGIA
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS



```
catch(InterruptedException e)
{
    System.out.println("Interrupcao tratada");
} // Fim de catch.
System.out.println("Got: " + n);
vlrConsumido = false;
notify();
return n;
} // Fim de get()
synchronized void put(int n)
{
    if(vlrConsumido)
        try
        {
            wait();
        } // Fim de try.
        catch(InterruptedException e)
        {
            System.out.println("Interrupcao tratada");
        } // Fim de catch.
    this.n = n;
    vlrConsumido = true;
    System.out.println("Put: " + n);
    notify();
} // Fim de put()
} // Fim de class NovaFila.
class NovaProdutora implements Runnable
{
    NovaFila nFila;
    NovaProdutora(NovaFila nf)
    {
        nFila = nf;
        new Thread(this, "NovaProdutora").start();
    } // Fim do construtor NovaProdutora()
    public void run()
    {
        int i = 0;
        while(true)
        {
            nFila.put(i++);
        } // Fim de while.
    } // Fim de run()
} // Fim de class NovaProdutora.
class NovaConsumidora implements Runnable
{
    NovaFila nFila;
    NovaConsumidora(NovaFila nf)
    {
        nFila = nf;
        new Thread(this, "NovaConsumidora").start();
    } // Fim do construtor NovaConsumidora()
    public void run()
    {
        while(true)
        {
            nFila.get();
        } // Fim de while.
    } // Fim de run()
} // Fim de class NovaConsumidora.
class NovaPCMain
{

```



UNIVERSIDADE DO AMAZONAS
FACULDADE DE TECNOLOGIA
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS



```
public static void main(String args[])
{
    NovaFila nFila = new NovaFila();
    new NovaProdutora(nFila);
    new NovaConsumidora(nFila);
    System.out.println(
        "Pressione Ctrl-C para parar...");
} // Fim de main()
} // Fim de class NovaPCMain.
```

Saída gerada por este programa:

```
C:\$user\NovoCursoJava\Source>java NovaPCMain
Pressione Ctrl-C para parar...
Put: 0
Got: 0
Put: 1
Got: 1
Put: 2
Got: 2
Put: 3
Got: 3
Put: 4
Got: 4
Put: 5
Got: 5
Put: 6
Got: 6
Put: 7
Got: 7
Put: 8
Got: 8
Put: 9
Got: 9
Put: 10
Got: 10
Put: 11
Got: 11
Put: 12
Got: 12
Put: 13
Got: 13
```

Dentro do método `get()` da classe `NovaFila`, o método `wait()` é chamado. Isso faz com que a execução seja suspensa até que a classe `NovaProdutora` notifique a existência de um dado pronto. Quando isso acontece, a execução é retomada dentro de `get()`.

Depois que os dados são obtidos, `get()` chama `notify()`. Isso informa a `NovaProdutora` que ela já pode colocar um novo dado na fila. Dentro de `put()`, `wait()` faz com que a execução seja suspensa até que `NovaConsumidora` tenha removido o item da fila.

Quando a execução é retomada, o item seguinte de dados é colocado na fila e `notify()` é chamado. Isso avisa a `NovaConsumidora` para remover o item.



3.15.11 Deadlock

Um erro muito comum em programação multithreaded é o chamado *deadlock*. O deadlock acontece quando duas threads têm uma dependência mútua **dentro de um par de objetos sincronizados**. Por exemplo, digamos que uma thread entra no monitor do objeto *A* e outra thread entra no monitor do objeto *B*. Se a thread que está em *A* tentar chamar qualquer método sincronizado do objeto *B*, ela ficará bloqueada. Ao mesmo tempo, se a thread que está em *B* tentar chamar qualquer método sincronizado do objeto *A*, ficará esperando para sempre, já que para acessar *A* precisaria liberar seu próprio lock sobre *B*, para que a primeira thread pudesse continuar. O deadlock é um erro particularmente complexo por dois motivos:

A complexidade do deadlock

Em geral, a condição de deadlock demora muito para acontecer, porque as duas threads precisam compartilhar o tempo de CPU exatamente de uma determinada forma.

O deadlock pode envolver mais de duas threads e mais de dois objetos sincronizados. Ou seja, o deadlock pode ocorrer como resultado de uma série de eventos muito mais complexa do que a descrita acima.

Para melhor entender o deadlock, vamos vê-lo acontecendo na prática. O exemplo abaixo cria duas classes, *DependA* e *DependB*. A classe *DependA* tem um método chamado *metDependA()*; a classe *DependB* tem um método chamado *metDependB()*. Cada método faz uma pequena pausa antes de tentar chamar um método da outra classe. A classe principal, chamada *DeadlockDemo*, cria uma instância de cada classe, *DependA* e *DependB*. Depois, dá partida numa nova thread, que criará a condição de deadlock. Os métodos *metDependA()* e *metDependB()* chamam *sleep()* para forçar a ocorrência da condição de deadlock.

```
class DependA {
    synchronized void metDependA(DependB db)
    {
        String nome = Thread.currentThread().getName();
        System.out.println(nome +
            " entrou em DependA.metDependA()...");
        try
        {
            Thread.sleep(1000);
        } // Fim de try.
        catch(Exception e)
        {
            System.out.println("Thread interrompida");
        } // Fim de catch.
        System.out.println(nome + " tentando chamar DependB.ultimo()");
        db.ultimo();
    } // Fim de metDependA()
    synchronized void ultimo()
    {
        System.out.println(
            "Estamos em DependA.ultimo()...");
    } // Fim de ultimo()
}
```



```
} // Fim de class DependA.

class DependB {
    synchronized void metDependB(DependA da)
    {
        String nome = Thread.currentThread().getName();
        System.out.println(nome +
            " entrou em DependB.metDependB()...");
        try
        {
            Thread.sleep(1000);
        } // Fim de try.
        catch(Exception e)
        {
            System.out.println("Thread interrompida");
        } // Fim de catch.
        System.out.println(nome + " tentando chamar DependA.ultimo()");
        da.ultimo();
    } // Fim de metDependB()
    synchronized void ultimo()
    {
        System.out.println(
            "Estamos em DependB.ultimo()...");
    } // Fim de ultimo()
} // Fim de class DependB.

class DeadlockDemo implements Runnable {
    DependA da = new DependA();
    DependB db = new DependB();
    DeadlockDemo()
    {
        Thread.currentThread().setName("ThreadPrincipal");
        Thread thr = new Thread(this, "RacingThread");
        thr.start();
        // Obtém lock sobre da
        // nesta thread.
        da.metDependA(db);
        System.out.println("Voltamos a thread principal...");
    } // Fim do construtor DeadlockDemo()
    public static void main(String args[])
    {
        new DeadlockDemo();
    } // Fim de main().
    public void run()
    {
        // Obtém lock sobre db
        // em outra thread.
        db.metDependB(da);
        System.out.println("Voltamos a outra thread...");
    } // Fim de run()
} // Fim de class DeadlockDemo.
```

Saída gerada por este programa:

```
C:\$user\NovoCursoJava\Source>java DeadlockDemo
ThreadPrincipal entrou em DependA.metDependA()...
RacingThread entrou em DependB.metDependB()...
ThreadPrincipal tentando chamar DependB.ultimo()
RacingThread tentando chamar DependA.ultimo()
```



Como o programa entra em deadlock, será preciso pressionar `Ctrl-C` para terminá-lo. Se o programa estiver rodando no ambiente Windows, é possível ver listagem completa do status das threads e dos monitores pressionando `Ctrl-Break`. No ambiente Sun Solaris, pressione `Ctrl-\` para obter o mesmo efeito. Vemos abaixo que `RacingThread` está de posse do monitor de `DependB`, enquanto espera para obter `DependB`.

Monitor Cache Dump:

```
DependA@E64080/EAEA68: owner "ThreadPrincipal" (0x7917d0, 1 entry)
DependB@E64070/EAEAE0: owner "RacingThread" (0x7994f0, 1 entry)
```

A execução do programa nunca se completa.

Conforme ilustra esse exemplo, se um programa multithreaded trava de vez em quando, um bom lugar para começar a investigar é a possibilidade de estar acontecendo uma condição de deadlock.

3.15.12 Usando multithreading

Para a maioria dos programadores, a existência de suporte a multithreading na própria linguagem é uma novidade. A chave para uma utilização eficiente deste suporte é passar a pensar em termos de simultaneidade, e não mais de serialidade. Por exemplo, quando temos dois subsistemas dentro de um programa que podem ser executados simultaneamente, esses subsistemas são candidatos ao uso de threads. Entretanto, é preciso certa precaução no uso de threads. Um excesso de threads pode degradar a performance ao invés de melhorá-la. Lembre-se que existe certa sobrecarga associada ao chaveamento de contexto. Se tivermos threads demais, a CPU passará mais tempo gerenciando o chaveamento de contexto do que executando as threads propriamente ditas.



3.16 DESENVOLVIMENTO DE APLICAÇÕES GRÁFICAS EM JAVA

A linguagem Java oferece, dentre as funcionalidades incorporadas à sua API padrão, um extenso conjunto de classes e interfaces para o desenvolvimento de aplicações gráficas. Esse conjunto facilita a criação de saídas na forma gráfica e de interfaces gráficas com usuários (GUIs), tanto na forma de aplicações autônomas como na forma de applets .

Aplicações gráficas são criadas através da utilização de componentes gráficos e recursos associados, que em Java estão agrupados em dois grandes pacotes, `java.awt` e `javax.swing`.

AWT é o Abstract Windowing Toolkit, sendo definido através das classes do pacote `java.awt` e seus subpacotes, tais como `java.awt.event` e `java.awt.color`. As classes desses pacotes agrupam as funcionalidades gráficas que estão presentes desde as primeiras versões de Java, que operam tendo por base as funcionalidades do gerenciador de janelas e bibliotecas nativas do sistema onde a aplicação é executada.

Já o framework Swing, associado ao pacote `javax.swing` e seus subpacotes, é uma extensão definida a partir de AWT que define componentes gráficos que utilizam exclusivamente Java (lightweight components), com funcionalidades e aparência independentes do sistema onde a aplicação é executada. Na plataforma Java 2, Swing passou a fazer parte da distribuição padrão.

Obs.: Para mais informações acesse os sites:

<http://java.sun.com/j2se/1.3/docs/api/java/awt/package-summary.html>

<http://java.sun.com/j2se/1.3/docs/api/javax/swing/package-summary.html>

<http://java.sun.com/j2se/1.4/docs/api/java/awt/event/package-summary.html>

<http://java.sun.com/j2se/1.4/docs/api/java/awt/color/package-summary.html>



3.16.1 Saída na forma Gráfica

A base para tudo que é desenhado na tela de uma aplicação gráfica é o contexto gráfico, um objeto da classe Graphics. São nesses objetos que linhas, retângulos, círculos, strings, etc. são desenhados, compondo a apresentação visual da aplicação gráfica.

Todo componente gráfico tem associado a si um contexto gráfico e alguns métodos-chaves para manipulá-lo. A um objeto da classe Component pode-se aplicar os métodos `getGraphics()`, que permite obter uma referência para um objeto da classe Graphics, `paint(Graphics)`, que determina o que é feito no contexto gráfico, e `repaint()`, que solicita uma atualização no contexto gráfico.

Para determinar o conteúdo gráfico do contexto, os métodos da classe Graphics são utilizados. Esses métodos incluem facilidades para definição de cores, fontes e definição de figuras geométricas. O sistema de coordenadas para definir a posição do desenho no contexto tem origem no canto superior esquerdo da área gráfica, tomando valores inteiros para x e y representando quantidades de pixels na horizontal e na vertical, respectivamente.

A cor corrente para o contexto gráfico pode ser obtida e definida pelos métodos `getColor()` e `setColor()`, respectivamente. Esses métodos usam cores representadas por objetos da classe Color, que oferece constantes para usar cores pré-definidas, métodos para definir cores especificando níveis RGB (red, green, blue) ou HSB (hue, saturation, brightness) e métodos para manipular as cores existentes.

Similarmente, a fonte usada para a apresentação de strings no contexto pode ser obtida e definida através de métodos `getFont()` e `setFont()`. Fontes são representadas por objetos da classe Font.

Entre os diversos métodos usados para desenhar no contexto, há métodos para desenhar texto usando a fonte de texto e a cor corrente, `drawString()`; desenhar linhas retas, `drawLine()`; e desenhar diversas formas geométricas, tais como `drawArc()`, `drawRect()`, `drawRoundRect()`, `draw3DRect()`, `drawPolygon()`, `drawOval()` e suas correspondentes versões preenchidas, `fillArc()`, `fillRect()`, `fillRoundRect()`, `fill3DRect()`, `fillPolygon()`, `fillOval()`.

Essa imagem mostra o resultado da execução de um código que apresenta um texto e desenha alguns retângulos em uma janela gráfica, ilustrando a utilização de alguns dos métodos aqui apresentados:



Código:

```
import java.awt.*;
import java.awt.event.*;

public class JanelaDesenho extends Frame {
    private final int startPos = 50;
    private int incremento = 15;
    private int tamanho = 40;

    class WindowHandler extends WindowAdapter {
        public void windowClosing(WindowEvent we) {
            dispose();
            System.exit(0);
        }
    }

    public JanelaDesenho() {
        setTitle("Janela e desenho");
        setSize(320,200);
        addWindowListener(new WindowHandler());
    }

    public void paint(Graphics g) {
        int x=startPos;
        int y=startPos;

        Dimension d = getSize();
        int finalPos = Math.min(d.width, d.height)-tamanho;

        g.drawString("Desenhando...", 10, 40);

        while (x < finalPos) {
            g.setColor(new Color((float)Math.random(),
```

```

        (float)Math.random(),
        (float)Math.random()));
    g.drawRect(x, y, tamanho, tamanho);
    x += incremento;
    y += incremento;
}
g.setFont(g.getFont().deriveFont(Font.BOLD));
g.setColor(Color.red);
g.drawString("...pronto!", 100, 40);
}

public static void main(String[] args) {
    JanelaDesenho j = new JanelaDesenho();
    j.setVisible(true);
}
}

```

Este outro exemplo ilustra o uso de recursos disponibilizados, na plataforma Java2, através da classe `java.awt.Graphics2D` -- neste caso, o uso de linhas grossas para fazer o desenho dos retângulos:



Código:

```

// Uso de Graphics2D

import java.awt.*;
import java.awt.event.*;

public class JanelaDesenho2 extends Frame {
    private final int startPos = 50;
    private int incremento = 15;
    private int tamanho = 40;

    class WindowHandler extends WindowAdapter {
        public void windowClosing(WindowEvent we) {
            dispose();
        }
    }
}

```



```
        System.exit(0);
    }
}

public JanelaDesenho2() {
    setTitle("Janela e desenho 2");
    setSize(320,200);
    addWindowListener(new WindowHandler());
}

public void paint(Graphics g) {
    int x=startPos;
    int y=startPos;

    Dimension d = getSize();
    int finalPos = Math.min(d.width, d.height)-tamanho;

    Graphics2D g2 = (Graphics2D) g;

    g2.setStroke(new
        BasicStroke(10.0f, BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND));

    g2.drawString("Desenhando...", 10, 40);

    while (x < finalPos) {
        g2.setColor(new Color((float)Math.random(),
                               (float)Math.random(),
                               (float)Math.random()));
        g2.drawRect(x, y, tamanho, tamanho);
        x += incremento;
        y += incremento;
    }
    g2.setFont(g.getFont().deriveFont(Font.BOLD));
    g2.setColor(Color.red);
    g2.drawString("...pronto!", 100, 40);
}

public static void main(String[] args) {
    JanelaDesenho2 j = new JanelaDesenho2();
    j.setVisible(true);
}
}
```

Para obter uma referência para o contexto Graphics2D, basta fazer o downcast a partir da referência para o objeto Graphics.



3.16.1.1 Componentes gráficos

Java oferece um amplo conjunto de classes pré-definidas e re-utilizáveis que simplificam o desenvolvimento de interfaces gráficas com usuários (GUI). A raiz desse conjunto de classes gráficas no pacote `java.awt` é a classe abstrata `Component`, que representa qualquer objeto que pode ser apresentado na tela e ter interação com usuários. Componentes definidos no pacote `Swing` têm por raiz a classe `JComponent`, uma extensão de `Component`.

Subclasses de `Component` que são amplamente utilizadas no desenvolvimento de aplicações gráficas incluem os containers, um objeto gráfico que pode conter outros componentes, e os componentes de interface gráfica com usuários.

A classe `Component` define propriedades e funcionalidades que dizem respeito à manipulação de qualquer componente gráfico em Java. Por exemplo, todo componente tem uma propriedade que é seu tamanho. O método `getSize()` permite obter as dimensões do componente, expressas na forma de um objeto da classe `Dimension` -- um objeto dessa classe tem campos públicos `width` e `height` que indicam respectivamente as dimensões horizontais e verticais do componente em pixels. Já o método `setSize()` permite definir a dimensão do componente.

Componentes são objetos que geram eventos, os quais serão relevantes para determinar como cada elemento da GUI irá responder às ações dos usuários. Alguns eventos são genéricos e associados a qualquer tipo de componente, como por exemplo a detecção de eventos de mouse (movimentação, "cliques") e de teclado. Outros estão associados a alguns tipos de componentes, apenas.

Obs: mais informações no site:

<http://java.sun.com/j2se/1.4.2/docs/api/javawx/swing/package-frame.html>

<http://java.sun.com/j2se/1.4.2/docs/api/java/awt/package-frame.html>

<http://java.sun.com/j2se/1.4.2/docs/api/java/awt/Graphics.html>



3.16.1.1.1 CONTAINERS

Difícilmente uma aplicação gráfica é composta por um único componente, mas sim por vários componentes inter-relacionados. Para este tipo de aplicação, um componente fundamental é a área onde os demais componentes da aplicação estarão dispostos. Um componente que pode conter outros componentes é denominado um container.

Em Java, a classe Container é a classe abstrata que define as funcionalidades básicas associadas a um container, tais como adicionar e remover componentes, o que é possível através dos métodos `add()` e `remove()`, respectivamente. É possível também estabelecer qual a estratégia de disposição de componentes no container, ou seja, qual o método de gerência de layout, através do método `setLayout()`.

Window é uma classe derivada de Container cujos objetos estão associadas a janelas. Cada objeto Window é uma janela independente em uma aplicação, embora a essa janela não estejam associadas as funcionalidades usualmente oferecidas por gerenciadores de janela. Raramente um objeto desse é usado diretamente, mas objetos dessa classe são muito utilizados através de suas subclasses, tais como Frame.

Outra classe derivada de Container de extensa aplicação é Panel, que define uma área de composição de componentes contida em alguma janela. A classe Applet é uma extensão de Panel que permite criar applets.

Embora a classe JComponent, raiz da hierarquia para todos os componentes do novo framework para interfaces gráficas de Java, seja derivada da classe Container, não se pode acrescentar diretamente um componente gráfico a qualquer componente Swing. Para as classes de Swing que correspondem a containers no sentido definido em AWT, ou seja, às quais podem ser acrescentados outros componentes, deve-se obter uma referência ao objeto Container através do método `getContentPane()`.

3.16.1.1.1.1 Gerenciadores de Layout

Quando um container tem mais de um componente, é preciso especificar como esses componentes devem ser dispostos na apresentação. Em Java, um objeto que implemente a interface `LayoutManager` é responsável por esse gerenciamento de disposição. A interface `LayoutManager2` é uma extensão de `LayoutManager` que incorpora o conceito de restrições de posicionamento de componentes em um container.

Os principais métodos da classe Container relacionados ao layout de componentes são:

- `void setLayout(LayoutManager l)`, recebe como argumento o objeto que implementa `LayoutManager` e que determina qual a política de gerência de disposição de componentes adotada;



- void validate(), usado para rearranjar os componentes em um container se o gerenciador de layout sofreu alguma modificação ou novos componentes foram adicionados.

O pacote java.awt apresenta vários gerenciadores de layout pré-definidos. As classes FlowLayout e GridLayout são implementações de LayoutManager; BorderLayout, CardLayout e GridBagLayout são implementações de LayoutManager2. Swing acrescenta ainda um gerenciador BoxLayout.

Para determinar que o gerenciador de layout em um container c seja do tipo XXXLayout, é preciso invocar setLayout() passando como argumento um objeto gerenciador:

```
c.setLayout(new XXXLayout());
```

Containers derivados da classe Window têm como padrão um gerenciador de layout do tipo BorderLayout, enquanto aqueles derivados de Panel usam como padrão FlowLayout.

3.16.1.1.1.1 BorderLayout

BorderLayout é uma implementação de LayoutManager2 adequado para janelas com até cinco componentes. Ele permite arranjar os componentes de um container em cinco regiões, cujo posicionamento é representado pelas constantes BorderLayout.NORTH, BorderLayout.SOUTH, BorderLayout.EAST, BorderLayout.WEST e BorderLayout.CENTER:



Código:

```
import java.awt.*;
```

```
public class JanelaBorder extends Frame {
```

```
    public JanelaBorder() {  
        setTitle("BorderLayout");  
        setSize(240,100);  
        add(new Button("North"), BorderLayout.NORTH);  
        add(new Button("South"), BorderLayout.SOUTH);  
        add(new Button("East"), BorderLayout.EAST);  
        add(new Button("West"), BorderLayout.WEST);  
        add(new Button("Center"), BorderLayout.CENTER);  
    }
```



```
public static void main(String[] args) {  
    JanelaBorder j = new JanelaBorder();  
    j.validate();  
    j.setVisible(true);  
}  
}
```

A janela acima foi criada com esse código. Observe a utilização do método `add(Component c, Object o)` da classe `Container`, que incorpora a especificação das restrições de posicionamento.

Além do construtor padrão, outro construtor permite especificar o gap horizontal e vertical (em pixels) entre os componentes.

Esse tipo de layout é o padrão para containers do tipo `Frame`. Para utilizar esse tipo de gerenciador para janelas com mais de cinco componentes, basta definir que o componente inserido em um `BorderLayout` seja um `Panel`, que é um container que pode ter seu próprio gerenciador de layout.

3.16.1.1.1.2 Windows (Janelas)

Janelas são áreas retangulares que podem ser exibidas em qualquer parte da tela gráfica de um usuário. Em AWT, objetos da classe `Window` estão associados a essa funcionalidade, sendo que nessa classe são definidos os métodos aplicáveis a qualquer tipo de janela. O pacote `Swing` oferece a classe `JWindow`, derivada da classe `Window`, com esse mesmo tipo de funcionalidade porém adaptada a esse novo framework.

Um objeto `Window`, no entanto, corresponde a uma janela sem nenhum tipo de "decoração". É mais comum que aplicações gráficas criem janelas através de uma das classes derivadas de `Window`. Um frame agrega a uma janela as funcionalidades mínimas para que ela possa ser identificada e manipulada pelo usuário da aplicação, tais como uma borda e a barra de títulos com botões de controle da janela. Outro tipo de janela existente em várias aplicações é a janela de diálogo.

Convém destacar da classe `Window` os métodos `show()`, para trazer a janela para a frente da tela; `dispose()`, para eliminar uma janela e liberar os recursos usados por ela; e `addWindowListener()` e `removeWindowListener()` para associar ou remover objetos manipuladores de eventos de janelas.

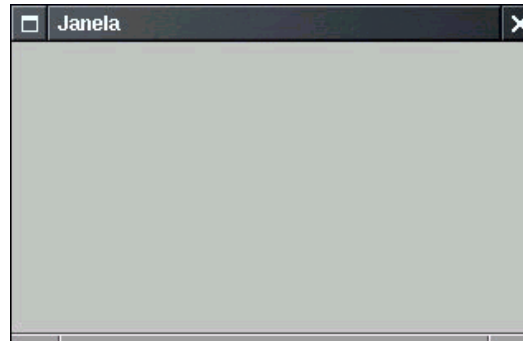
3.16.1.1.1.2.1 Frames

Um frame é uma janela com propriedades adicionais que a habilitam a ter uma "vida independente" no sistema de janelas gráficas. Para que um frame seja apresentado pela aplicação, um objeto desse tipo de classe deve ser criado e algumas de suas propri-



idades básicas -- tais como sua dimensão e o fato de estar visível -- devem ser estabelecidas.

Em AWT, frames estão associados a objetos da classe `Frame`. Esse exemplo ilustra as operações essenciais para que um frame AWT seja criado e exibido em um ambiente gráfico. O resultado da execução desse código é a criação e exibição de uma janela gráfica sem conteúdo:



Código:

```
import java.awt.*;
import java.awt.event.*;

public class Janela extends Frame {

    class WindowHandler extends WindowAdapter {
        public void windowClosing(WindowEvent we) {
            dispose();
            System.exit(0);
        }

        public void windowActivated(WindowEvent we) {
            we.getWindow().validate();
        }
    }

    public Janela() {
        this("Janela");
    }

    public Janela(String titulo) {
        setTitle(titulo);
        setSize(320,200);
        addWindowListener(new WindowHandler());
    }

    public static void main(String[] args) {
        Janela j = new Janela();
        j.setVisible(true);
    }
}
```


}

Se o tamanho da janela (em pixels, horizontal, vertical) não for definido (método `setSize()`), então o default ($x=0$, $y=0$) é assumido, criando uma janela de dimensões nulas. Mesmo que o tamanho seja definido, a janela quando criada é por padrão invisível -- para exibi-la é preciso invocar o método `setVisible()`.

Frame é uma extensão da classe `Window`. Esta, por sua vez, é uma concretização de `Container`, que é uma extensão de `Component`. Assim, é importante tomar contato com os diversos métodos definidos ao longo dessa hierarquia para ter uma noção mais precisa sobre o que é possível realizar com um objeto `Frame`. Por exemplo, para tratar eventos relativos a janelas é preciso associar ao frame um `WindowListener` através do método `addWindowListener()`, definido na classe `Window`.

A classe `Frame` propriamente dita define os métodos que dizem respeito ao que um frame tem de específico que uma janela (objeto da classe `Window`) não tem -- por exemplo, `setTitle()` para definir um título para a barra da janela, `setMenuBar()` para incluir uma barra de menus e `setCursor()` para determinar o tipo de cursor a ser utilizado, com alternativas especificadas como constantes da classe `Cursor`, tais como `Cursor.HAND_CURSOR` e `Cursor.TEXT_CURSOR`.

O pacote `Swing` também oferece a sua versão de um frame através da classe `JFrame`, que é uma extensão da classe `Frame`.

3.16.1.1.2.2 Janelas de diálogo

É usual que algumas aplicações gráficas precisem criar janelas temporárias, seja para transmitir uma mensagem ou para obter alguma informação ou seleção.

Em AWT, a classe `Dialog` oferece esse tipo de funcionalidade. Uma janela de diálogo é subordinada à janela principal da aplicação; por exemplo, se a janela principal for minimizada, a janela de diálogo também o será. Um objeto dessa classe pode ser configurado para diálogo modal (a aplicação não responde enquanto o diálogo não for concluído) ou não-modal (a aplicação pode continuar a responder ao usuário mesmo com a janela de diálogo aberta).

Esse exemplo ilustra a criação de uma janela de diálogo que tem apenas um botão, que deve ser pressionado para fechar a janela





Código:

```
import java.awt.*;
import java.awt.event.*;

public class JanelaDialog extends Frame {
    Dialog d;
    Frame f;

    class WindowHandler extends WindowAdapter {
        public void windowClosing(WindowEvent we) {
            Window w = we.getWindow();
            w.dispose();
            if (w == f)
                System.exit(0);
        }
    }

    class ShowHandler implements ActionListener {
        public void actionPerformed(ActionEvent ae) {
            d.show();
        }
    }

    class CloseHandler implements ActionListener {
        public void actionPerformed(ActionEvent ae) {
            d.dispose();
        }
    }

    public JanelaDialog() {
        d = new Dialog(this, true);
        WindowListener wl = new WindowHandler();
        addWindowListener(wl);
        d.addWindowListener(wl);
        d.setSize(100,100);
        d.setResizable(false);

        Button show = new Button("Mostra");
        show.addActionListener(new ShowHandler());
        add(show);
        pack();

        Button close = new Button("OK");
        close.addActionListener(new CloseHandler());
```

```
d.add(close);

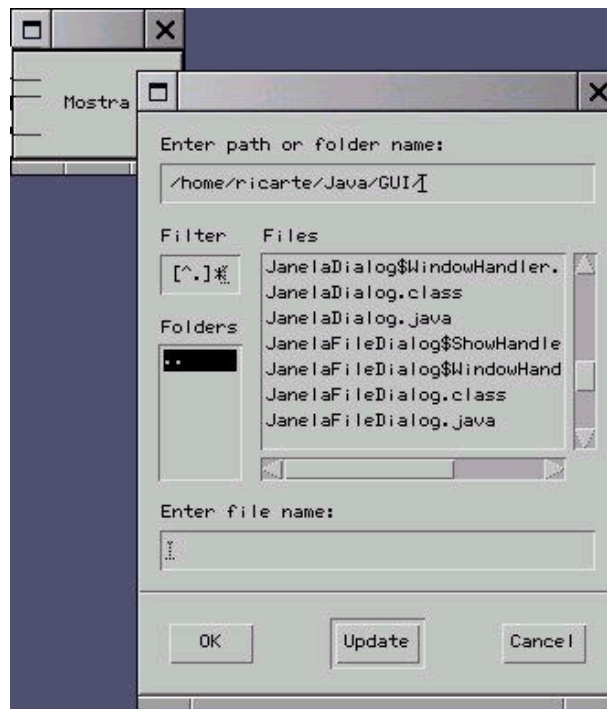
f = this;
}

public static void main(String[] args) {
    JanelaDialog jd = new JanelaDialog();
    jd.show();
}
}
```

A classe equivalente à essa em Swing é JDialog; uma alternativa simples para janelas pop-up é oferecida pelos métodos da classe JOptionPane.

Uma especialização de Dialog oferecida em AWT é a classe FileDialog cria uma caixa de diálogo que permite ao usuário da aplicação selecionar um arquivo. Essa classe oferece métodos que permite obter, após a seleção do usuário, strings com o caminho e o nome do arquivo selecionado, permitindo então operar com esse arquivo através das operações de entrada e saída.

Esse exemplo ilustra a utilização da classe FileDialog; nesse caso, o diretório e o nome do arquivo selecionado são simplesmente ecoados para o console:



Código:



```
import java.awt.*;
import java.awt.event.*;

public class JanelaFileDialog extends Frame {
    FileDialog d;
    Frame f;

    class WindowHandler extends WindowAdapter {
        public void windowClosing(WindowEvent we) {
            Window w = we.getWindow();
            w.dispose();
            if (w == f)
                System.exit(0);
        }
    }

    class ShowHandler implements ActionListener {
        public void actionPerformed(ActionEvent ae) {
            d.show();
            System.out.println("Diretorio: " + d.getDirectory());
            System.out.println("Arquivo: " + d.getFile());
        }
    }

    public JanelaFileDialog() {
        d = new FileDialog(this);
        WindowListener wl = new WindowHandler();
        addWindowListener(wl);
        d.addWindowListener(wl);
        d.setSize(200,100);

        Button show = new Button("Mostra");
        show.addActionListener(new ShowHandler());
        add(show);
        pack();

        f = this;
    }

    public static void main(String[] args) {
        JanelaFileDialog jd = new JanelaFileDialog();
        jd.show();
    }
}
```

Em Swing, funcionalidade equivalente é oferecida por JFileChooser, que no entanto não é uma especialização de JDialog. Outro tipo de janela de diálogo pré-estabelecida em Swing é JColorChooser, que permite à aplicação selecionar uma cor interativamente.



3.16.1.1.1.2.3 Manipuladores de eventos de janela

Eventos associados à manipulação de janelas são definidos na interface `WindowListener`. Essa interface especifica os seguintes métodos:

`windowOpened()`

trata evento que é apenas gerado quando a janela é criada e aberta pela primeira vez.

`windowClosing()`

usuário solicitou que a janela fosse fechada, seja através de um menu do sistema ou através de um botão da barra de títulos ou através de uma sequência de teclas do sistema.

`windowClosed()`

trata evento que indica que a janela foi fechada.

`windowIconified()`

trata evento que indica que janela foi iconificada.

`windowDeiconified()`

trata evento indicando que o ícone da janela foi aberto.

`windowActivated()`

trata evento que indica que a janela ganhou o foco do teclado e tornou-se a janela ativa.

`windowDeactivated()`

trata evento que indica que a janela deixou de ser a janela ativa, provavelmente porque outra janela foi ativada.

Todos esses métodos têm como argumento um objeto da classe `WindowEvent`, que representa um evento de janela.

Eventos de janela estão associados a objetos da classe `WindowEvent` -- um evento de janela indica que, de alguma forma, o estado da janela sob consideração foi alterado. Um objeto desse tipo inclui indicações de que a janela foi minimizada ou maximizada, recebeu ou perdeu o foco do mouse e se foi recebida uma solicitação para fechar a janela, entre outras. O método `getWindow()` retorna o objeto `Window` ao qual o evento está associado.

Esse tipo de evento é manipulado pelos métodos especificados em `WindowListener`.

Como a interface `WindowListener` especifica diversos métodos, uma classe adaptadora abstrata, `WindowAdapter`, é fornecida com implementações vazias de todos os métodos.

3.16.1.1.2 COMPONENTES DE INTERFACES GRÁFICAS COM USUÁRIOS

A criação de uma interface gráfica com usuários utiliza tipicamente uma série de componentes pré-estabelecidos para tal fim. Java, como qualquer linguagem de programação moderna, oferece vários componentes desse tipo em sua API. Um atrativo no de-



envolvimento de GUIs com Java é que essas aplicações podem ser executadas em várias plataformas.

O framework Swing de Java oferece componentes para distintas funcionalidades:

3.16.1.1.2.1 Exibir informação de orientação ao usuário:

Nesta categoria estão os componentes do tipo rótulo (label), dicas que surgem quando o mouse está sobre um determinado componente (tooltip) e bordas com título.

Também se encaixam nesta categoria as janelas para apresentação de mensagens.

3.16.1.1.2.1.1 Rótulos(Label)

Rótulos são componentes que têm uma apresentação gráfica que não reagem a interações com usuários. O objetivo principal de um rótulo é dar orientações ao usuário. A classe JLabel define o componente *Swing* para implementar rótulos. Um rótulo Swing pode estar associando tanto a texto como a imagem. Para criar um objeto rótulo com texto, é possível usar o construtor que recebe uma String ou usar o construtor padrão e depois o método `setText()`. Por exemplo, para criar um rótulo como abaixo, as duas possibilidades são:

	<pre>JLabel l1 = new JLabel(); l1.setText("Digite seu nome: ");</pre>	<pre>JLabel l1 = new JLabel("Digite seu nome: ");</pre>
---	---	---

Rótulos de imagem podem ser criados da mesma forma, sendo necessário neste caso criar o objeto que encapsula a imagem a ser apresentada (um objeto `javax.swing.Icon`). Por exemplo:

	<pre>Icon img = new ImageIcon("bighand.right.gif"); JLabel l2 = new JLabel(); l2.setIcon(img);</pre>	<pre>JLabel l2 = new JLabel(new ImageIcon("bighand.right.gif"));</pre>
---	--	--

Um mesmo rótulo pode ter as duas representações:

	<pre>Icon img = new ImageIcon("bighand.right.gif"); JLabel l3 = new JLabel(); l3.setText("Digite seu nome"); l3.setIcon(img);</pre>	<pre>JLabel l3 = new JLabel("Digite seu nome", new ImageIcon("bighand.right.gif"));</pre>
---	---	---

Outra propriedade de um rótulo é seu alinhamento (horizontal, vertical), que pode ser alterado com os métodos `setHorizontalAlignment()` e `setVerticalAlignment()`. A



relação entre o texto e o ícone, quando ambos estão presentes, pode ser alterada com os métodos `setHorizontalTextPosition()`, `setVerticalTextPosition()` e `setIconTextGap()`.

3.16.1.1.2.1.2 Dicas sobre componentes (tooltip)

Uma dica pode ser associada a qualquer componente Swing através do método `setToolTipText(String)`, onde a string passada como argumento contém o texto a ser exibido quando o mouse estiver sobre o componente. Por exemplo, considerando que um `JLabel` foi anteriormente criado e referenciado como `l1`, o efeito do comando

```
l1.setToolTipText("Digite seu nome aqui");
```

será



quando o mouse estiver sobre o componente rótulo (no exemplo, o ícone da mão apontando para a direita).

3.16.1.1.2.1.3 Bordas

Bordas podem ser associadas a qualquer componente Swing, havendo um grande conjunto de bordas pré-especificadas no framework. Um objeto borda é uma implementação da interface `javax.swing.border.Border`; na classe `javax.swing.BorderFactory` há diversos métodos para fabricar bordas de diferentes tipos.

O exemplo a seguir ilustra o uso de uma borda com título ao redor de um componente de campo de texto referenciado como `tf`:

```
tf.setBorder(BorderFactory.createTitledBorder("Nome:"));
```



3.16.1.1.2.1.4 Janelas de opções e mensagens

Swing oferece uma classe com vários tipos de janelas para obter entradas simples ou para enviar mensagens ao usuário: `JOptionPane`.

As janelas de `JOptionPane` enquadram-se em quatro grandes categorias:

Confirmação

Pede resposta a uma pergunta simples (sim, não, cancela)

Entrada

Pede alguma entrada textual simples

Mensagem

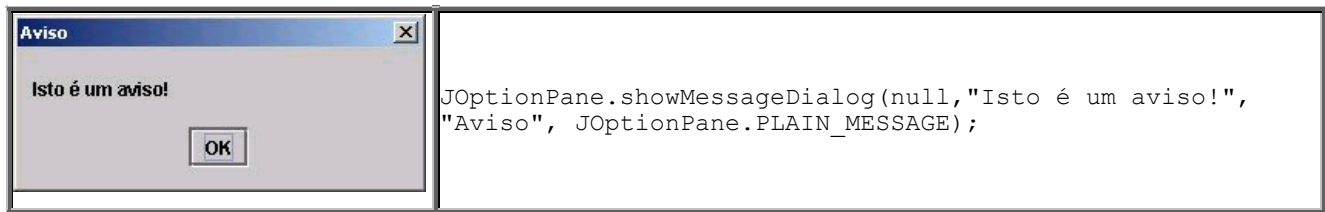
Avisa o usuário sobre alguma coisa que aconteceu

Opção

Uma janela genérica, juntando os aspectos acima.

Para cada tipo de janela, JOptionPane oferece métodos de fabricação de objetos desses diferentes tipos.

O exemplo abaixo ilustra a criação de uma janela de mensagem:



Outros métodos estáticos de JOptionPane incluem showConfirmDialog, showInputDialog e showOptionDialog.

Janelas de diálogo mais flexíveis e genéricas, com funcionalidade não tão específica, podem ser criadas através de objetos da classe JDialog.

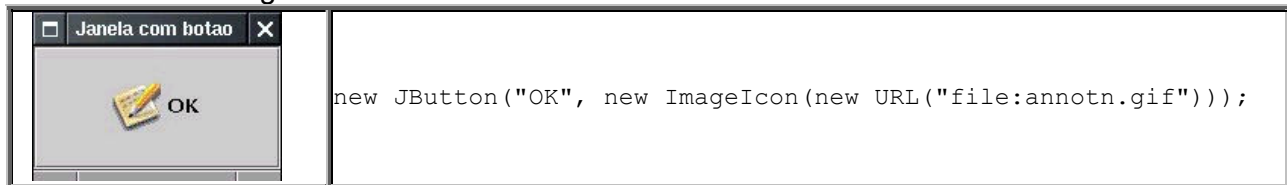
3.16.1.1.2.2 Disparar ações selecionadas pelo usuário:

Nesta categoria estão os componentes do tipo botão e menus.

3.16.1.1.2.2.1 Botões

O principal tipo de botão em Swing está associado à classe JButton. Um objeto dessa classe representa um botão que pode ser "pressionado" através de um "clique" do mouse, podendo assim responder à ativação por parte de um usuário.

Além de poder ser definido com rótulos de texto, um botão Swing pode conter também ícones. Esse exemplo ilustra a criação de uma janela com um botão contendo um texto e uma imagem:



3.16.1.1.2.2.2 Menu

Menus são criados em Java pela associação de vários objetos. O primeiro deles é a barra de menus, um objeto da classe `JMenuBar`. A barra de menus, por sua vez, pode conter vários objetos do tipo `JMenu`, que estão associados aos nomes que aparecem na barra de menus.

Cada objeto `JMenu` contém vários itens de menu, que estão associados a objetos do tipo `JMenuItem`. O exemplo abaixo ilustra um menu criado dessa forma:

	<pre> JMenuBar bar = new JMenuBar(); JMenu m1 = new JMenu("Arquivos"); JMenu m2 = new JMenu("Ferramentas"); JMenuItem i1 = new JMenuItem("Martelo"); JMenuItem i2 = new JMenuItem("Alicate"); bar.add(m1); bar.add(m2); m2.add(i1); m2.add(i2); </pre>
---	--

Um item de menu cuja funcionalidade é simplesmente estar ou não selecionada é associado a um objeto do tipo `JCheckBoxMenuItem`.

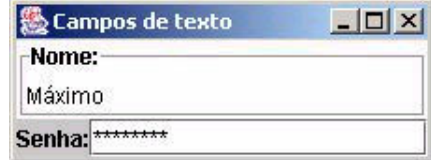
3.16.1.1.2.3 Obter entradas ou escolhas do usuário:

Nesta categoria estão os componentes para entradas de texto e senhas, componentes para seleção dentre um conjunto de opções ou de uma lista de opções, janelas para seleção de arquivos, para seleção de cores ou para diálogos.

3.16.1.1.2.3.1 Entradas de Texto e Senhas (Campos de texto)

Os campos de texto são componentes para entrada de uma linha de texto. Em Swing, objetos associados a campos de texto podem ser da classe `JTextField` (um campo normal) ou `JPasswordField` (um campo para entrada de senha).

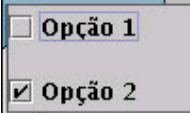
O exemplo abaixo ilustra a criação de dois campos de texto em uma janela, um de cada tipo:

	<pre> JTextField tf = new JTextField(20); JPasswordField t2 = new JPasswordField(10); tf.setBorder(BorderFactory.createTitledBorder("Nome:")); getContentPane().add(tf, BorderLayout.NORTH); getContentPane().add(new JLabel("Senha:"), BorderLayout.WEST); getContentPane().add(t2); </pre>
---	--

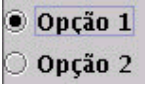
3.16.1.1.2.3.2 Componentes para seleção

Há dois tipos básicos de componentes voltados para a criação de itens que podem estar selecionados ou não: checkboxes e radio buttons.

Um objeto JCheckBox está associado a um item que pode estar selecionado ou não, independentemente de outros.

	<pre>JCheckBox b1 = new JCheckBox("Opção 1"); JCheckBox b2 = new JCheckBox("Opção 2"); b2.setSelected(true); java.awt.Container c = getContentPane(); c.add(b1); c.add(b2); boolean b1Status = b1.isSelected();</pre>
---	---

Um objeto JRadioButton é usado juntamente com um objeto ButtonGroup para criar grupos de botões de seleção onde apenas um botão pode estar selecionado, exclusivamente.

	<pre>JRadioButton b1 = new JRadioButton("Opção 1"); JRadioButton b2 = new JRadioButton("Opção 2"); ButtonGroup bg = new ButtonGroup(); bg.add(b1); bg.add(b2); java.awt.Container c = getContentPane(); c.add(b1); c.add(b2);</pre>
--	---

Observe que o objeto ButtonGroup não é um componente gráfico; os botões ainda devem ser explicitamente incluídos no container correspondente.

Assim como outros componentes Swing, o rótulo de um checkbox ou radio button também pode conter uma imagem.

3.16.1.1.2.3.3 Lista de Opções (ou Listas de seleção)

Listas de seleção permitem apresentar ao usuário um conjunto de opções na forma de uma lista, deixando que o usuário selecione uma ou mais dessas opções.

Objetos da classe JList permitem a criação de listas de seleção a partir de um arranjo de objetos:

Opção 1 Opção 2 Opção 3	<pre>String[] opcoes = { "Opção 1", "Opção 2", "Opção 3" }; JList l1 = new JList(opcoes); java.awt.Container c = getContentPane(); c.add(l1);</pre>
---	---

No exemplo acima, no momento da captura da imagem as opções 1 e 3 estavam selecionadas.

Um JComboBox combina as funcionalidades de uma lista com um campo de texto editável (através da aplicação do método `setEditable(true)`).

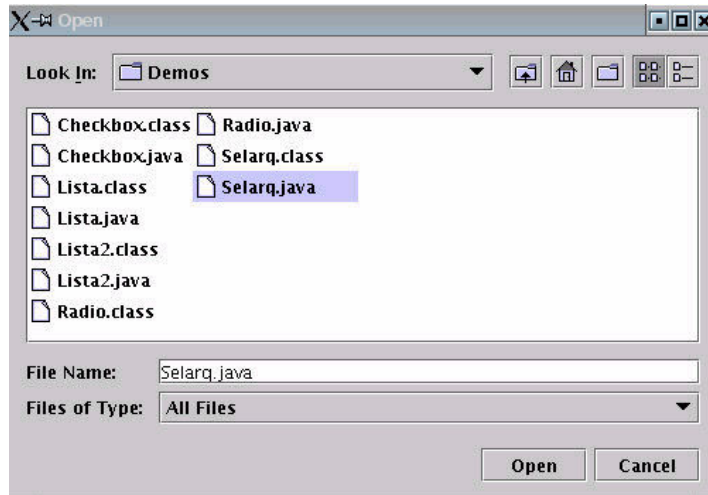
Quando a lista de opções é longa, é preciso adicionar uma barra de rolagem ao componente gráfico. Isto é feito através da criação de um objeto da classe `JScrollPane`, que acrescenta a barra de rolagem como uma decoração associada ao componente gráfico, especificado no construtor:

	<pre>String[] opcoes = { "Opção 1", "Opção 2", "Opção 3", ... } JList l1 = new JList(opcoes); JScrollPane sp = new JScrollPane(l1); java.awt.Container c = getContentPane(); c.add(sp);</pre>
---	---

3.16.1.1.2.3.4 Janelas para Diálogo para seleção de arquivo (ou Seleção de Arquivos)

A classe `JFileChooser` oferece funcionalidades para a criação de uma janela de diálogo com o usuário com o fim específico de selecionar um arquivo no sistema local. A abertura da janela de diálogo dá-se através do método `showOpenDialog`:

<pre>JFileChooser fc = new JFileChooser(); int opcao = fc.showOpenDialog(this); if (opcao == JFileChooser.APPROVE_OPTION) { System.out.println("Arquivo selecionado: " + fc.getSelectedFile().getName()); }</pre>

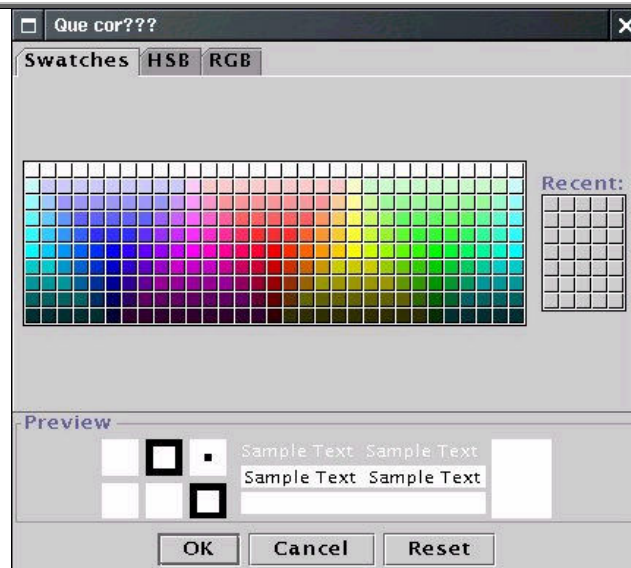


Ao "clique" o botão "Open", a mensagem Arquivo selecionado: Selarq.java é enviada para o console e a janela de diálogo é fechada.

3.16.1.1.2.3.5 Caixa de Diálogo para seleção de cores

Outro tipo de janela de diálogo pré-estabelecida em Swing é JColorChooser, que permite à aplicação selecionar uma cor interativamente.

```
Color c = JColorChooser.showDialog(this, "Que cor???", Color.white);
```





3.16.1.1.2.3.6 Caixa de Janelas de opções e mensagens

Swing oferece uma classe com vários tipos de janelas para obter entradas simples ou para enviar mensagens ao usuário: `JOptionPane`.

As janelas de `JOptionPane` enquadram-se em quatro grandes categorias:

Confirmação

Pede resposta a uma pergunta simples (sim, não, cancela)

Entrada

Pede alguma entrada textual simples

Mensagem

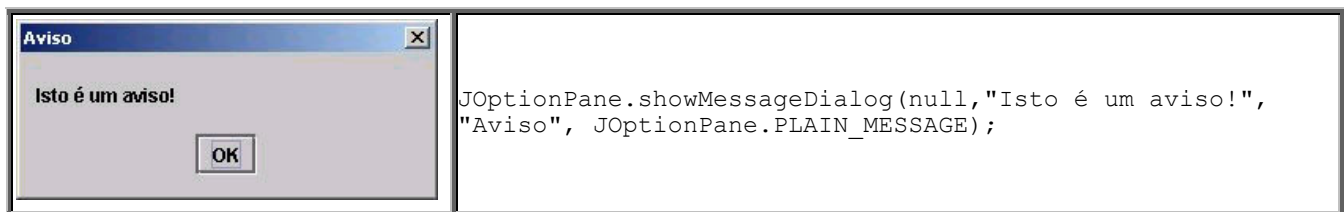
Avisa o usuário sobre alguma coisa que aconteceu

Opção

Uma janela genérica, juntando os aspectos acima.

Para cada tipo de janela, `JOptionPane` oferece métodos de fabricação de objetos desses diferentes tipos.

O exemplo abaixo ilustra a criação de uma janela de mensagem:



Outros métodos estáticos de `JOptionPane` incluem `showConfirmDialog`, `showInputDialog` e `showOptionDialog`.

Janelas de diálogo mais flexíveis e genéricas, com funcionalidade não tão específica, podem ser criadas através de objetos da classe `JDialog`.



3.16.1.1.3 EVENTOS

No modelo de eventos suportado a partir da versão 1.1 de Java, um componente gráfico qualquer pode reconhecer alguma ação do usuário e a partir dela disparar um evento -- indicando, por exemplo, que o botão do mouse foi pressionado ou que um texto foi modificado -- que será capturado por um objeto registrado especificamente para registrar aquele tipo de evento ocorrido naquele componente.

O pacote `java.awt.event` define as diversas classes de eventos que podem ocorrer através das interfaces gráficas. Eventos gráficos são objetos derivados da classe `AWTEvent`, que por sua vez são derivados de objetos de evento genéricos definidos pela classe `EventObject`. Desta, eventos gráficos herdam o método `getSource()`, que permite identificar o objeto que deu origem ao evento.

Eventos gráficos genéricos são especializados de acordo com o tipo de evento sob consideração -- por exemplo, pressionar um botão do mouse gera um `MouseEvent`. A mesma ação sobre um botão, no entanto, gera também um `ActionEvent`, enquanto que sobre o botão de minimizar na barra de título de um frame gera um `WindowEvent` (explicado anteriormente).

Outros eventos de interesse definidos em `java.awt.event` incluem `ItemEvent`, indicando que um item de uma lista de opções foi selecionado, `TextEvent`, quando o conteúdo de um componente de texto foi modificado, e `KeyEvent`, quando alguma tecla foi pressionada no teclado.

A resposta que uma aplicação dá a qualquer evento que ocorre em algum componente gráfico é determinada por métodos específicos, registrados para responder a cada evento. O nome e a assinatura de cada um desses métodos é determinado por uma interface Java do tipo listener. Assim, para responder a um `ActionEvent` será utilizado o método definido na interface `ActionListener`; para responder a um `WindowEvent`, os métodos de `WindowListener`; e similarmente há interfaces `ItemListener`, `TextListener` e `KeyListener`. Os eventos do mouse são manipulados através de métodos especificados em duas interfaces, `MouseListener` (para ações sobre o mouse) e `MouseMotionListener` (para tratar movimentos realizados com o mouse).

Apesar de existir um grande número de eventos e possibilidades de resposta que a aplicação poderia dar a cada um deles, cada aplicação pode especificar apenas aqueles para os quais há interesse que sejam tratados. Para os eventos que o projetista da aplicação tem interesse de oferecer uma reação, ele deve definir classes manipuladoras de eventos (handlers), implementações de cada listener correspondente ao tipo de evento de interesse. Para interfaces listener que especificam diversos métodos, classes adaptadoras são definidas.



Uma vez que uma classe handler tenha sido criada para responder aos eventos de um componente, é preciso associar esse componente ao objeto handler. Para tanto, cada tipo de componente gráfico oferece métodos na forma `addXXXListener(XXXListener l)` e `removeXXXListener(XXXListener l)`, onde XXX está associado a algum evento do tipo `XXXEvent`. Por exemplo, para o componente `Window` (e por herança para todas os componentes derivados desse) são definidos os métodos:

```
public void addWindowListener(WindowListener l) e  
public void removeWindowListener(WindowListener l),
```

pois nesse tipo de componente é possível ocorrer um `WindowEvent`.

3.16.1.1.3.1 Eventos relacionados ao mouse

Eventos de mouse são associados a objetos da classe `MouseEvent`. Através desse evento é possível detectar quando o mouse entra ou sai da região ocupada pelo componente, detectar movimentos do mouse e ações realizadas com os botões do mouse.

De posse de um objeto `MouseEvent`, é possível invocar métodos que permitem detectar a posição na qual o evento ocorreu -- `getPoint()`, `getX()`, `getY()` -- ou indicar o número de mouse clicks associado ao evento, `getClickCount()`. Além disso, como um `MouseEvent` é derivado da classe `InputEvent`, é possível detectar se algum modificador estava associado ao evento através de métodos como `isShiftDown()`, `isAltDown()` e `isControlDown()`.

Eventos de mouse são tratados por dois tipos de listeners de mouse, `MouseListener` para eventos "interessantes" e `MouseMotionListener` para eventos de movimentação do mouse.

Obs.: mais informações:

<http://java.sun.com/j2se/1.3/docs/api/java/awt/event/MouseEvent.html>
<http://java.sun.com/j2se/1.3/docs/api/java/awt/event/MouseListener.html>
<http://java.sun.com/j2se/1.3/docs/api/java/awt/event/MouseAdapter.html>
<http://java.sun.com/j2se/1.3/docs/api/java/awt/event/MouseMotionListener.html>
<http://java.sun.com/j2se/1.3/docs/api/java/awt/event/MouseMotionAdapter.html>



3.16.1.1.3.1.1 Manipuladores de eventos do mouse

Eventos de mouse são tratados pelos métodos definidos em dois listeners distintos definidos em `java.awt.event`.

A interface `MouseListener` especifica os métodos para os eventos de maior destaque na interação do usuário com o mouse. A classe abstrata `MouseAdapter` oferece implementações vazias para todos esses métodos.

Esse exemplo ilustra a implementação de uma classe manipuladora para eventos de mouse que ecoa para a saída padrão a descrição do evento ocorrido.
Código:

```
import java.awt.event.*;

public class MouseEventHandler implements MouseListener {
    private void showEvent(MouseEvent me) {
        System.out.println(me + " na posicao " +
                           me.getPoint());
    }

    public void mouseEntered(MouseEvent me) {
        showEvent(me);
    }

    public void mouseExited(MouseEvent me) {
        showEvent(me);
    }

    public void mousePressed(MouseEvent me) {
        showEvent(me);
    }

    public void mouseReleased(MouseEvent me) {
        showEvent(me);
    }

    public void mouseClicked(MouseEvent me) {
        showEvent(me);
    }
}
```

Esse código ilustra a associação desse manipulador com um componente. Nesse caso, o manipulador de eventos simplesmente envia mensagens para o console.

```
import java.awt.*;

public class EventosMouse extends Frame {
    public EventosMouse() {
        setTitle("Eventos de Mouse");
    }
}
```




```
setSize(240,120);
addMouseListener(new MouseEventHandler());
}

public static void main(String[] args) {
    EventosMouse j = new EventosMouse();
    j.setVisible(true);
}
}
```

Uma possível saída de uma execução seria:

```
java.awt.event.MouseEvent[MOUSE_ENTERED,(147,118),mods=0,clickCount=0] on frame0 na posicao
java.awt.Point[x=147,y=118]
java.awt.event.MouseEvent[MOUSE_PRESSED,(86,53),mods=16,clickCount=1] on frame0 na posicao
java.awt.Point[x=86,y=53]
java.awt.event.MouseEvent[MOUSE_RELEASED,(137,74),mods=16,clickCount=0] on frame0 na posicao
java.awt.Point[x=137,y=74]
java.awt.event.MouseEvent[MOUSE_EXITED,(203,20),mods=0,clickCount=0] on frame0 na posicao
java.awt.Point[x=203,y=20]
java.awt.event.MouseEvent[MOUSE_ENTERED,(228,24),mods=0,clickCount=0] on frame0 na posicao
java.awt.Point[x=228,y=24]
```

Na descrição de eventos de mouse, indica-se a ação, a posição x,y, os modificadores associados ao evento e a quantidade de "cliques" associada ao evento. Nesse caso, houve uma seqüência de entrar em uma janela, pressionar o primeiro botão na posição x=165, y=137 uma vez e sair da janela. Essas informações podem ser obtidas isoladamente pela aplicação através dos métodos da classe MouseEvent.

A interface MouseMotionListener especifica métodos para manipular eventos de movimentação do mouse. A classe abstrata MouseMotionAdapter é uma adaptadora para esse listener.

Como acima, esse exemplo(MouseMotionHandler) ilustra a implementação de uma classe manipuladora para esse tipo de eventos e esse código(EventosMouseMotion) ilustra sua associação a um componente. Como no exemplo anterior, esse manipulador simplesmente envia a descrição do evento ocorrido para o console.

```
import java.awt.event.*;

public class MouseMotionHandler implements MouseMotionListener {
    private void showEvent(MouseEvent me) {
        System.out.println(me + " na posicao " +
            me.getPoint());
    }

    public void mouseMoved(MouseEvent me) {
        showEvent(me);
    }
}
```



```
public void mouseDragged(MouseEvent me) {
    showEvent(me);
}
}
// F i m

import java.awt.*;

public class EventosMouseMotion extends Frame {

    public EventosMouseMotion() {
        setTitle("Eventos MouseMotion");
        setSize(240,120);
        addMouseListener(new MouseMotionHandler());
    }

    public static void main(String[] args) {
        EventosMouseMotion j = new EventosMouseMotion();
        j.setVisible(true);
    }
}
// F i m
```

Um segmento da saída associada a um breve "passeio" do mouse sobre a janela mostra as posições percorridas, inicialmente sem nenhum botão do mouse pressionado e, no segundo segmento, com o primeiro botão pressionado:

```
java.awt.event.MouseEvent[MOUSE_MOVED,(209,199),mods=0,clickCount=0] on frame0
na posicao java.awt.Point[x=209,y=199]
java.awt.event.MouseEvent[MOUSE_MOVED,(219,187),mods=0,clickCount=0] on frame0
na posicao java.awt.Point[x=219,y=187]
java.awt.event.MouseEvent[MOUSE_MOVED,(229,175),mods=0,clickCount=0] on frame0
na posicao java.awt.Point[x=229,y=175]
java.awt.event.MouseEvent[MOUSE_MOVED,(241,165),mods=0,clickCount=0] on frame0
na posicao java.awt.Point[x=241,y=165]
java.awt.event.MouseEvent[MOUSE_MOVED,(255,153),mods=0,clickCount=0] on frame0
na posicao java.awt.Point[x=255,y=153]
[...]
java.awt.event.MouseEvent[MOUSE_DRAGGED,(276,330),mods=16,clickCount=0] on
frame0 na posicao java.awt.Point[x=276,y=330]
java.awt.event.MouseEvent[MOUSE_DRAGGED,(276,329),mods=16,clickCount=0] on
frame0 na posicao java.awt.Point[x=276,y=329]
java.awt.event.MouseEvent[MOUSE_DRAGGED,(276,328),mods=16,clickCount=0] on
frame0 na posicao java.awt.Point[x=276,y=328]
```



3.16.1.1.3.2 Eventos de ação

Objetos da classe `ActionEvent` representam eventos associados a uma ação aplicada a algum componente AWT.

Uma descrição (string) da ação pode ser obtida através do método `getActionCommand()`. Dessa forma, se mais de um componente compartilha um mesmo listener, esse método permite diferenciar qual componente foi o gerador da ação.

Outro atributo que está representado em um objeto evento diz respeito aos modificadores, teclas que podem estar apertadas simultaneamente à ocorrência do evento para requisitar um tratamento alternativo. A classe `ActionEvent` define o método `getModifiers()`, que retorna um valor inteiro que pode ser analisado em relação a quatro constantes da classe, `ALT_MASK`, `CTRL_MASK`, `META_MASK` e `SHIFT_MASK`.

Eventos desse tipo são tratados pelo método definido em `ActionListener`.

Obs.: mais inf. <http://java.sun.com/j2se/1.3/docs/api/java/awt/event/ActionEvent.html>

3.16.1.1.3.3 Eventos de seleção de itens

Um `ItemEvent` sinaliza que um elemento de algum tipo de lista de opções foi selecionado ou desselecionado. Sua ocorrência é tratada através do método de `ItemListener`.

Para implementar a interface `ItemListener` é preciso definir a implementação do método `itemStateChanged`, que recebe como argumento um objeto evento do tipo `ItemEvent`. Componentes que emitem esse tipo de evento implementam a interface `ItemSelectable`.

Um evento de tipo item pode estar associado a qualquer componente que implemente a interface `ItemSelectable`, tais como `Checkbox`, `Choice` e `List`. O método `addItemListener()` da classe que implementa `ItemSelectable` permite associar um manipulador de item selecionado ao componente que gera a seleção.

Obs.: mais informações:

<http://java.sun.com/j2se/1.3/docs/api/java/awt/event/ItemEvent.html>
<http://java.sun.com/j2se/1.3/docs/api/java/awt/event/ItemListener.html>
<http://java.sun.com/j2se/1.3/docs/api/java/awt/ItemSelectable.html>
<http://java.sun.com/j2se/1.3/docs/api/java/awt/Checkbox.html>
<http://java.sun.com/j2se/1.3/docs/api/javaw/swing/JCheckBox.html>
<http://java.sun.com/j2se/1.3/docs/api/javaw/swing/JRadioButton.html>
<http://java.sun.com/j2se/1.3/docs/api/javaw/swing/ButtonGroup.html>

3.16.1.1.3.3.1 Caixas de seleção



Uma caixa de seleção oferece um mecanismo de seleção em que uma opção pode assumir um de dois valores, selecionada ou não-selecionada. Diz-se que tais componentes têm comportamento booleano.

Em AWT, essa funcionalidade é suportada pela classe `Checkbox`. Além do construtor padrão, essa classe oferece construtores que permitem expressar um rótulo (`String`) para o botão e o estado inicial (`boolean`). O método `getState()` permite obter o estado da seleção; o método `getLabel()`, o rótulo associado à caixa de seleção.

`Checkbox` implementa a interface `ItemSelectable`, permitindo tratar reações a eventos do tipo `item` através de um `ItemListener`.

Esse exemplo ilustra a manipulação de três componentes `Checkbox` independentes. Nesse caso, na tela inicial nenhum box está selecionado:



Código:

```
import java.awt.*;
import java.awt.event.*;

public class JanelaCheckbox extends Janela {
    protected Checkbox c1 = new Checkbox("Primeira");
    protected Checkbox c2 = new Checkbox("Segunda");
    protected Checkbox c3 = new Checkbox("Terceira");

    class CheckboxHandler implements ItemListener {
        public void itemStateChanged(ItemEvent ie) {
            Checkbox ch = (Checkbox) ie.getSource();
            System.out.println("[Selecionou " +
                               ch.getLabel() + "]:\t " +
                               c1.getState() + "\t" +
                               c2.getState() + "\t" +
                               c3.getState() );
        }
    }

    public JanelaCheckbox() {
        super("Janela e Checkbox");
        CheckboxHandler cHandler = new CheckboxHandler();
        c1.addItemListener(cHandler);
        c2.addItemListener(cHandler);
    }
}
```



UNIVERSIDADE DO AMAZONAS
FACULDADE DE TECNOLOGIA
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS



```
c3.addItemListener(cHandler);
add(c1, BorderLayout.NORTH);
add(c2, BorderLayout.CENTER);
add(c3, BorderLayout.SOUTH);
}

public static void main(String[] args) {
    JanelaCheckbox jch = new JanelaCheckbox();
    jch.setVisible(true);
}
}
```

Após a seleção do primeiro e do segundo itens, a tela apresenta:



e as seguintes mensagens são enviadas para o console pelo manipulador de eventos:

```
[Selecionou Primeira]: true false false
[Selecionou Segunda]: true true false
```

Se na sequência o primeiro box for novamente "clicado" para ser desselecionado, a mensagem enviada para o console será:

```
[Selecionou Primeira]: false true false
```

Em Swing, essa funcionalidade é suportada de forma similar pela classe JCheckBox.

É também possível definir grupos de caixas de seleção das quais apenas uma pode estar selecionada em um dado instante, constituindo assim opções exclusivas entre si. Em AWT, esse tipo de funcionalidade é oferecido através do conceito de um grupo de checkboxes, suportado através da classe CheckboxGroup. Um CheckboxGroup não é um componente gráfico, como um container, mas sim um agrupamento lógico entre os componentes gráficos do tipo Checkbox, que devem especificar na sua construção ou através do método setCheckboxGroup() a qual grupo pertencem, se for de pertencer a algum.

Obs.: mais inf. <http://java.sun.com/j2se/1.3/docs/api/java/awt/CheckboxGroup.html>

Nesse exemplo é mostrado como três componentes Checkbox são agrupados através de CheckboxGroup. O formato das caixas (em losango) dá uma indicação visual de que as opções são exclusivas:





Código:

```
import java.awt.*;

public class JanelaCheckboxGroup extends JanelaCheckbox {
    CheckboxGroup cg = new CheckboxGroup();

    public JanelaCheckboxGroup() {
        c1.setCheckboxGroup(cg);
        c2.setCheckboxGroup(cg);
        c3.setCheckboxGroup(cg);
    }

    public static void main(String[] args) {
        JanelaCheckboxGroup jch = new JanelaCheckboxGroup();
        jch.setVisible(true);
    }
}
```

Se na seqüência as opções Segunda, Terceira e novamente a Primeira forem selecionadas, o manipulador de eventos envia as seguintes mensagens para o console:

```
[Selecionou Segunda]: false true false
[Selecionou Terceira]: false false true
[Selecionou Primeira]: true false false
```

Funcionalidade similar a essa é oferecida em Swing através das classes JRadioButton e ButtonGroup.

Obs.: mais informações:

<http://java.sun.com/j2se/1.3/docs/api/javaw/swing/JRadioButton.html>
<http://java.sun.com/j2se/1.3/docs/api/javaw/swing/ButtonGroup.html>

3.16.1.1.3.3.2 Seleção em drop-down

Um componente de interação em drop-down é aquele que deixa visível uma única opção (aquela que está selecionada) ao usuário, mas que permite que esse visualize e selecione as outras opções através da ativação de um mini-botão associado ao componente. Quando esse tipo de componente permite também a entrada de valores em um campo de texto combinado à lista, é usualmente chamado de Combo box.

Em AWT, a funcionalidade de lista em drop-down é suportada através da classe Choice, que permite definir uma lista de itens não-editáveis. Para essa classe, apenas o



construtor padrão é oferecido. Os principais métodos para manipular objetos dessa classe são:

```
void add(String s),  
    permite adicionar itens à lista;  
int getSelectedIndex(),  
    retorna a posição numérica do item selecionado na lista (base 0);  
String getSelectedItem(),  
    retorna o rótulo do item selecionado na lista;  
String getItem(int index),  
    retorna o rótulo do item cujo índice foi especificado.
```

A classe Choice implementa a interface `java.awt.ItemSelectable`, o que permite associar objetos dessa classe a um manipulador de eventos do tipo item, `ItemListener`.

Esse exemplo mostra como incluir um componente Choice em um frame e lidar com a seleção de seus itens. Quando criado, a aparência desse componente é:

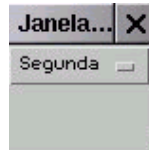


Código:

```
import java.awt.*;  
import java.awt.event.*;  
  
public class JanelaChoice extends Janela {  
    Choice ch;  
  
    class ChoiceHandler implements ItemListener {  
        public void itemStateChanged(ItemEvent ie) {  
            System.out.println(ch.getSelectedIndex() +  
                               ": " +  
                               ch.getSelectedItem());  
        }  
    }  
  
    public JanelaChoice() {  
        super("Janela e Choice");  
        ch = new Choice();  
        ch.add("Primeira");  
        ch.add("Segunda");  
        ch.add("Terceira");  
        ch.addItemListener(new ChoiceHandler());  
        add(ch);  
    }  
}
```

```
public static void main(String[] args) {  
    JanelaChoice jc = new JanelaChoice();  
    jc.setVisible(true);  
}  
}
```

Se o usuário selecionar a segunda opção desse componente, a aparência muda para:



Adicionalmente, pelo comportamento especificado no método de tratamento do evento a seguinte mensagem é enviada para a saída padrão:

1: Segunda

indicando que a segunda opção da lista foi selecionada (a primeira tem índice 0, a segunda, 1 e assim sucessivamente) e que a string especificada nessa opção é "Segunda".

Em Swing, o mecanismo de lista em drop-down é suportado pela classe JComboBox, que pode ser configurado para ser editável ou não.

Obs.: mais informações:

<http://java.sun.com/j2se/1.3/docs/api/java/awt/Choice.html>

<http://java.sun.com/j2se/1.3/docs/api/java/awt/ItemSelectable.html>

<http://java.sun.com/j2se/1.3/docs/api/javawx/swing/JComboBox.html>

3.16.1.1.3.3.3 Listas

Um componente do tipo lista oferece a funcionalidade de apresentar uma lista de opções dos quais um ou vários itens podem ser selecionados. Além disso, é possível exibir várias opções simultaneamente.

Em AWT, a classe List oferece essa funcionalidade. Além do construtor padrão, essa classe oferece um construtor que permite especificar quantas linhas da lista serão visíveis. Um terceiro construtor permite especificar um valor booleano indicando se o modo de seleção de vários itens está habilitado (true) ou não. Se não especificado, apenas um item pode ser selecionado da lista. Os principais métodos dessa classe incluem:



```
void add(String item),  
    acrescenta rótulo à lista;  
void add(String item, int index),  
    acrescenta rótulo à lista na posição especificada;  
int getSelectedIndex(),  
    retorna a posição numérica do item selecionado na lista (o primeiro item está na  
    posição 0);  
String getSelectedItem(),  
    retorna o rótulo do item selecionado na lista;  
String getItem(int index),  
    retorna o rótulo do item cujo índice foi especificado.
```

Além desses métodos, para a manipulação de seleções múltiplas há:

```
int[] getSelectedIndexes(), e  
String[] getSelectedItems().
```

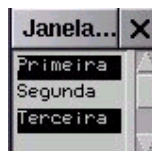
Assim como as classes Choice e Checkbox, a classe List implementa a interface ItemSelectable, que permite associar um objeto dessa classe a um manipulador de eventos do tipo item, ou seja, a um objeto ItemListener. Esse tipo de evento é gerado quando o item é alvo de um único toque do mouse.

Além disso, um item da lista pode reagir a um toque duplo do mouse através da geração de um evento do tipo ação, usando para o tratamento do evento nesse caso um manipulador do tipo ActionListener.

Esse exemplo ilustra como realizar a manipulação de um component List. Nesse caso, a múltipla seleção é habilitada e tanto eventos de item como de ação são tratados. A tela inicial apresenta:



Após a seleção (clique simples) sobre a primeira e terceira opções, o estado do componente fica:





e as seguintes mensagens são enviadas para o console pelo manipulador de eventos de itens:

Itens selecionados: Primeira
Itens selecionados: Primeira Terceira

Se na seqüência houver um clique duplo sobre a segunda opção e então um clique simples sobre a primeira e a terceira opções (para remover essas seleções), as seguintes mensagens serão enviadas ao console pelos manipuladores de eventos:

Itens selecionados: Primeira Segunda Terceira
Clique duplo em: Segunda
Itens selecionados: Terceira
Itens selecionados: nenhum.

Observe que o tratamento de múltiplas seleções através de eventos de itens não é amigável, pois a cada seleção um novo evento é gerado. Por esse motivo, a documentação da API Java recomenda que esse tipo de seleção múltipla em AWT seja efetivada através de algum controle externo, como um botão.

Em Swing, funcionalidade similar à apresentada por List é oferecida pela classe JList, embora algumas diferenças devam ser observadas. Por exemplo, ao contrário de List, um JList não suporta barras de rolagem diretamente, o que deve ser acrescentado através de um objeto JScrollPane. Esse exemplo ilustra a construção de um componente JList.

Código:

```
import javax.swing.*;

public class JanelaBox extends JFrame {

    public JanelaBox() {
        setTitle("BoxLayout");
        setSize(240,120);

        Box h = Box.createHorizontalBox();
        Box v = Box.createVerticalBox();
        String[] lista = {"Um", "Dois", "Tres", "Quatro",
                        "Cinco", "Seis", "Sete"};
        JList jl = new JList(lista);
        jl.setFixedCellWidth(70);
        jl.setSelectionMode(ListSelectionModel.MULTIPLE_INTERVAL_SELECTION);
        JScrollPane lEsq = new JScrollPane(jl);
        lEsq.setMinimumSize(new java.awt.Dimension(100,100));
        JButton add = new JButton(">>");
```



```
JButton clear = new JButton("Clear");
JButton close = new JButton("Close");
JTextArea tDir = new JTextArea();

v.add(add);
v.add(clear);
v.add(close);
h.add(lEsq);
h.add(v);
h.add(tDir);
getContentPane().add(h);
}

public static void main(String[] args) {
    JanelaBox jb = new JanelaBox();
    jb.setVisible(true);
    jb.validate();
}
}
```

Obs.: mais informações

<http://java.sun.com/j2se/1.3/docs/api/javaw/swing/JList.html>

<http://java.sun.com/j2se/1.3/docs/api/javaw/swing/JScrollPane.html>

3.16.1.1.3.4 Eventos de modificação de texto

Um objeto da classe `TextEvent` está relacionado a um evento indicando que o usuário modificou o valor de texto que aparece em um `TextComponent` -- qualquer modificação no texto apresentado no componente gera esse evento.

Esse tipo de evento é tratado através de uma implementação de `TextListener`.

3.16.1.1.3.4.1 Componentes de texto

Componentes de texto permitem a interação com o usuário através da apresentação e entrada direta de strings.

Em AWT, a classe `TextComponent` é a superclasse para os componentes visuais de apresentação e entrada de texto, `TextField` e `TextArea`, definindo funcionalidades comuns a ambas as classes. Como não tem construtor público, não é possível instanciar diretamente objetos dessa classe. Algumas das funcionalidades básicas definidas nessa classe, portanto acessíveis tanto de `TextField` como de `TextArea`, incluem os métodos

`String getText()`,

para obter o texto corrente no componente,
`void setText(String t)`,
 para definir o texto a ser apresentado no componente,
`void setEditable(boolean b)`,
 para permitir ($b=true$) ou impedir ($b=false$) que o conteúdo do componente seja modificável, e

`boolean isEditable()`,
 para saber se o componente tem conteúdo modificável ou não.

Caso o conteúdo apresentado em um `TextComponent` seja editável e alguma modificação ocorra, um evento do tipo texto é gerado e, em havendo um objeto `TextListener` registrado para esse evento, este é notificado. Para registrar e remover manipuladores para eventos do tipo texto, a classe `TextComponent` provê os métodos `addTextListener()` e `removeTextListener()`, respectivamente.

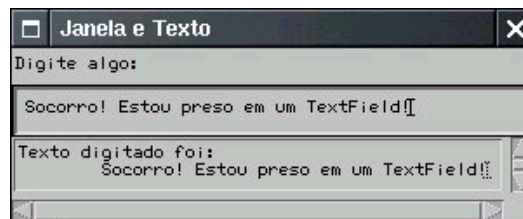
Similarmente, Swing oferece uma raiz comum a campos e área de texto, que é a classe `JTextComponent`. Na verdade, Swing oferece -- além das funcionalidades básicas de campos e áreas de texto -- todo um sub-pacote dedicado à manipulação de textos, `javax.swing.text`.

4

3.16.1.1.3.4.1.1 Campos de texto

Um campo de texto é uma área retangular, de tamanho correspondente a uma linha, que pode ser utilizada para apresentar ou adquirir strings do usuário.

A classe `TextField` é uma extensão da classe `TextComponent` que implementa um campo de texto no toolkit AWT. Os construtores dessa classe permitem a criação de um campo de texto que pode estar vazio ou conter algum texto inicial. O número de colunas no campo também pode ser especificado através dos construtores. Esse exemplo ilustra a utilização de um `TextField` para obter uma string do usuário que, no tratamento do evento do tipo ação, é ecoada para uma área de texto na mesma janela:



Código:

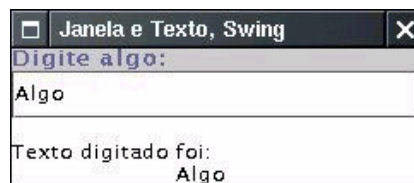
```
import java.awt.*;
import java.awt.event.*;

public class JanelaText extends Janela {
    TextField tf = new TextField(20);
```



```
TextArea ta = new TextArea(2,20);
class TextHandler implements ActionListener {
    public void actionPerformed(ActionEvent ae) {
        ta.append("\nTexto digitado foi:\n\t" + tf.getText());
    }
}
public JanelaText() {
    super("Janela e Texto");
    add(new Label("Digite algo:"), BorderLayout.NORTH);
    add(tf, BorderLayout.CENTER);
    add(ta, BorderLayout.SOUTH);
    tf.addActionListener(new TextHandler());
}
public static void main(String[] args) {
    JanelaText jt = new JanelaText();
    jt.setVisible(true);
}
}
```

O componente correspondente a esse em Swing é JTextField. Além de funcionalidades associadas a TextField, esse componente oferece outras que permitem definir campos de texto customizados. Esse exemplo mostra como a mesma janela exibida para AWT pode ser construída com Swing:



Código:

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class SText extends SJanela {
    JTextField tf = new JTextField(20);
    JTextArea ta = new JTextArea(2,20);

    class TextHandler implements ActionListener {
        public void actionPerformed(ActionEvent ae) {
            ta.append("\nTexto digitado foi:\n\t" + tf.getText());
        }
    }

    public SText() {
        super("Janela e Texto, Swing");
        Container c = getContentPane();
        c.add(new JLabel("Digite algo:"), BorderLayout.NORTH);
        c.add(tf, BorderLayout.CENTER);
        c.add(ta, BorderLayout.SOUTH);
    }
}
```



```
tf.addActionListener(new TextHandler());  
}  
  
public static void main(String[] args) {  
    SText jt = new SText();  
    jt.setVisible(true);  
}  
}
```

Quando o usuário entra um texto em um campo de texto e tecla enter, é gerado um evento do tipo ação. Para indicar o objeto manipulador para esse evento, os métodos `addActionListener()` e `removeActionListener()` são utilizados. Ambos têm como argumento um objeto `ActionListener`. Eventos relativos à modificação de um valor em um campo de texto (sem pressionar a tecla enter) estão associados a eventos do tipo texto. Desse modo, é possível monitorar se alguma modificação ocorreu no campo de texto.

É possível não ecoar a apresentação dos caracteres de entrada para a tela definindo um caráter alternativo que deve ser apresentado. Em AWT, utiliza-se o método `setEchoChar()` da classe `TextField`. Em Swing, para tal fim deve ser utilizado o componente `JPasswordField`.

3.16.1.1.3.4.1.2 Áreas de texto

Uma área de texto permite apresentar e opcionalmente editar textos de múltiplas linhas.

Em AWT, a classe `TextArea` oferece essa funcionalidade, sendo uma extensão da classe `TextComponent`. Assim como para um campo de texto de uma linha, o texto apresentado inicialmente e o número de colunas visíveis pode ser definido em construtores. Adicionalmente, o número de linhas visíveis pode ser definido no momento da construção do objeto. Para a apresentação de textos que ocupam uma área maior que a visível, um construtor tem um argumento que indica se e quantas barras de rolagem estarão presentes na área de texto. Os valores possíveis, definidos como constantes estáticas dessa classe, são `SCROLLBARS_BOTH` (barras de rolagem nas direções vertical e horizontal), `SCROLLBARS_HORIZONTAL_ONLY` (apenas na horizontal), `SCROLLBARS_VERTICAL_ONLY` (apenas na vertical) e `SCROLLBARS_NONE` (sem barras de rolagem).

O componente correspondente a esse em Swing é o `JTextArea`.

É possível adicionar texto a uma área através do método `append()`, que recebe uma string como argumento. Esse exemplo em AWT mostra como usar esse método para registrar strings digitadas em um componente `TextField` na mesma janela, enquanto esse outro mostra a funcionalidade equivalente em Swing:

3.16.1.1.3.4.1. Manipuladores de eventos de modificação de texto

A interface `TextListener` especifica o método que trata eventos do tipo texto.



Um método apenas é definido, `textValueChanged(TextEvent e)`. O corpo desse método determina o que deve ser executado quando o texto em algum componente de texto é alterado.

Obs.: Para mais informações:

<http://java.sun.com/j2se/1.3/docs/api/java/awt/event/TextEvent.html>
<http://java.sun.com/j2se/1.3/docs/api/java/awt/TextComponent.html>
<http://java.sun.com/j2se/1.3/docs/api/javawx/swing/text/JTextComponent.html>
<http://java.sun.com/j2se/1.3/docs/api/javawx/swing/text/package-summary.html>
<http://java.sun.com/j2se/1.3/docs/api/java/awt/TextField.html>
<http://java.sun.com/j2se/1.3/docs/api/javawx/swing/JPasswordField.html>
<http://java.sun.com/j2se/1.3/docs/api/javawx/swing/JTextField.html>
<http://java.sun.com/j2se/1.3/docs/api/javawx/swing/JEditorPane.html>
<http://java.sun.com/j2se/1.3/docs/api/javawx/swing/JTextPane.html>
<http://java.sun.com/j2se/1.3/docs/api/java/awt/event/TextListener.html>

3.16.1.1.3.5 Eventos de teclado

Um evento do tipo `KeyEvent` está associado à pressão de qualquer tecla do teclado. É um evento de baixo nível, utilizado por eventos como `TextEvent` e modificadores de `MouseEvent`.

Esse tipo de evento distingue as ações de pressionar uma tecla e de soltá-la. Se a tecla pressionada corresponder a um carácter imprimível, a combinação das duas ações é indicada no evento como "key typed".

Eventos desse tipo são tratados através de um objeto `KeyListener`.

3.16.1.1.3.5.1 Manipuladores de eventos de teclado

A interface `KeyListener` especifica os métodos necessários para detectar e tratar eventos de teclado. São eles:

`keyPressed()`, para indicar que a tecla foi pressionada;
`keyReleased()`, para indicar que a tecla foi solta; e
`keyTyped()`, para indicar que uma tecla que não de controle foi pressionada e solta.

Todos os métodos recebem um objeto `KeyEvent` como parâmetro.

Códigos:



```
import java.awt.event.*;

public class KeyEventHandler implements KeyListener {
    private void showEvent(KeyEvent ke) {
        System.out.println(ke);
    }

    public void keyPressed(KeyEvent ke) {
        showEvent(ke);
    }

    public void keyReleased(KeyEvent ke) {
        showEvent(ke);
    }

    public void keyTyped(KeyEvent ke) {
        showEvent(ke);
    }
}
// F i m
```

```
import java.awt.*;

public class EventosKey extends Frame {

    public EventosKey() {
        setTitle("Eventos de Teclado");
        setSize(240,120);
        addKeyListener(new KeyEventHandler());
    }

    public static void main(String[] args) {
        EventosKey j = new EventosKey();
        j.setVisible(true);
    }
}
// F i m
```

Esse exemplo ilustra a criação de um frame que detecta eventos de teclado. Nesse caso, o manipulador de eventos simplesmente ecoa para o console a descrição do evento. O segmento da saída correspondente a pressionar as teclas '1' e depois 'Shift' sobre a janela indica a ocorrência de cinco eventos de teclado:

```
java.awt.event.KeyEvent[KEY_PRESSED,keyCode=49,keyChar='1'] on frame0
java.awt.event.KeyEvent[KEY_TYPED,keyCode=0,keyChar='1'] on frame0
java.awt.event.KeyEvent[KEY_RELEASED,keyCode=49,keyChar='1'] on frame0
java.awt.event.KeyEvent[KEY_PRESSED,keyCode=16,keyChar='?'] on frame0
```




```
java.awt.event.KeyEvent[KEY_RELEASED,keyCode=16,keyChar='?',modifiers=Shift] on frame0
```

Como essa interface listener especifica mais de um método, uma classe adaptadora, KeyAdapter, é oferecida.

Obs.: mais informações:

<http://java.sun.com/j2se/1.3/docs/api/java/awt/event/KeyEvent.html>

<http://java.sun.com/j2se/1.3/docs/api/java/awt/event/KeyListener.html>

<http://java.sun.com/j2se/1.3/docs/api/java/awt/event/KeyAdapter.html>

3.16.1.1.3.6 Manipuladores de eventos de ação

A interface ActionListener especifica o método actionPerformed() para tratar eventos do tipo ação. Um objeto de uma classe que implemente essa interface deve ser associado a um componente através do método addActionListener(), que é definido para componentes do tipo botão, lista e campo de texto.

Quando um evento desse tipo ocorre, o objeto registrado é notificado. O método actionPerformed() é então invocado, recebendo como argumento o objeto que representa o evento.

3.16.1.1.3.6.1 Eventos de ação

Objetos da classe ActionEvent representam eventos associados a uma ação aplicada a algum componente AWT.

Uma descrição (string) da ação pode ser obtida através do método getActionCommand(). Dessa forma, se mais de um componente compartilha um mesmo listener, esse método permite diferenciar qual componente foi o gerador da ação.

Outro atributo que está representado em um objeto evento diz respeito aos modificadores, teclas que podem estar apertadas simultaneamente à ocorrência do evento para requisitar um tratamento alternativo. A classe ActionEvent define o método getModifiers(), que retorna um valor inteiro que pode ser analisado em relação a quatro constantes da classe, ALT_MASK, CTRL_MASK, META_MASK e SHIFT_MASK.

Eventos desse tipo são tratados pelo método definido em ActionListener.

Obs.: mais informações:

<http://java.sun.com/j2se/1.3/docs/api/java/awt/event/ActionListener.html>

<http://java.sun.com/j2se/1.3/docs/api/java/awt/event/ActionEvent.html>



3.16.2 Interfaces gráficas com usuários

A construção de uma interface gráfica com usuários em Java começa com a criação de um container , um componente que pode receber outros. Em uma aplicação gráfica autônoma , tipicamente o container usado como base é um frame , enquanto que em um applet o container base é um panel. Em aplicações complexas, qualquer tipo de combinação desses e de outros tipos de componentes é utilizado.

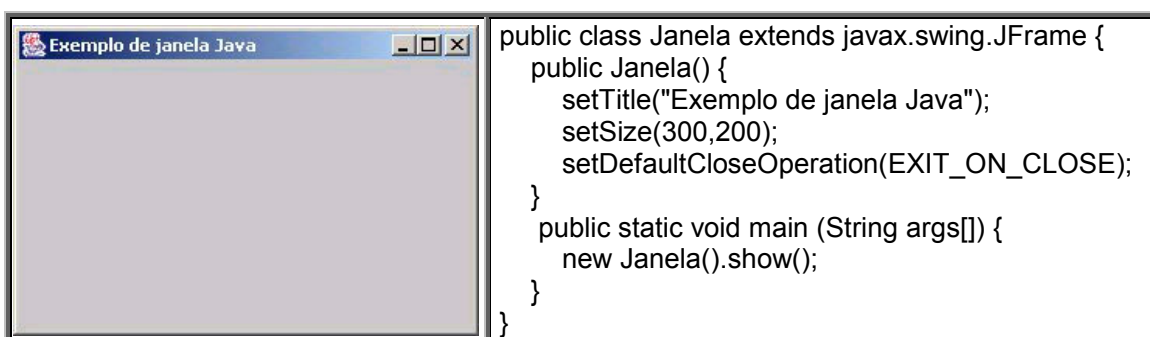
Em uma segunda etapa, os containers da aplicação recebem componentes de interface com usuários , que permitirão apresentar e receber dados aos e dos usuários. Embora seja possível posicionar esses componentes de forma absoluta em cada container, é recomendável que o projetista utilize sempre um gerenciador de layout para realizar essa tarefa, pois deste modo garante-se que a aplicação possa ser executada independentemente das características da plataforma onde será executada.

Finalmente, é preciso especificar quais devem ser os efeitos das ações dos usuários -- tais como um "clique" do mouse ou uma entrada de texto -- quando realizadas sobre cada um desses componentes. Isto se dá através da especificação de classes manipuladoras de eventos , projetadas para a aplicação. Ao associar objetos dessas classes aos componentes gráficos, o projetista determina o comportamento da aplicação.

3.16.2.1 Aplicações autônomas

Aplicações gráficas autônomas são aquelas cujas janelas são criadas a partir da execução do método main de alguma classe, não estando incorporadas a outras aplicações.

Em Java, a janela que é inicialmente criada para uma aplicação gráfica autônoma é um objeto do tipo `javax.swing.JFrame`. O exemplo abaixo ilustra a criação e exibição de um objeto desse tipo.



Uma vez que uma janela como essa tenha sido criada, ela pode ter sua área interna utilizada para saídas gráficas ou para a inclusão de componentes de interação com usuários, neste caso usando a disposição de componentes segundo estabelecida por um gerenciador de layout.



4 APPLETS EM JAVA

Applets são programas projetados para ter uma execução independente dentro de alguma outra aplicação, eventualmente interagindo com esta -- tipicamente, um browser (navegador) Web. Assim, applets executam no contexto de um outro programa, o qual interage com o applet e determina assim sua seqüência de execução. Funcionalidades associadas a applets Java são agregadas no pacote java.applet.

O processo de criação de um applet é similar ao processo de criação de uma aplicação -- o programa deve ser criado por um editor de programas e o arquivo de bytecodes (.class) deve ser gerado a partir da compilação do arquivo fonte.

Uma vez criado ou disponibilizado o bytecode, é preciso incluir uma referência a ele em alguma página Web. Essa página Web, uma vez carregada em algum navegador, irá reconhecer o elemento que faz a referência ao applet, transferir seu bytecode para a máquina local e dar início à sua execução.

Toda Applet Deve, obrigatoriamente, herdar da classe java.applet.Applet

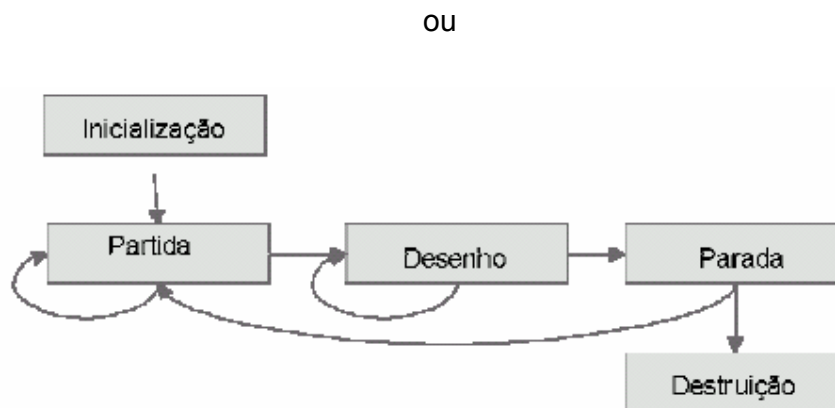
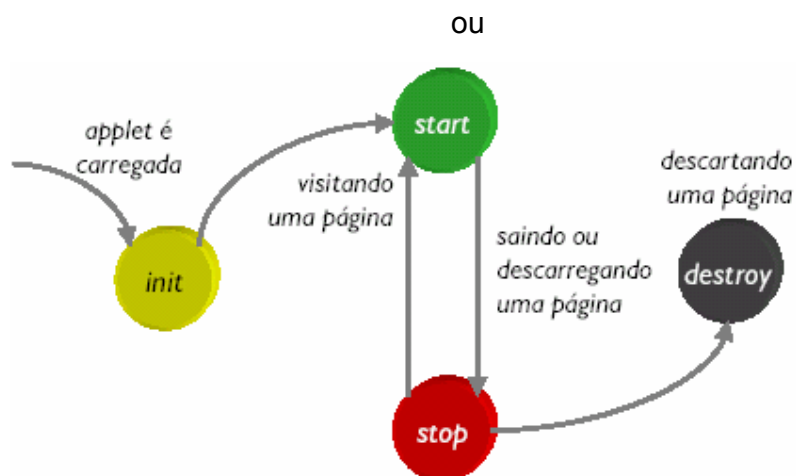
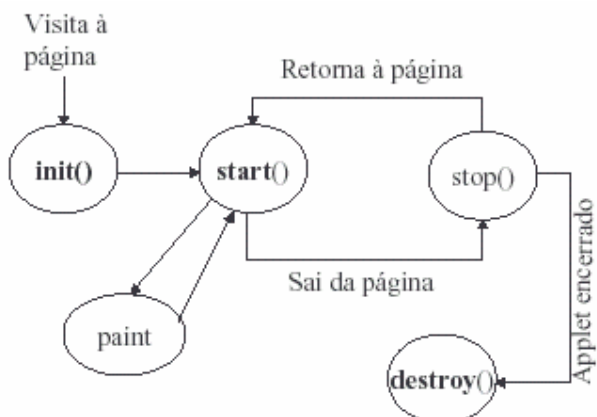
4.1 A ESTRUTURA BÁSICA

Normalmente um applet contém um bloco de instruções import e um bloco de declaração da classe do applet. Este último é delimitado pela chave inicial { e pela chave final }. A declaração de variáveis e os métodos que compõem a classe do applet ficam entre as chaves { } mencionadas.

Os quatro métodos mencionados pertencem à classe Applet da linguagem Java e podem ser substituídos para gerenciar o ciclo de existência do applet. É essencial entender tais métodos para produzir applets estáveis e "comportados".

```
(Declaração do pacote package)
Bloco de Instruções import
Declaração da classe {
    declaração de variáveis
    método init()
    método start()
    método stop()
    método destroy()
}
```

Funcionamento





4.2 DECLARAÇÃO DA CLASSE APLET

A declaração da classe de um novo applet é apenas a extensão da definição da classe Applet do pacote java.applet (veja acima), ou seja, declaramos uma nova classe do "tipo Applet" que herda todas as características (variáveis e métodos) da classe original. Em outras palavras, criamos uma subclasse (nosso applet) que é a cópia fiel de uma superclasse (a classe original). A palavra-chave para "clonar" uma classe é extends.

Exemplo:

```
public class meuApplet extends Applet {  
    ... variáveis e métodos  
}
```

Se o método run() for substituído, a declaração da classe do novo applet precisa implementar esta característica e a declaração seria:

```
public class meuApplet extends Applet implements Runnable {  
    ... variáveis e métodos  
}
```

4.3 DECLARAÇÃO DE VARIÁVEIS

O formato para a declaração de variáveis é <tipo><nome da variável>(<atribuição>), por exemplo:

```
int i; ou  
int i = 10;
```

Além disso, pode-se utilizar um dos modificadores de acesso: public, private, protected ou package.

O modificador public torna uma variável acessível por qualquer classe ou método.

O modificador private indica que a variável não está disponível para qualquer outra classe, com exceção daquela na qual ela aparece. O modificador protected restringe o acesso apenas às subclasses. O modificador package ou a ausência de um modificador restringe o acesso apenas às classes pertencentes ao pacote (como no exemplo acima). Para tornar a variável privada, a declaração poderia ser a seguinte:

```
private int i = 10;
```

MÉTODO init()

O método init() só é chamado quando o applet for carregado, ou seja, apenas uma vez durante seu ciclo de existência. Caso seja necessário inicializar variáveis ou configurar recursos que serão usados em todo o applet, então é aconselhável substituir este método.

Substituir um método pode significar duas coisas:



1. Trocar o método que a classe recebeu como herança da superclasse por um método próprio totalmente novo. Exemplo:

```
public void init() {  
    int i = 10;  
    ...  
}
```

2. Adicionar comportamentos ao método herdado. Para manter o método herdado utiliza-se a palavra-chave super como no exemplo:

```
public void init() {  
    super.init();  
    int i = 10;  
    ...  
}
```

MÉTODO start()

O método start() é chamado sempre que o documento HTML no qual reside o applet passa a ser a página ativa de um navegador. Quando um applet é executado pela primeira vez, o método start() é chamado logo depois do init(). Este método também pode ou não ser substituído. É excelente para iniciar threads e o melhor local para chamar o método show() de instâncias da classe Frame.

MÉTODO stop()

Este método é o "inverso" de start(), sendo chamado sempre que o usuário deixar a página na qual o applet reside. É ideal para ocultar quadros e encerrar threads.

MÉTODO destroy()

Raramente este método é substituído porque o momento que ele é chamado é um tanto imprevisível: é o momento em que o applet é desativado - geralmente quando o navegador é fechado. É um bom lugar para a realização de algumas tarefas de limpeza.

Conceito Básico:



Applet e documento HTML

AS TAGS <applet>...</applet>

As tags especiais <applet> e </applet> servem para delimitar uma área do documento HTML onde será inserido um applet. A sintaxe mínima obrigatória é a seguinte:

```
<applet code="meuApplet" width=100 height=100>  
</applet>
```

O atributo code especifica o nome da classe que executa o applet. Os parâmetros width (largura) e height (altura) correspondem ao tamanho do applet em pixels.

ATRIBUTOS ADICIONAIS

O atributo codebase indica o caminho no qual a classe especificada por code pode ser encontrada.

O atributo align é usado para controlar o alinhamento do applet na página. Pode ser: left, right, top, bottom, middle, baseline, etc.

```
<applet code="meuApplet" codebase="../teste" width=100 height=100 align="left">  
</applet>
```

PARÂMETROS

Entre as tags <applet> e </applet> podemos utilizar tags especiais para enviar parâmetros ao nosso applet. São as tags <param> com o par de atributos name e value. Name é o nome da variável de parâmetro e value seu valor correspondente. Por exemplo:

```
<applet code="meuApplet" codebase="../teste" width=100 height=100 align="left">  
<param name="imagem" value="gato.jpg">  
<param name="pausa" value="1000">  
</applet>
```

4.4 CRIAÇÃO DE APPLET

Para criar um applet, é preciso desenvolver uma classe que estenda a classe Applet:



```
import java.applet.*;  
    // ou javax.swing.JApplet  
public class MeuApplet extends Applet {  
    ...  
}
```

A classe Applet é uma extensão de um container do tipo Panel. Assim, o desenvolvimento de um applet segue as estratégias de desenvolvimento de qualquer aplicação gráfica.

4.5 EXECUÇÃO DE APPLETS

Ao contrário das aplicações Java, a execução de um applet em uma página não é iniciada pelo método `main()`. Um applet é executado como uma thread subordinada ao navegador (ou à aplicação `appletviewer`, presente no ambiente de desenvolvimento Java). O navegador será responsável por invocar os métodos da classe Applet que contro- lam a execução de um applet.

Quando o applet é carregado pela primeira vez pelo navegador, o método `init()` é invocado pelo navegador. Esse método pode ser considerado funcionalmente equivalente ao método construtor em aplicações, pois será apenas executado no momento em que o código for carregado.

Após o carregamento do applet para a máquina local e a execução do método `init()`, o método `start()` é invocado. Esse método será invocado também a cada vez que a área do applet torna-se visível no navegador, dando reinício a operações que eventual- mente tenham sido paralisadas pelo método `stop()`.

O método `stop()` é invocado cada vez que o applet torna-se temporariamente invi- sível, ou seja, que sua área não esteja visível no navegador. É uma forma de evitar que operações que demandem muita CPU continuem a executar desnecessariamente. Tam- bém é invocado imediatamente antes de `destroy()`, que é invocado quando o applet está para ser eliminado do navegador. Este método serve para liberar recursos -- além de memória -- que o applet tenha eventualmente alocado para sua execução.

Esse exemplo ilustra a definição de um applet que implementa apenas esses qua- tro métodos. Quando carregada em um navegador Web, a página que referencia esse código apresenta apenas uma área vazia; porém, se o console Java do navegador for aberto, será possível visualizar as mensagens:



Sendo um componente gráfico do tipo container, todas as funcionalidades descritas para o desenvolvimento de aplicações gráficas aplicam-se a applets. Por exemplo, é possível que um applet apresente formas geométricas em seu contexto gráfico usando o método `paint()` ou incluir componentes de interface com usuário e tratar a manipulação de eventos.

Alguns métodos da classe `Applet` permitem a comunicação do applet com o navegador no qual ele está inserido. O método `getParameter()` permite a passagem de parâmetros do navegador para o applet. O método `showStatus()` exibe uma mensagem na linha de status do navegador que executa o applet. O método `getAppletContext()` permite obter o contexto de execução do applet, o que permite por exemplo estabelecer a comunicação entre dois applets de uma mesma página.

Como código de um applet é obtido através de uma conexão remota, sua execução está restrita aos princípios de segurança impostos por Java para a execução de código em ambientes distribuídos. Tipicamente, um applet não pode acessar arquivos e informações do ambiente onde está executando nem pode estabelecer conexões remotas com outras máquinas (a não ser aquela de onde o próprio código foi obtido).

4.5.1 Passagem de argumentos

O método `getParameter()` da classe `Applet` permite obter parâmetros passados para o applet através do elemento `PARAM` em uma página HTML. O argumento para esse método é o nome do parâmetro, estabelecido no atributo `name` da tag `PARAM`, e o retorno é uma string com o valor do argumento (conteúdo do atributo `value` de `PARAM`). Esse applet recebe como parâmetro a cor de fundo, que pode ser branca, amarela ou cinza (o padrão):

Com

```
<p>
<APPLET code="AppletParameter.class" width="640" height="20">
  <PARAM name="background" value="White">
</APPLET>
```



Com

```
<p>  
<APPLET code="AppletParameter.class" width="640" height="20">  
  <PARAM name="background" value="yellow">  
</APPLET>
```



Ou sem argumentos:

```
<p>  
<APPLET code="AppletParameter.class" width="640" height="20">  
</APPLET>
```



Código:

```
import java.applet.*;  
import java.awt.*;
```

```
public class AppletParameter extends Applet {  
    private int altura;  
    private int largura;  
    private int incremento;  
  
    public void init() {  
        Dimension d = getSize();  
        altura = d.height - 2;  
        largura = d.width;  
        incremento = altura/3;  
  
        Color c = Color.gray;  
        String cor = getParameter("background");  
        try {  
            if (cor.equalsIgnoreCase("yellow"))  
                c = Color.yellow;  
            else if (cor.equalsIgnoreCase("white"))  
                c = Color.white;  
        }  
        catch (Exception e) {  
        }  
  
        setBackground(c);  
    }  
  
    public void paint(Graphics g) {
```



```
int x=1;
int y=1;
int finalPos = largura - altura;

while (x < finalPos) {
    g.setColor(new Color((float)Math.random(),
                        (float)Math.random(),
                        (float)Math.random()));
    g.drawRect(x, y, altura, altura);
    x += incremento;
}
}
```

4.5.2 Contexto de execução

Em algumas situações pode ser de interesse fazer com que o applet interaja com o contexto no qual ele está executando (usualmente, o navegador Web). A interface `AppletContext` especifica algumas funcionalidades que permitem essa interação.

Para obter o contexto no qual o applet está executando, o método `getAppletContext()` de `Applet` é utilizado. Esse método retorna um objeto `AppletContext`, a partir do qual é possível obter referências a outros applets no mesmo contexto através de uma enumeração, usando o método `getApplets()`. Alternativamente, se ao applet foi atribuído um nome usando o atributo `name` na tag `APPLET`, uma referência a esse applet pode ser obtida através do método `getApplet()`.

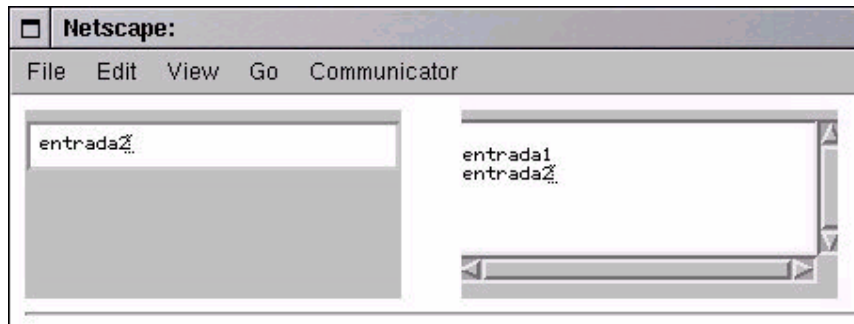
Como exemplo, considere a execução de dois applets em uma página, onde um applet tem um campo de texto para obter uma entrada do usuário e o outro tem uma área de texto para exibir as entradas que o usuário digitou na outra janela. O primeiro applet é definido em uma classe `Entrada`, enquanto que o segundo é definido em uma classe `TextAreaApplet`. A inclusão em uma página HTML dá-se através de dois elementos `APPLET`:

```
<p>
  <applet code="Entrada.class"
    width="200" height="100"
    name="entra">
  </applet>

  <applet code="TextAreaApplet.class"
    width="200" height="100"
    name="mostra">
  </applet>
</p>
```



A execução desse código deve resultar em algo similar a:



É possível também determinar que o navegador deve carregar um novo documento a partir de um applet, usando o método `showDocument()` de `AppletContext`. O argumento para esse método é um objeto localizador uniforme de recursos, do tipo `java.net.URL`.

Códigos:

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;

public class Entrada extends Applet {
    TextField tf = new TextField(30);
    TextAreaApplet ta;

    public void init() {
        add(tf);
        ta = (TextAreaApplet) getAppletContext().getApplet("mostra");
        tf.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent ae) {
                ta.append(tf.getText());
            }
        });
    }
}
// F i m
```

```
import java.awt.*;
import java.applet.*;

public class TextAreaApplet extends Applet {
    TextArea ta = new TextArea(5, 30);

    public void init() {
        add(ta);
    }
}
```

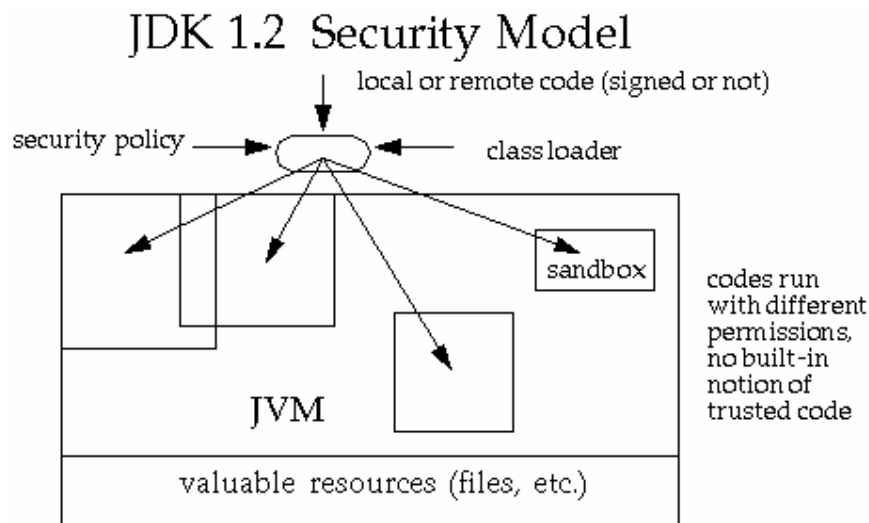
```

}

public void append(String s) {
    ta.append("\n" + s);
}
}
// F i m
  
```

4.5.3 Modelo de segurança Java

No modelo de segurança estabelecido a partir do JDK 1.2 (JDK security specification), todo código sendo carregado para uma máquina virtual Java requer o estabelecimento de uma política de segurança, visando evitar que algum objeto realize operações não-autorizadas na máquina local. Com a inclusão do conceito de política de segurança, é possível estabelecer permissões diferenciadas para as aplicações:



A política de segurança padrão é estabelecida em um arquivo do sistema, `java.policy`, localizado no diretório `<java_home>/lib/security/`. Cada usuário pode estabelecer adições a essa política através da criação de um arquivo particular de estabelecimento de política, `.java.policy`, em seu diretório home. Por exemplo, para permitir conexões soquete de qualquer máquina com origem no domínio `dca.fee.unicamp.br` a portas não-notáveis, o seguinte arquivo `.java.policy` seria criado:

```

grant {
    permission java.net.SocketPermission "*.dca.fee.unicamp.br:1024-", "ac-
cept,connect";
};
  
```



A sintaxe para o arquivo de políticas (Policy files syntax) permite estabelecer domínios de permissão com base na origem do código ou na sua assinatura. A ferramenta policytool permite criar um arquivo de políticas através de uma interface gráfica.

4.6 OUTROS MÉTODOS DA CLASSE APPLET

- `resize()`: redefine o tamanho da applet;
- `public void resize(int width,int height);`
- `getImage()`: retorna um objeto Image que pode ser exibido na tela pelo método `paint()`
- `public Image getImage(URL url);`
- `newAudioClip()`: busca um clip de áudio que pode ser executado pelo método `play()`
`public static final AudioClip newAudioClip(URL url);`

4.7 MÉTODOS URL

- `getCodeBase()`: retorna a URL base da classe `public URL getCodeBase()`
- `getDocumentBase()`: retorna a URL base do HTML `public URL getDocumentBase()`

Exemplo:

```
Image img, img2;  
img=getImage(getDocumentBase(), "tulipas.gif");  
img2=getImage(getCodeBase(), "coracao.gif");
```

4.8 MOSTRANDO IMAGENS DENTRO DE UM APPLET

Para exibir imagens dentro de um applet, utilizamos a classe abstrata Image, do pacote AWT. Podemos imaginar esta classe como um “apontador” de imagens. A classe Applet possui um método chamado `getImage` que serve para carregar imagens (nos formatos JPEG ou GIF). Os parâmetros desta função são a URL onde está localizada a imagem e o nome do arquivo da imagem.

Para desenhar a imagem, utilizamos o método `drawImage` da classe Graphics, passando como parâmetros um objeto da classe Image, além do tamanho e da largura da imagem.

Por exemplo:

```
import java.applet.Applet;  
import java.awt.*;  
public class MostrImagem extends Applet {  
    Image im;
```



```
public void init()
{
    im = getImage(getCodeBase(), "foto.jpg");
}
public void paint(Graphics g)
{
    g.drawImage(im, 10, 15, this);
}
}
```

4.9 APPLET: VANTAGENS / DESVANTAGENS

1. Desvantagens (Restrições)

- Dependência de plug-in e incompatibilidade em browsers
- Tempo de download

2. Vantagens

- Facilidade para realizar comunicação em rede
- Possibilidade de abrir janelas externas
- Capacidade de estender o browser em recursos de segurança, protocolos de rede, capacidade gráfica
- Aplicação sempre atualizada
- Capacidade de interagir com a página via JavaScript

4.10 RESTRIÇÕES DOS APPLET

Há várias coisas que aplicações comuns podem e que um applet não pode fazer:

- Não pode carregar bibliotecas ou definir métodos nativos;
- Não pode ler ou escrever arquivos na máquina cliente;
- Não pode fazer conexões de rede a não ser para a máquina de onde veio;
- Não pode iniciar a execução de nenhum programa na máquina do cliente;
- Não tem acesso à maior parte das propriedades do sistema;
- Janelas abertas sempre têm aviso de segurança.

4.11 PROGRAMAÇÃO ORIENTADA A EVENTOS

Applets, sendo programas gráficos e interativos, são direcionadas a eventos.



UNIVERSIDADE DO AMAZONAS
FACULDADE DE TECNOLOGIA
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS



Eventos correspondem à ocorrência de algum fato, o qual deve (ou não) desencadear um procedimento específico. Não há, em princípio, uma sequência para a execução de procedimentos. Procedimentos são acionados dependendo de ações do usuário.

Obs.: mais informações:

<http://java.sun.com/j2se/1.4.2/docs/api/java/applet/Applet.html>
<http://java.sun.com/products/jdk/1.4/docs/api/java/applet/package-summary.html>
<http://java.sun.com/j2se/1.4/docs/api/java/applet/AppletContext.html>
<http://java.sun.com/j2se/1.4/docs/guide/security/spec/security-spec.doc.html>
<http://java.sun.com/j2se/1.4/docs/guide/security/PolicyFiles.html#FileSyntax>
<http://java.sun.com/j2se/1.4.2/docs/api/javax/swing/JApplet.html>



APENDICE A : EXEMPLOS

3. PROGRAMAÇÃO EM JAVA

3.1 TIPOS PRIMITIVOS

3.1.1 Classes wrapper (Classes embrulho) 21

3.1.1.1 Classe Boolean 21

3.1.1.2 Classe Character 22

```
public class CharacterDemo {
    public static void main(String args[]) {
        Character a = new Character('a');
        Character a2 = new Character('a');
        Character b = new Character('b');

        int difference = a.compareTo(b);
        String resp = new String();
        if (difference == 0) {
            System.out.println("a é igual a b.");
        } else if (difference < 0) {
            { System.out.println("a é menor que b.");}
            else
                if(difference> 0) {
                    System.out.println("a é maior que b.");}
        }
        if (a.equals(a2))
            { resp = "igual a"; }
        else
            { resp = "diferente de";}
        System.out.println("a é " + resp + " a2");
        if(Character.isUpperCase(a.charValue()))
            { resp = "Maiuscula";}
        else
            { resp = "Minuscula";}
        System.out.println("O Caracter " + a.toString() + " é " + resp);
    }
}
```

3.1.1.3 Classes para manipular valores numéricos inteiros 22

3.1.1.4 Classes para manipular valores numéricos reais 23

//Exemplo dos tipo primitivos: Boolean, byte, int, short, long float e double junto com a
// classe wrappers

```
public class ValoresDosTiposPrimitivos {
```



```
public static void main(String args[]){

    //valores inteiros
    byte maiorByte = Byte.MAX_VALUE;
    byte menorByte = Byte.MIN_VALUE;
    short maiorShort = Short.MAX_VALUE;
    short menorShort = Short.MIN_VALUE;
    int maiorInt = Integer.MAX_VALUE;
    int menorInt = Integer.MIN_VALUE;
    long maiorLong = Long.MAX_VALUE;
    long menorLong = Long.MIN_VALUE;

    //valores reais
    float maiorFloat = Float.MAX_VALUE;
    float menorFloat = Float.MIN_VALUE;
    double maiorDouble = Double.MAX_VALUE;
    double menorDouble = Double.MIN_VALUE;

    //outros tipos primitivos
    char aChar = 'S';
    boolean aBoolean = true;

    //imprime todos na tela
    System.out.println("O valor de um byte deve estar entre "+menorByte+" e "+ maiorByte);
    System.out.println("O valor de um short deve estar entre "+menorShort+" e "+maiorShort);

    System.out.println("O valor de um int deve estar entre "+menorInt+" e "+maiorInt);

    System.out.println("O valor de um long deve estar entre "+menorLong+" e "+maiorLong);

    System.out.println("O valor de um float deve estar entre "+menorFloat+" e "+maiorFloat);

    System.out.println("O valor de um double deve estar entre "+menorDouble+" e "+maiorDouble);

    System.out.println("O valor da variavel aBoolean e: "+aBoolean+" ou "+!aBoolean);

    if(Character.isUpperCase(aChar))

    { System.out.println("O caracter "+aChar+" esta em Maiúsculo ");}

    else
```



```
{  
    System.out.println("O caracter "+aChar+" esta em Minúsculo ");  
}  
}  
}
```

3.1.1.6 Classe String 23

```
class StringDemo {  
    public static void main(String args[]) {  
        String strOb1 = "First String";  
        String strOb2 = "Second String";  
        String strOb3 = strOb1 + " and " + strOb2;  
  
        System.out.println(strOb1);  
        System.out.println(strOb2);  
        System.out.println(strOb3);  
    }  
}
```

3.2 ESCOPO25

3.3 OPERADORES EM JAVA 26

3.3.1 Operadores Básicos 26

```
class BasicMath {  
    public static void main(String args[]) {  
        int a = 1 + 1;  
        int b = a * 3;  
        int c = b / 4;  
        int d = b - a;  
        int e = -d;  
        System.out.println("a = " + a);  
        System.out.println("b = " + b);  
        System.out.println("c = " + c);  
        System.out.println("d = " + d);  
        System.out.println("e = " + e);  
    }  
}
```

3.3.2 Operadores Unários 26

```
class IncDec {  
    public static void main(String args[]) {  
        int a = 1;  
        int b = 2;
```



```
        int c = ++b;
        int d = a++;
        c++;
        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("c = " + c);
        System.out.println("d = " + d);
    }
}
```

3.3.3 Operadores de Atribuição 27

```
class OpEquals {
    public static void main(String args[]) {
        int a = 1;
        int b = 2;
        int c = 3;
        a += 5;
        b *= 4;
        c += a * b;
        c %= 6;
        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("c = " + c);
    }
}
```

3.3.4 Operadores Bit-a-Bit 27

```
class OpBitEquals {
    public static void main(String args[]) {
        int a = 1;
        int b = 2;
        int c = 3;
        a |= 4;
        b >>= 1;
        c <<= 1;
        a ^= c;
        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("c = " + c);
    }
}
```

3.3.5 Operadores relacionais e condicionais 28

```
class BoolLogic {
```



```
public static void main(String args[]) {  
    boolean a = true;  
    boolean b = false;  
    boolean c = a | b;  
    boolean d = a & b;  
    boolean e = a ^ b;  
    boolean f = (!a & b) | (a & !b);  
    boolean g = !a;  
    System.out.println("    a = " + a);  
    System.out.println("    b = " + b);  
    System.out.println("    a|b = " + c);  
    System.out.println("    a&b = " + d);  
    System.out.println("    a^b = " + e);  
    System.out.println("    !a&b|a&!b = " + f);  
    System.out.println("    !a = " + g);  
}  
}
```

3.3.6 Operador de Type Cast 28

```
class Conversion {  
    public static void main(String args[]) {  
        byte b;  
        int i = 257;  
        double d = 323.142;  
  
        System.out.println("\nConversion of int to byte.");  
        b = (byte) i;  
        System.out.println("i and b " + i + " " + b);  
  
        System.out.println("\nConversion of double to int.");  
        i = (int) d;  
        System.out.println("d and i " + d + " " + i);  
  
        System.out.println("\nConversion of double to byte.");  
        b = (byte) d;  
        System.out.println("d and b " + d + " " + b);  
    }  
}
```

3.4 ESTRUTURAS DE CONTROLE EM JAVA 29

3.4.1 If (ou Se) em Java 29

```
class IfElse {  
    public static void main(String args[]) {  
        int month = 4;  
        String season;
```



```
    if (month == 12 || month == 1 || month == 2) {  
        season = "Winter";  
    } else if (month == 3 || month == 4 || month == 5) {  
        season = "Spring";  
    } else if (month == 6 || month == 7 || month == 8) {  
        season = "Summer";  
    } else if (month == 9 || month == 10 || month == 11) {  
        season = "Autumn";  
    } else {  
        season = "Bogus Month";  
    }  
  
    System.out.println("April is in the " + season + ".");  
}  
}
```

3.4.2 O comando Switch 29

```
class SwitchSeason {  
    public static void main(String args[]) {  
        int month = 4;  
        String season;  
        switch (month) {  
            case 12:  
            case 1:  
            case 2:  
                season = "Winter";  
                break;  
            case 3:  
            case 4:  
            case 5:  
                season = "Spring";  
                break;  
            case 6:  
            case 7:  
            case 8:  
                season = "Summer";  
                break;  
            case 9:  
            case 10:  
            case 11:  
                season = "Autumn";  
                break;  
            default:  
                season = "Bogus Month";  
        }  
    }  
}
```



```
    }  
    System.out.println("April is in the " + season + ".");  
}  
}
```

3.5 LAÇOS DE REPETIÇÃO EM JAVA 30

3.5.1 Laço For 30

```
class ForDemo {  
    public static void main(String args[]) {  
        for (int i = 1; i <= 10; i++)  
            System.out.println("i = " + i);  
    }  
}
```

```
class ForTick {  
    public static void main(String args[]) {  
        for (int n = 10; n > 0; n--)  
            System.out.println("tick " + n);  
    }  
}
```

```
class ForVar {  
    public static void main(String args[]) {  
        int i;  
        boolean done = false;  
  
        i = 0;  
        for( ; !done; ) {  
            System.out.println("i is " + i);  
            if(i == 10) done = true;  
            i++;  
        }  
    }  
}
```

3.5.2 Laço While 30

```
class WhileDemo {  
    public static void main(String args[]) {  
        int n = 10;  
        while (--n >= 0)  
            System.out.println("tick " + (n + 1));  
    }  
}
```




3.5.3 Laço Do ... While 30

```
class DoWhile {  
    public static void main(String args[]) {  
        int n = 10;  
        do  
            System.out.println("tick " + n);  
        while (--n > 0);  
    }  
}
```

3.5.4 Comandos de salto 31

3.5.4.1 O Comando Break 31

3.5.4.2 O Comando Continue 32

```
class Continue {  
    public static void main(String args[]) {  
        for (int i = 0; i < 10; i++) {  
            System.out.print(i + " ");  
            if (i % 2 == 0)  
                continue;  
            System.out.println("");  
        }  
    }  
}
```

3.5.4.3 O comando Return 32

```
class ReturnDemo {  
    public static void main(String args[]) {  
        boolean t = true;  
        System.out.println("Before the return");  
        if (t)  
            return;  
        System.out.println("This won't execute");  
    }  
}
```

3.6 COMENTÁRIOS 32

```
/**  
 * Exemplo de um Comentário para documentação!  
 */
```

```
// Comentário de Linha
```



3.7 CLASSES EM JAVA	33
3.7.1 Classes	33
3.7.2 DEFINIÇÃO DE CLASSES EM JAVA	33
3.7.2.1 Palavra Reservada public	35
3.7.2.2 Palavra Reservada protected	35
3.7.2.3 Palavra Reservada private	35
3.7.2.4 Classes abstratas e finais	36
3.7.3 Atributos	38
3.7.3.1 Membros de Classe static	38
3.7.4 Métodos	39
3.7.6 Construtores	40
3.7.7 Interface	41
3.7.7.1 Declaração de interface	41
3.7.7.2 Definição de constantes	42
3.7.7.3 Implementação de interfaces	42
3.7.8 O MÉTODO MAIN	43
3.7.8.1 Valor de saída do main	43
3.7.8.2 Argumento do main	44
3.7.8.3 Assinatura do método main	44
3.7.9 Controlando o nível de acesso dos membros das classes	45
3.8 OBJETOS EM JAVA	45
3.8.1 Propriedades de objetos	46
3.8.2 Como criar um objeto em Java	47
3.8.3 Referência para o objeto	47
3.8.4 Operador this	48
3.8.5 Manipulação de objetos	49
3.9 VETORES EM JAVA	51
3.9.1 Criação de Vetores em Java	51
3.9.1.1 Copiando vetores	52
3.9.2 Matrizes em Java	53
3.9.2.1 Criação de Matrizes	53
3.9.3 Vetores e Matrizes de Objetos em Java	54
3.9.4 Passando Vetores para métodos	54
3.10 HERANÇA	55
3.10.1 Construção de objetos derivados	57
3.10.2 A palavra-chave super	57
3.10.3 Restrições de acesso	58
3.10.4 Herança e interfaces	60
3.10.5 Interfaces	61
3.10.5.1 Definindo Interfaces	61
3.10.5.2 Implementado interfaces	62
3.11 POLIMORFISMO EM JAVA	63
3.11.1 Sobrescrevendo métodos	67
3.11.2 Sobrecarregando métodos	68
3.12 PACOTES	69
3.12.1 Pacotes e Diretórios	70
3.12.2 Ocultando Informações	70



3.12.3 Importação de Pacotes	71
3.12.4 Nomeando Pacotes e Unidades de Compilação	72
3.13 EXCEÇÕES EM JAVA	73
3.13.1 Capturando e tratando exceções	74
3.13.2 Propagando exceções	76
3.13.3 A hierarquia de exceções	78
3.13.4 Capturando várias exceções	79
3.13.5 Gerando exceções	80
3.13.6 Definição de novas exceções	80
3.13.7 Repassando a exceção	81
3.14 ENTRADA E SAÍDA EM JAVA (I/O)	82
3.14.1 Streams	84
3.14.2 Definição de Arquivos	85
3.14.3 O que são fluxos de dados?	86
3.14.4 Entrada e saída de dados binários	86
3.14.4.1 Entrada binária: InputStream	86

```
import java.io.*;
public class Eco3 {
    public static void main(String[] args) {
        BufferedReader inReader;
        inReader = new BufferedReader(new InputStreamReader(System.in));
        String line;
        int[] vet = new int[11];
        int soma = 0;
        try {
            //while((line = inReader.readLine())!= null) {
            for(int cont=0;cont<11;cont++)
            {
                System.out.print("Entre com o "+cont+"º numero:");
                line = inReader.readLine();
                vet[cont] = Integer.parseInt(line);
            }
            System.out.println(soma);
            inReader.close();
        } catch (IOException e) {
            System.err.println(e.getMessage());
        }
        boolean ordenado;
        int c,s;

        do
        {
            ordenado = true;
            for(c=0;c<10;c++)
            {
                if(vet[c]>vet[c+1])
                {
```



```
s = vet[c];
vet[c] = vet[c+1];
vet[c+1]=s;
ordenado = false;
}
}

} while(!ordenado);
for(int cont=0;cont<11;cont++)
{
    System.out.print("Posição["+cont+"] = "+vet[cont]);
}
}

}

3.14.4.2 Saída de dados binários: OutputStream      87
3.14.5 Entrada e Saída de caracteres      87
3.14.5.1 Entrada de caracteres: Reader 87
3.14.5.2 Saída de caracteres: Writer      87
3.14.6 Transferência de objetos: Serialização  88
3.14.7 Buffers      89
3.14.8 Filtros 90
3.14.9 Comandos de Java para Entrada e Saída      91
3.15 MULTIPROCESSAMENTO EM JAVA (THREADS)      93
3.15.1 Estados de uma Thread      94
3.15.2 Prioridades das threads      95
3.15.3 Sincronização      95
3.15.4 Mensagens      96
3.15.5 A classe Thread e a interface Runnable 96
3.15.6 A thread principal      98
3.15.7 Criando uma thread 100
3.15.7.1 Implementando Runnable      100
3.15.7.3 Derivando de Thread      103
3.15.7.4 Escolhendo o enfoque certo      104
3.15.7.5 Criando várias threads      105
3.15.7.6 Os métodos isAlive() e join()      106
3.15.7.7 Os métodos suspend() e resume()      109
3.15.8 Definindo a prioridade das threads      112
3.15.9 Detalhes sobre a sincronização      115
3.15.9.1 Usando métodos sincronizados 116
3.15.9.2 Outro uso de synchronized      119
3.15.10 Comunicação inter-threads      122
3.15.11 Deadlock      127
3.15.12 Usando multithreading      129
3.16 DESENVOLVIMENTO DE APLICAÇÕES GRÁFICAS EM JAVA      130
3.16.1 Saída na forma Gráfica      131
3.16.1.1 Componentes gráficos      135
```



3.16.1.1.1 Containers	136
3.16.1.1.1.1 Gerenciadores de Layout	136
3.16.1.1.1.1.1 BorderLayout	137
3.16.1.1.1.2 Windows (Janelas)	138
3.16.1.1.1.2.1 Frames	139
3.16.1.1.1.2.2 Janelas de diálogo	140
3.16.1.1.1.2.3 Manipuladores de eventos de janela	144
3.16.1.1.2 Componentes de interfaces gráficas com usuários	145
3.16.1.1.2.1 Exibir informação de orientação ao usuário:	145
3.16.1.1.2.1.1 Rótulos(Label)	145
3.16.1.1.2.1.2 Dicas sobre componentes (tooltip)	146
3.16.1.1.2.1.4 Janelas de opções e mensagens	147
3.16.1.1.2.2 Disparar ações selecionadas pelo usuário:	148
3.16.1.1.2.2.1 Botões	148
3.16.1.1.2.2.2 Menus	148
3.16.1.1.2.3 Obter entradas ou escolhas do usuário:	149
3.16.1.1.2.3.1 Entradas de Texto e Senhas (Campos de texto)	149
3.16.1.1.2.3.2 Componentes para seleção	149
3.16.1.1.2.3.3 Lista de Opções (ou Listas de seleção)	150
3.16.1.1.2.3.4 Janelas para Diálogo para seleção de arquivo (ou Seleção de Arquivos)	151
3.16.1.1.2.3.5 Caixa de Diálogo para seleção de cores	152
3.16.1.1.2.3.6 Caixa de Janelas de opções e mensagens	152
3.16.1.1.3 Eventos	154
3.16.1.1.3.1 Eventos relacionados ao mouse	155
3.16.1.1.3.1.1 Manipuladores de eventos do mouse	156
3.16.1.1.3.2 Eventos de ação	159
3.16.1.1.3.3 Eventos de seleção de itens	159
3.16.1.1.3.3.1 Caixas de seleção	160
3.16.1.1.3.3.2 Seleção em drop-down	163
3.16.1.1.3.3.3 Listas	165
3.16.1.1.3.4 Eventos de modificação de texto	168
3.16.1.1.3.4.1 Componentes de texto	168
3.16.1.1.3.4.1.1 Campos de texto	169
3.16.1.1.3.4.1.2 Áreas de texto	171
3.16.1.1.3.4.1. Manipuladores de eventos de modificação de texto	171
3.16.1.1.3.5 Eventos de teclado	172
3.16.1.1.3.5.1 Manipuladores de eventos de teclado	172
3.16.1.1.3.6 Manipuladores de eventos de ação	174
3.16.1.1.3.6.1 Eventos de ação	174
3.16.2 Interfaces gráficas com usuários	175
3.16.2.1 Aplicações autônomas	176
4 APPLETS EM JAVA	177
4.1 A ESTRUTURA BÁSICA	177
4.2 DECLARAÇÃO DA CLASSE APPLET	179
4.3 DECLARAÇÃO DE VARIÁVEIS	179
4.4 CRIAÇÃO DE APPLET	182



UNIVERSIDADE DO AMAZONAS
FACULDADE DE TECNOLOGIA
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS



4.5 EXECUÇÃO DE APPLETS	182
4.5.1 Passagem de argumentos	183
4.5.2 Contexto de execução	185
4.5.3 Modelo de segurança Java	187
4.6 OUTROS MÉTODOS DA CLASSE APPLET	188
4.7 MÉTODOS URL	188
4.8 MOSTRANDO IMAGENS DENTRO DE UM APPLET	189
4.9 APPLETS: VANTAGENS /DESVANTAGENS	189
4.10 RESTRIÇÕES DOS APPLETS	190
4.11 PROGRAMAÇÃO ORIENTADA A EVENTOS	190



UNIVERSIDADE DO AMAZONAS
FACULDADE DE TECNOLOGIA
LABORATÓRIO DE PROCESSAMENTO DIGITAL DE SINAIS



Palavras reservadas em Java

As palavras a seguir são de uso reservado em Java e não podem ser utilizadas como nomes de identificadores:

abstract	double	int	static
boolean	else	interface	super
break	extends	long	switch
byte	final	native	synchronized
case	finally	new	this
catch	float	null	throw
char	for	package	throws
class	goto	private	transient
const	if	protected	try
continue	implements	public	void
default	import	return	volatile
do	instanceof	short	while

<http://www.dca.fee.unicamp.br/cursos/PooJava/sintaxe/expatrib.html>

<http://www.dca.fee.unicamp.br/cursos/PooJava/polimorf/polijex.html>

<http://www.inf.unisinos.br/~marcelow/ensino/grad/lab2/filadeque.html>

Site: <http://www.di.ufpe.br/~java/graduacao961/aulas/aula9/pacotes.html>

<http://br.sun.com/>